

Laporan M2 Tugas Besar 1

IF2010 Pemrograman Berorientasi Objek 2025/2026

Nimonspoli

Kode Kelompok : GDT

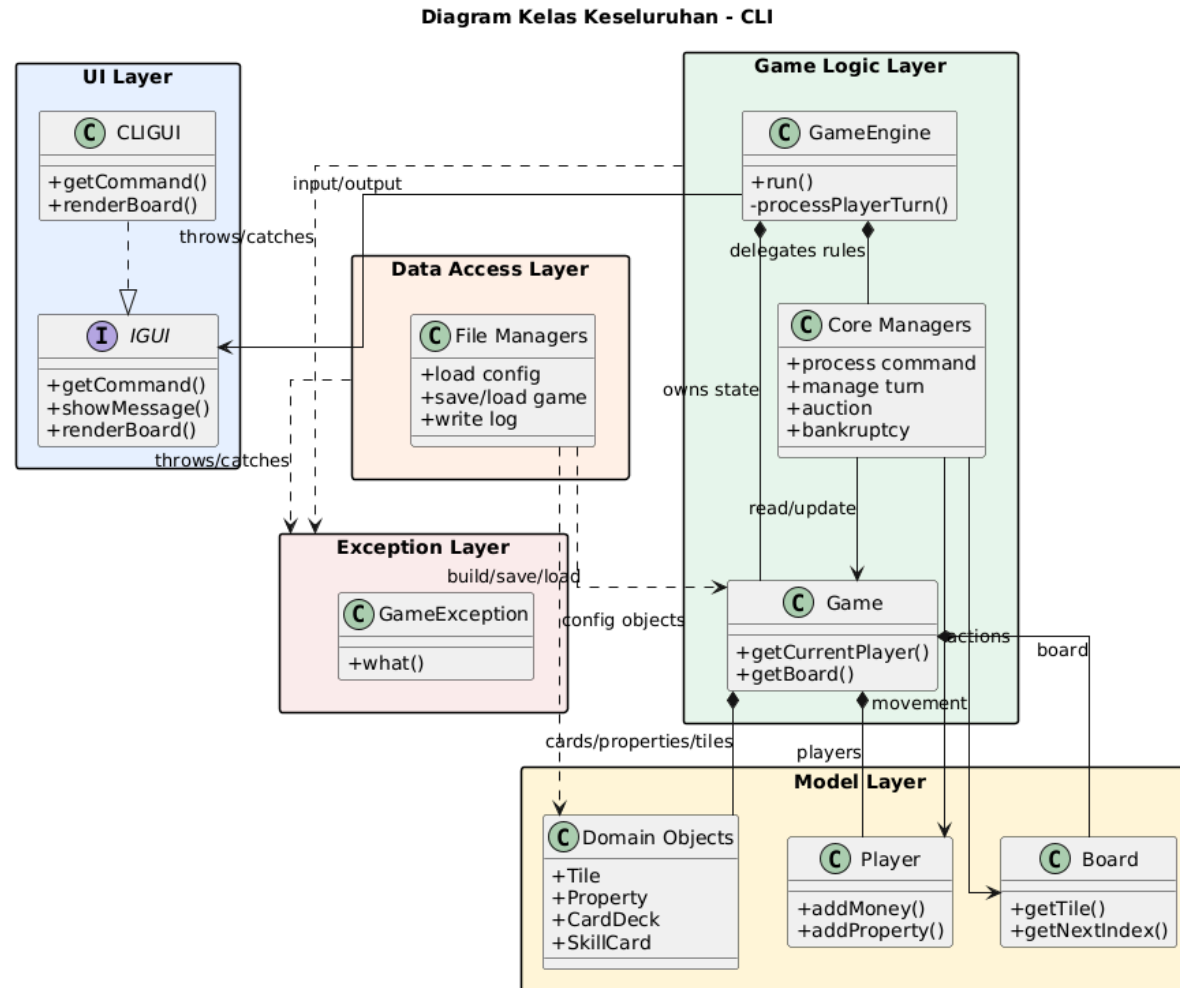
Nama Kelompok : Go Directly to Tubes

1. 13524015 / Mahatma Brahmana
2. 13524053 / Muhammad Haris Putra Sulastianto
3. 13524057 / Benedict Darrel Setiawan
4. 13524059 / Raymond Jonathan Dwi Putra J
5. 13524111 / Reynard Anderson Wijaya



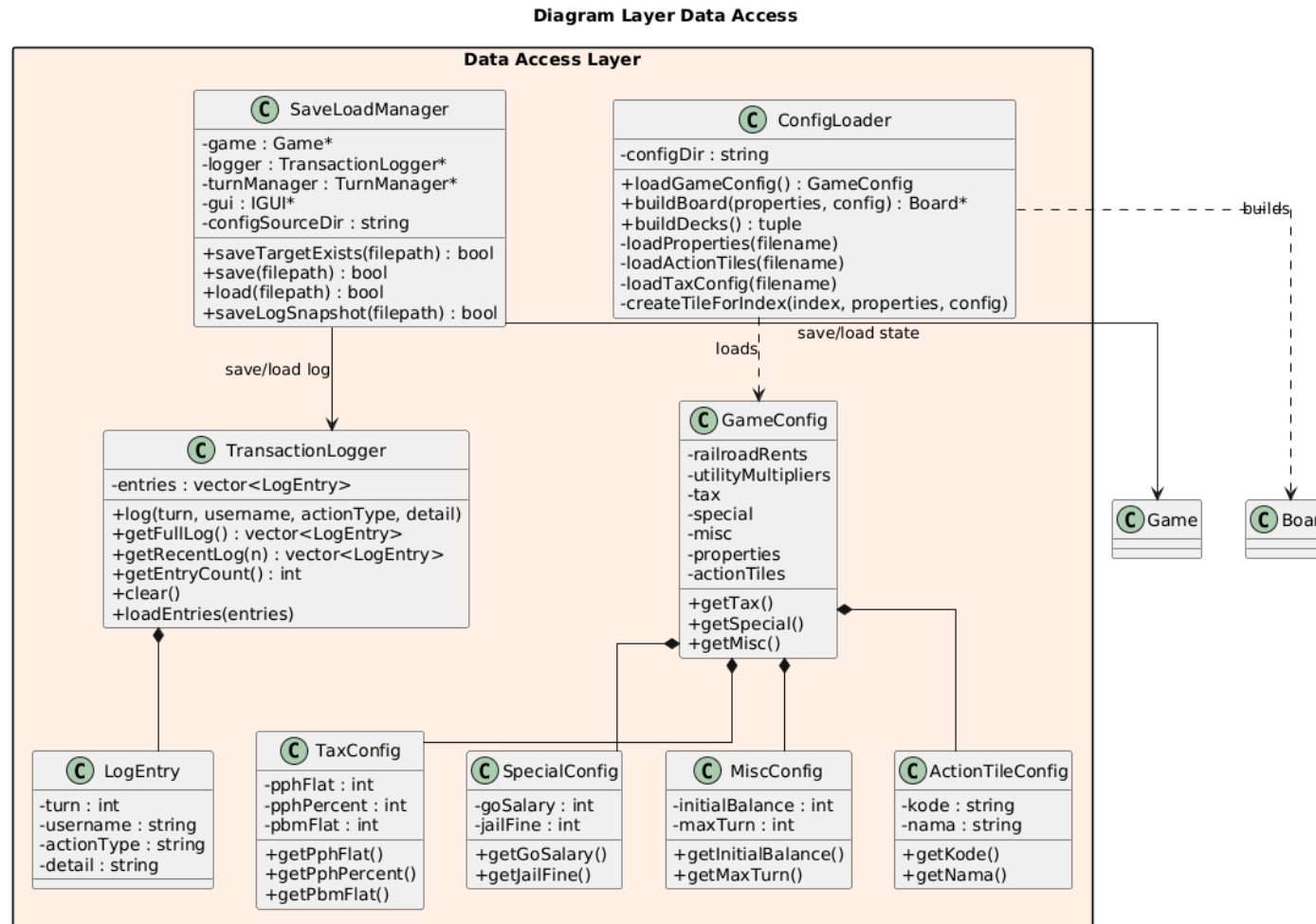
GDT - Go Directly to Tubes

1. Diagram Kelas Keseluruhan

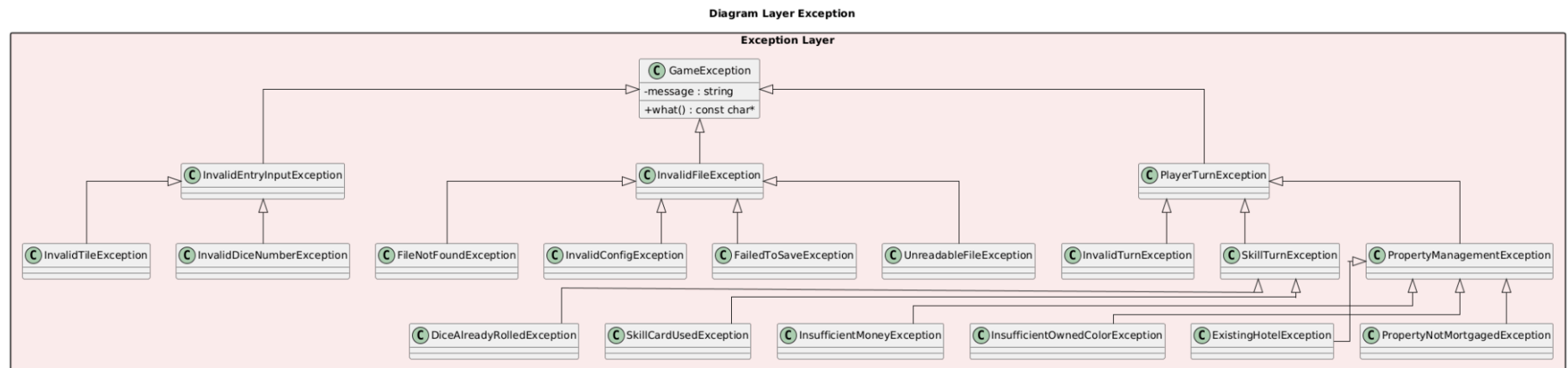


1.1. Diagram Kelas pada Masing-Masing layer

1.1.1. Diagram Kelas pada Layer Data Access

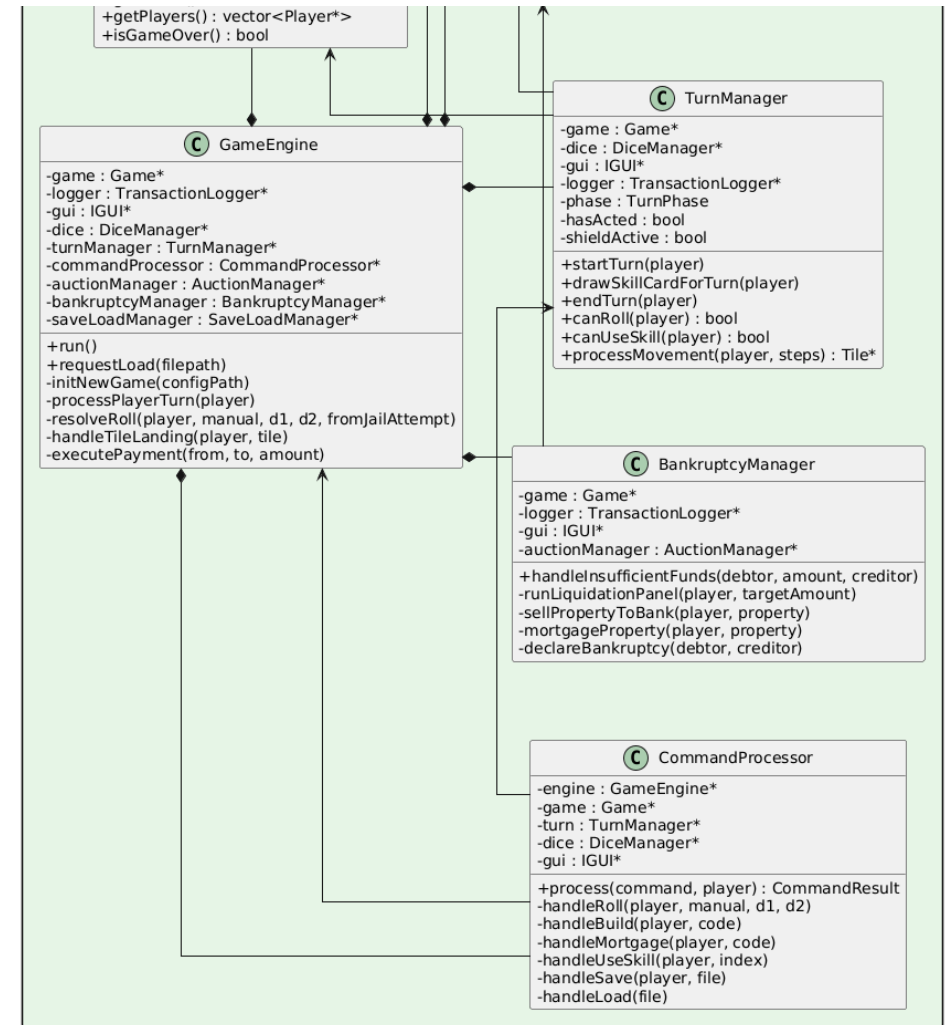
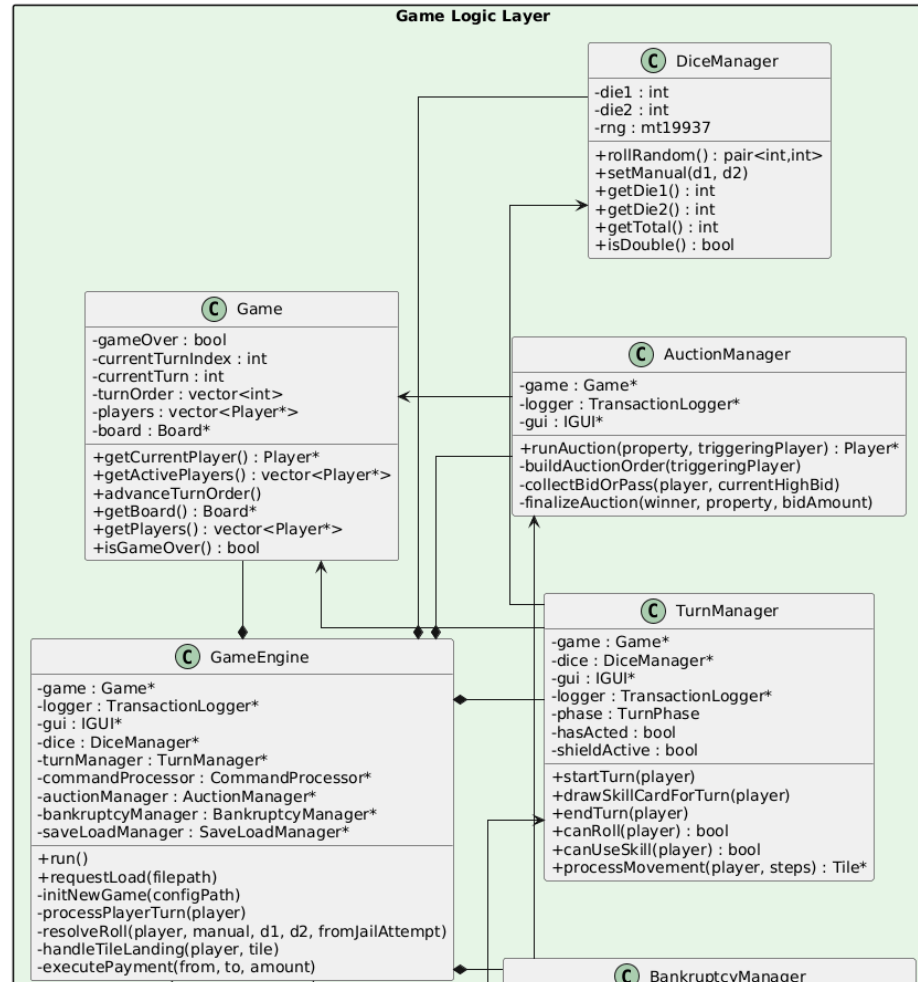


1.1.2. Diagram Kelas pada Layer Exception

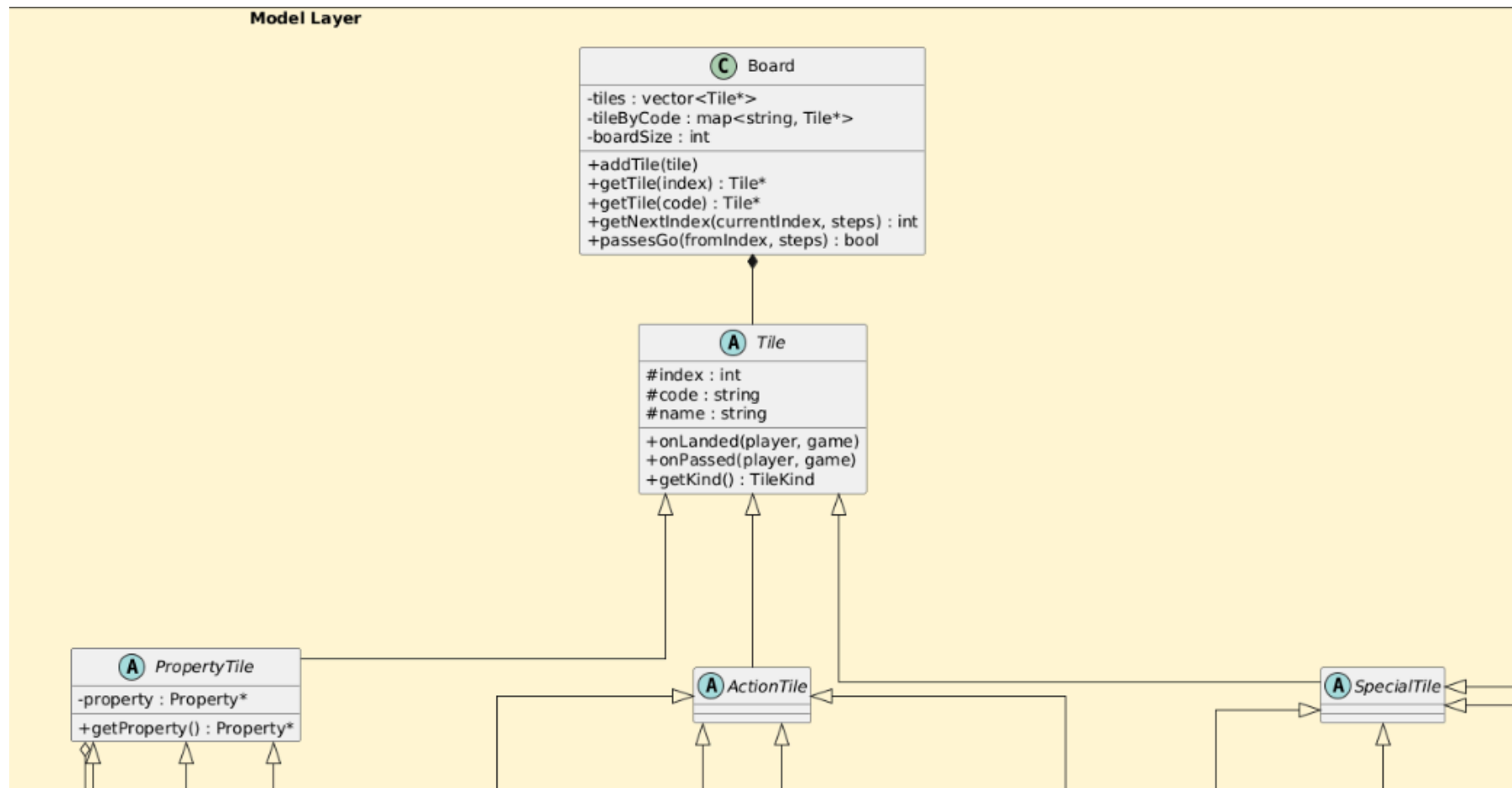


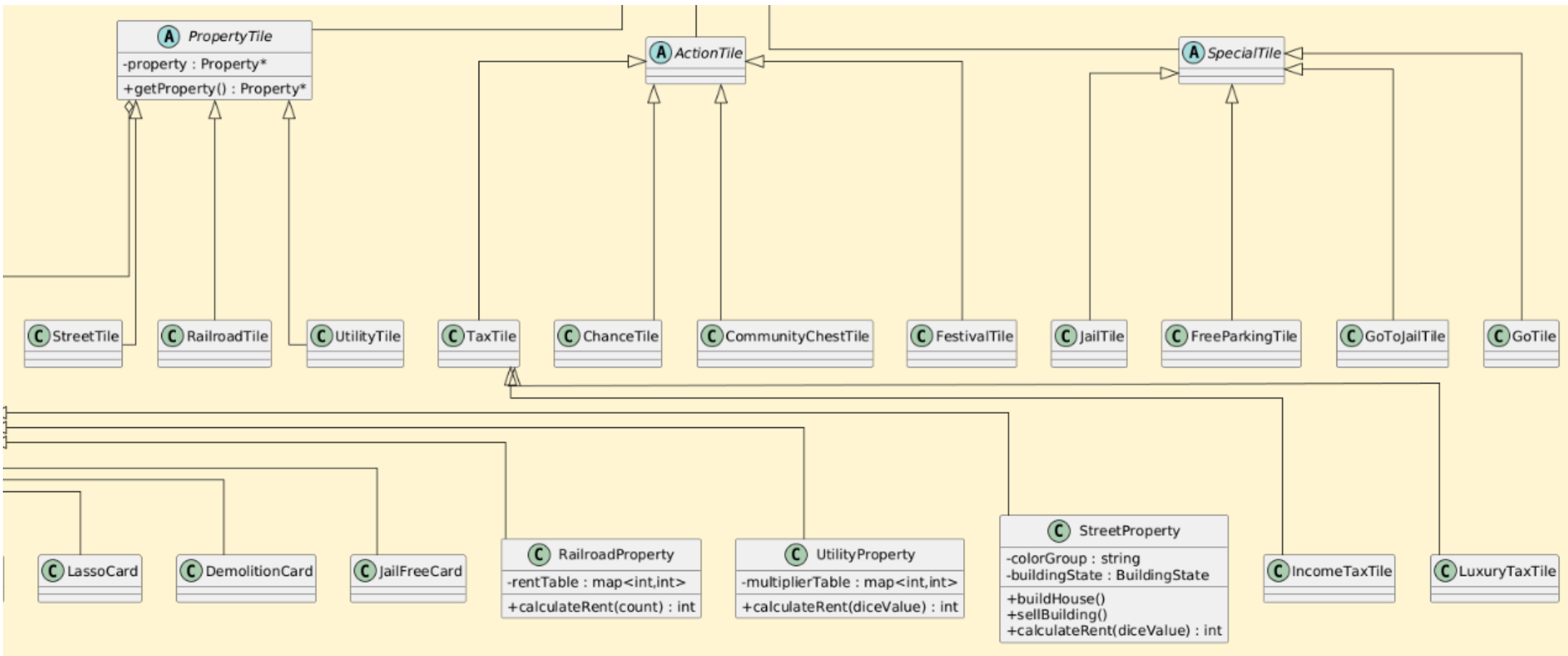
1.1.3. Diagram Kelas pada Layer Game Logic

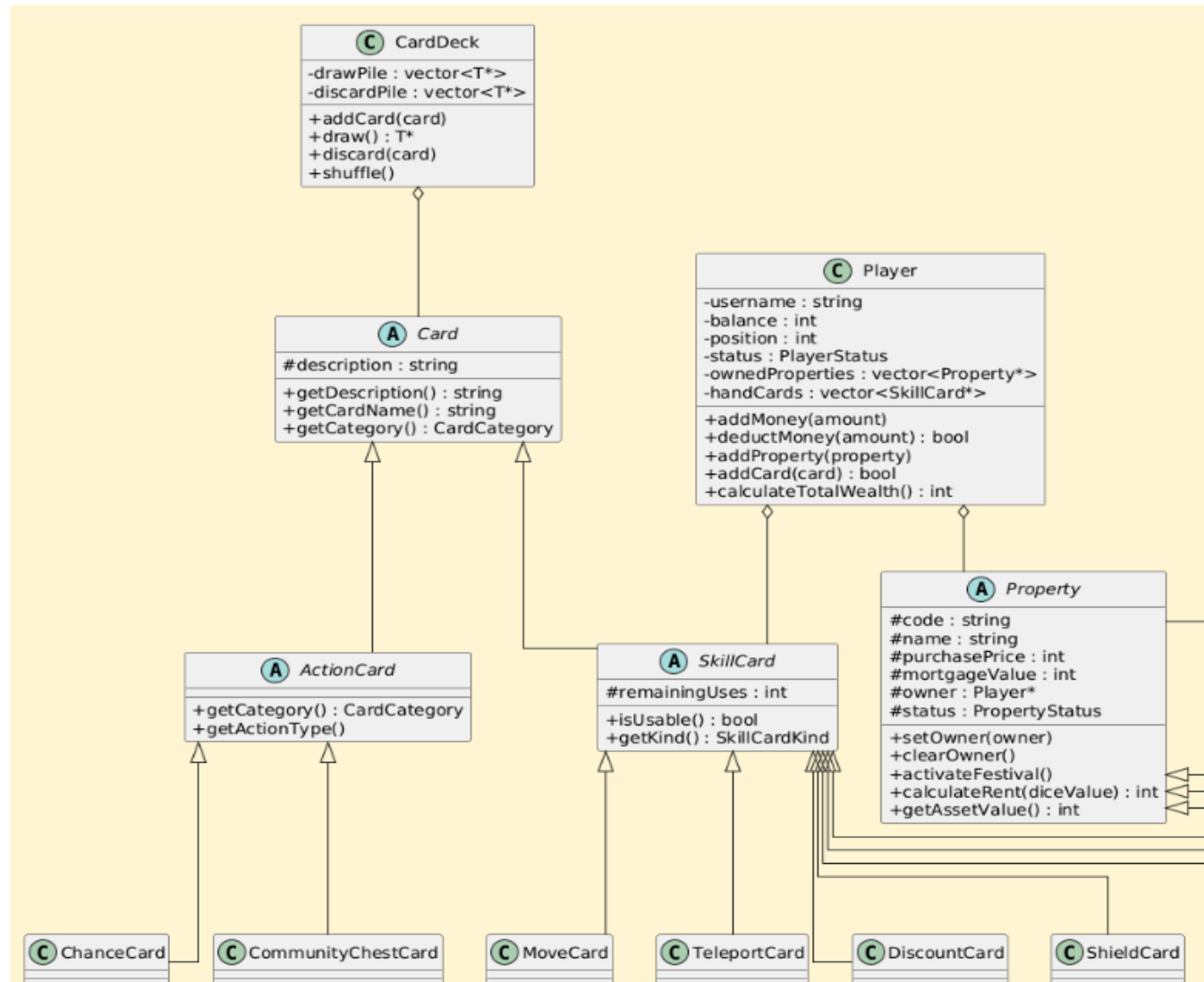
Diagram Layer Game Logic



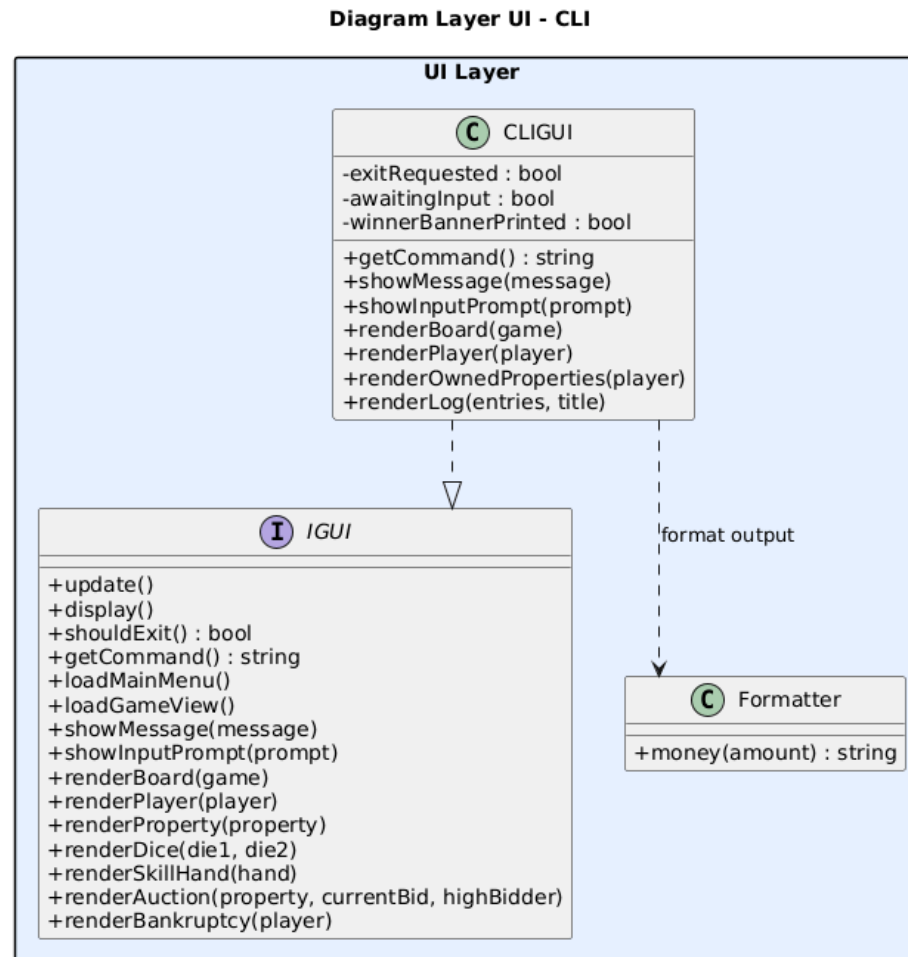
1.1.4. Diagram Kelas pada Layer Model







1.1.5. Diagram Kelas pada Layer UI



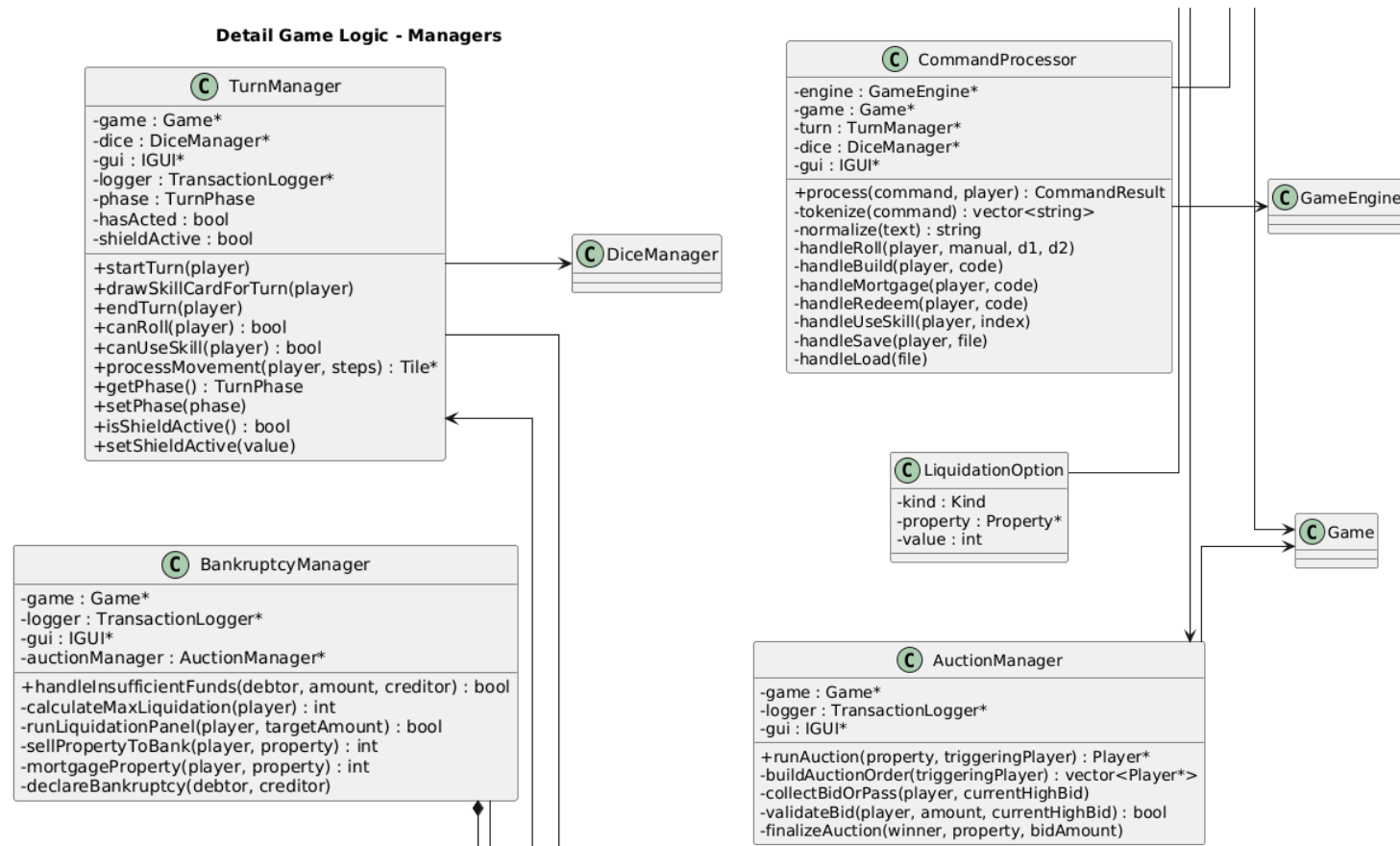
1.2. Detail Diagram Kelas pada Masing-Masing layer

1.2.1. Detail Diagram Kelas Core - Engine Game

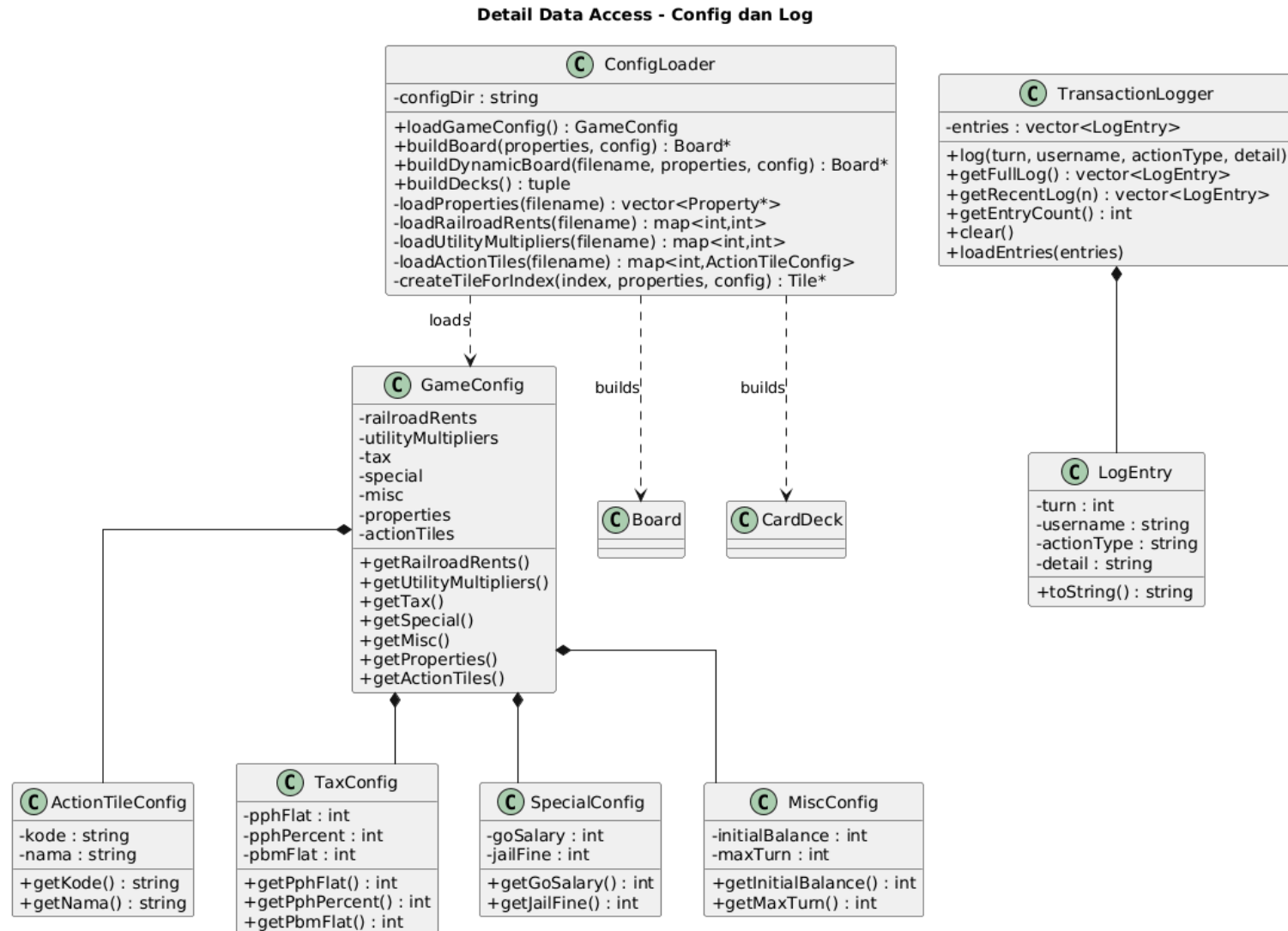


Gambar 1 Diagram Kelas Keseluruhan

1.2.2. Detail Diagram Kelas Core - Managers

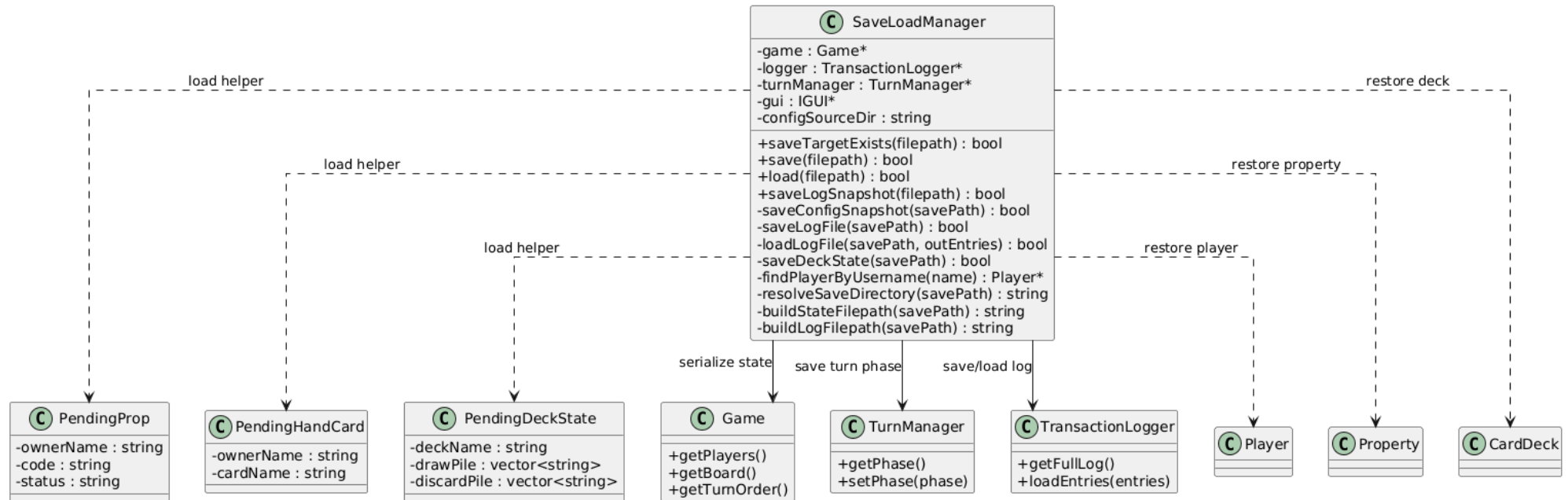


1.2.3. Detail Diagram Kelas Data - Config Log

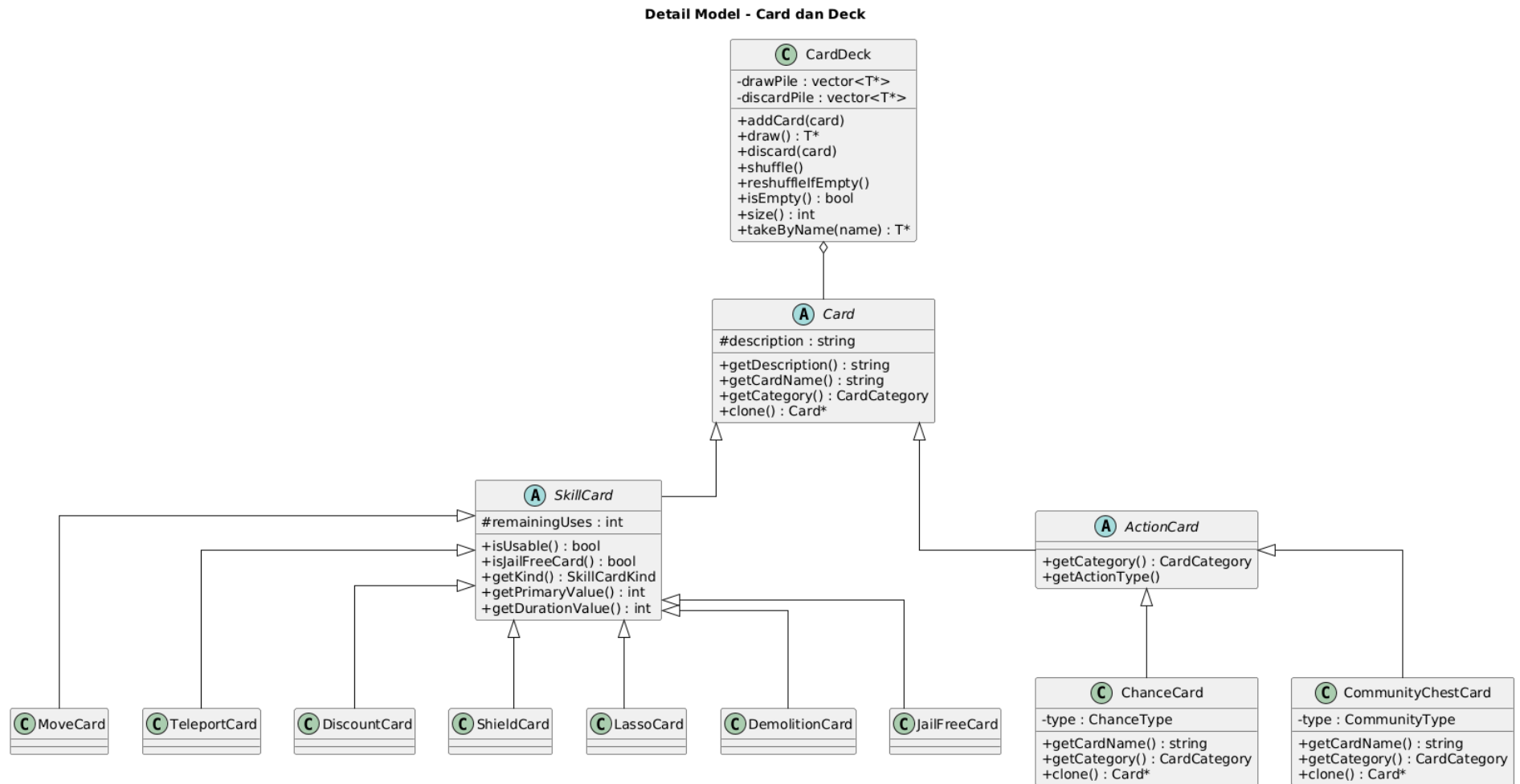


1.2.4. Detail Diagram Kelas Data - Save Load

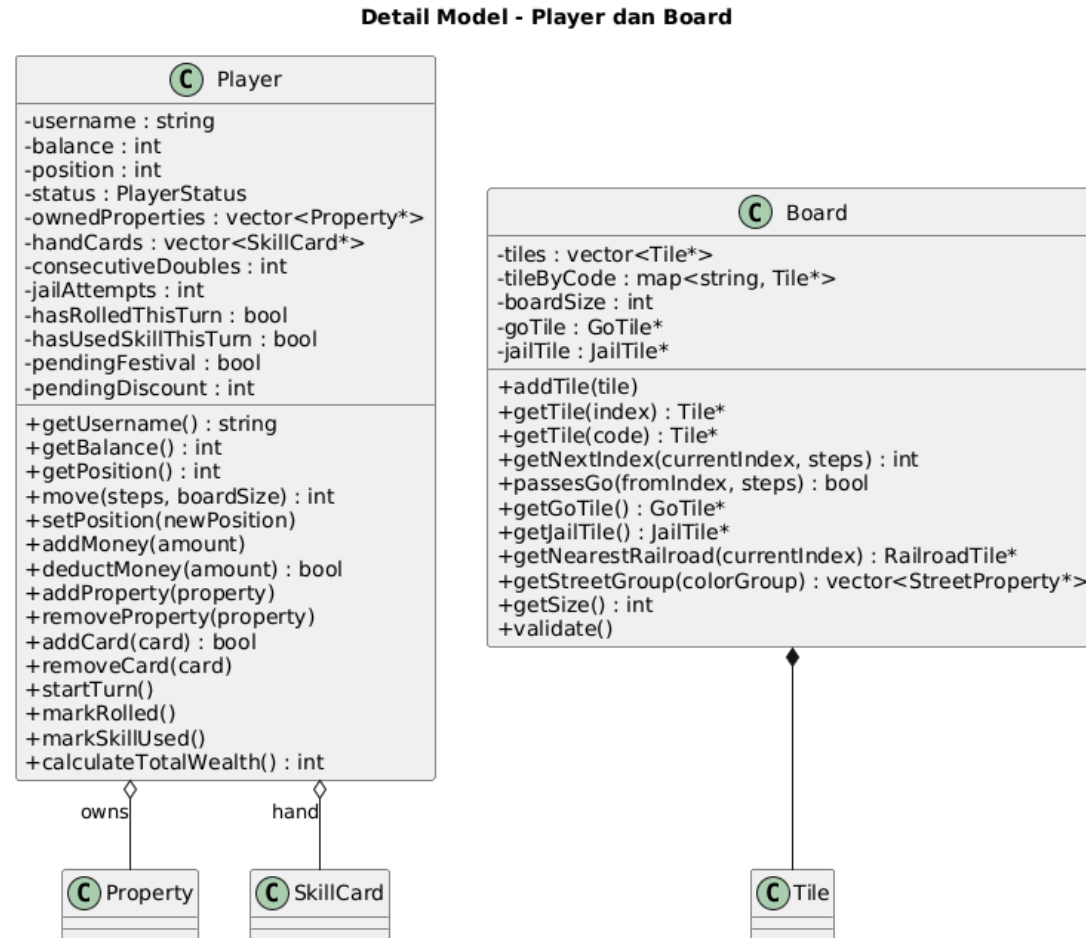
Detail Data Access - Save Load



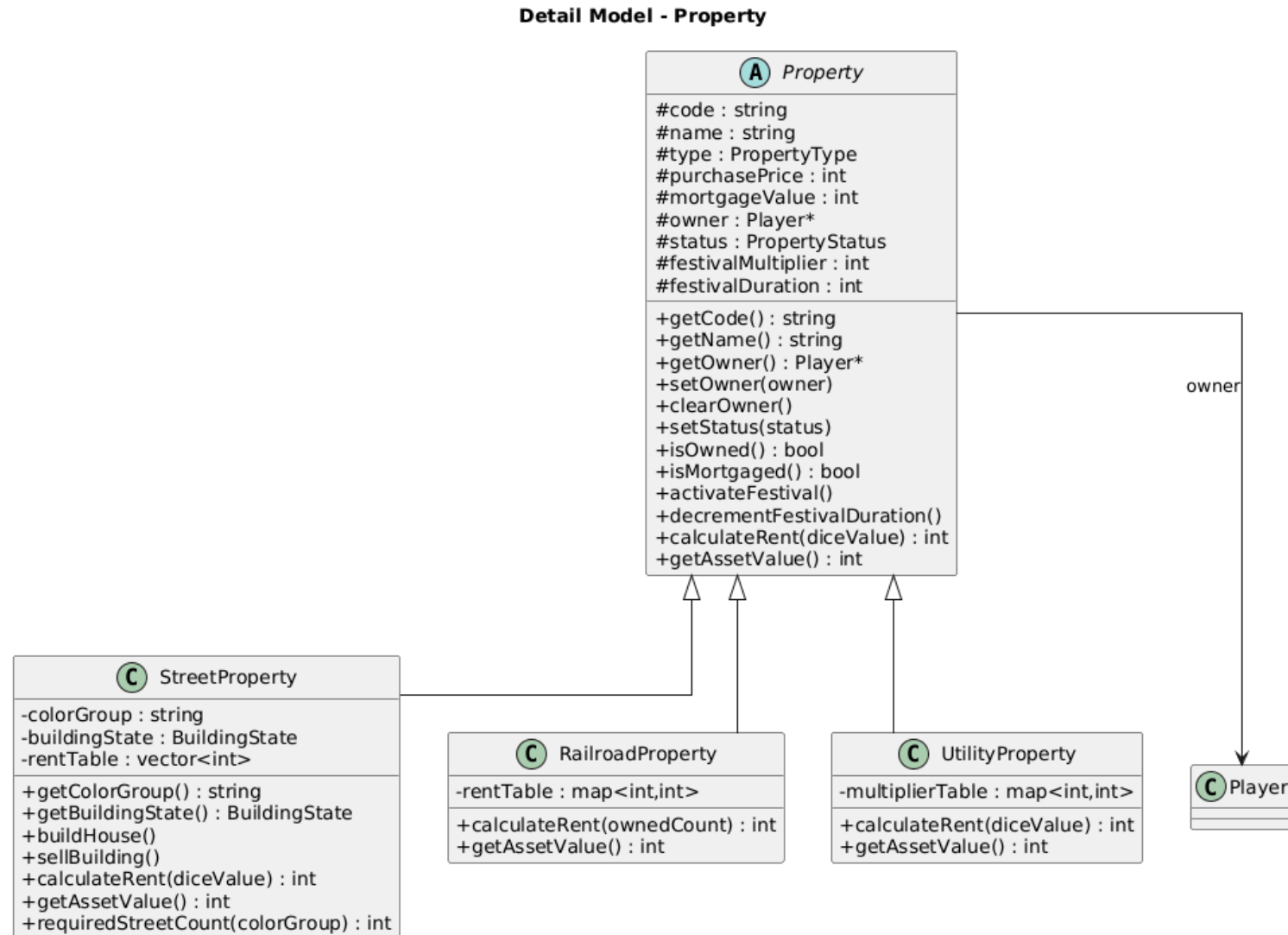
1.2.5. Detail Diagram Kelas Model - Card



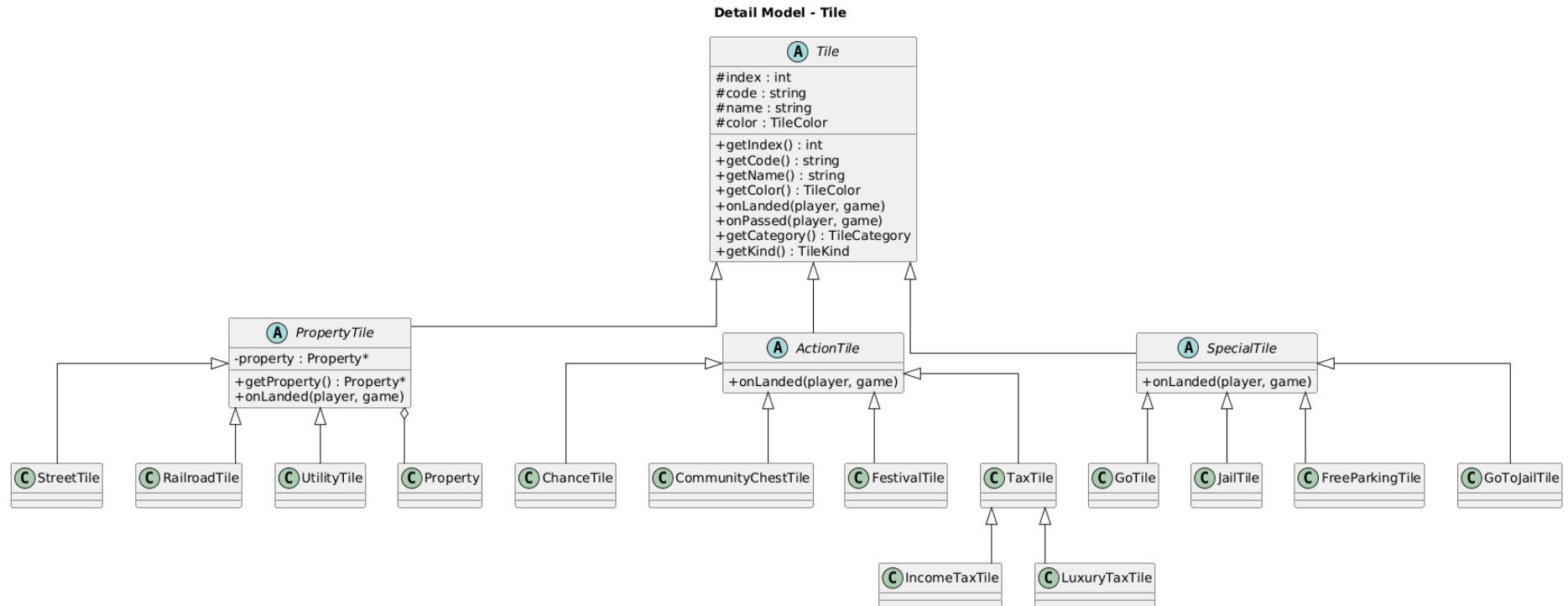
1.2.6. Detail Diagram Kelas Model - Player Board



1.2.7. Detail Diagram Kelas Model - Property



1.2.8. Detail Diagram Kelas Model - Tile



1.2.9. Detail Diagram Kelas UI - CLI



1.3. Diagram Kelas pada Masing-Masing Kelas

1.3.1. Diagram Kelas Core - AuctionAction & AuctionManager

Diagram Kelas - AuctionAction

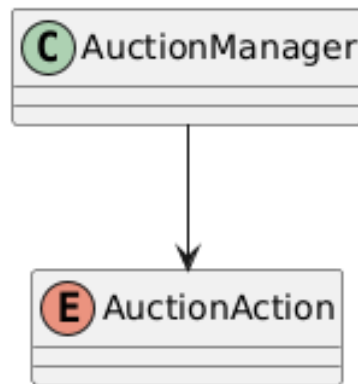
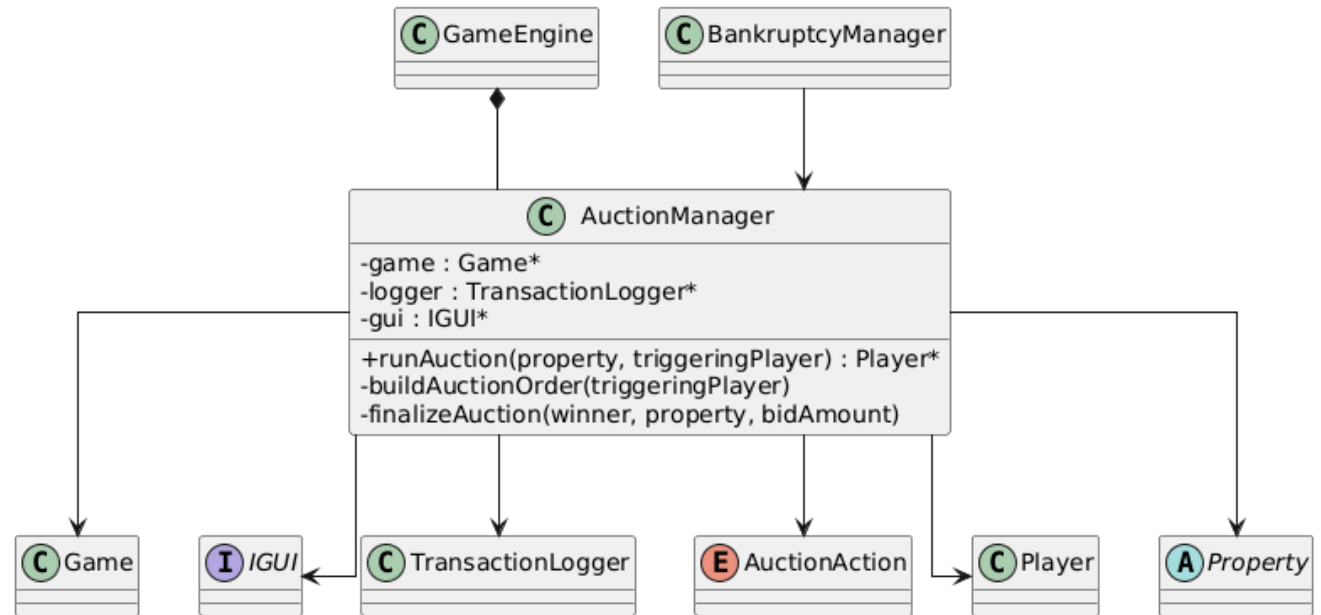
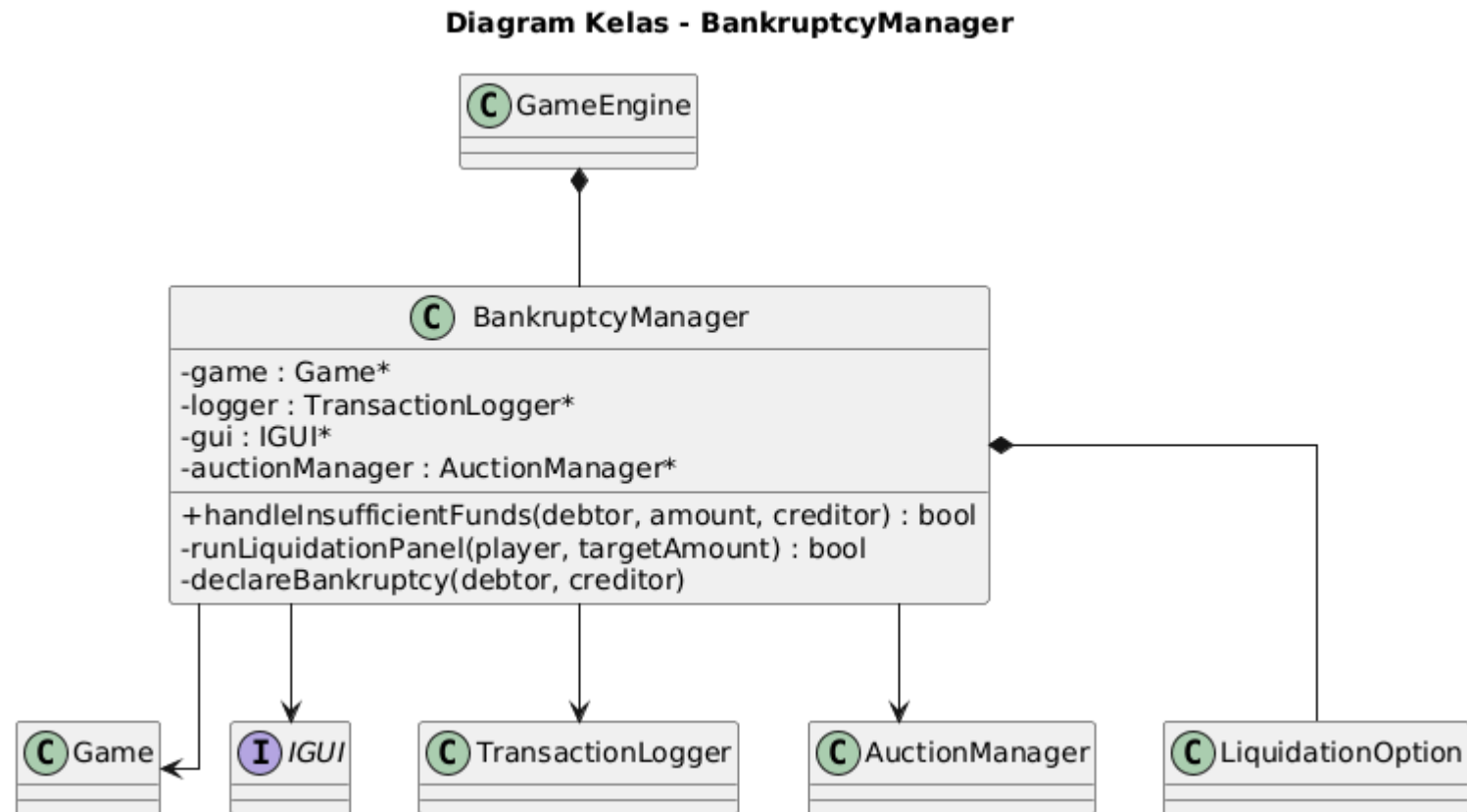


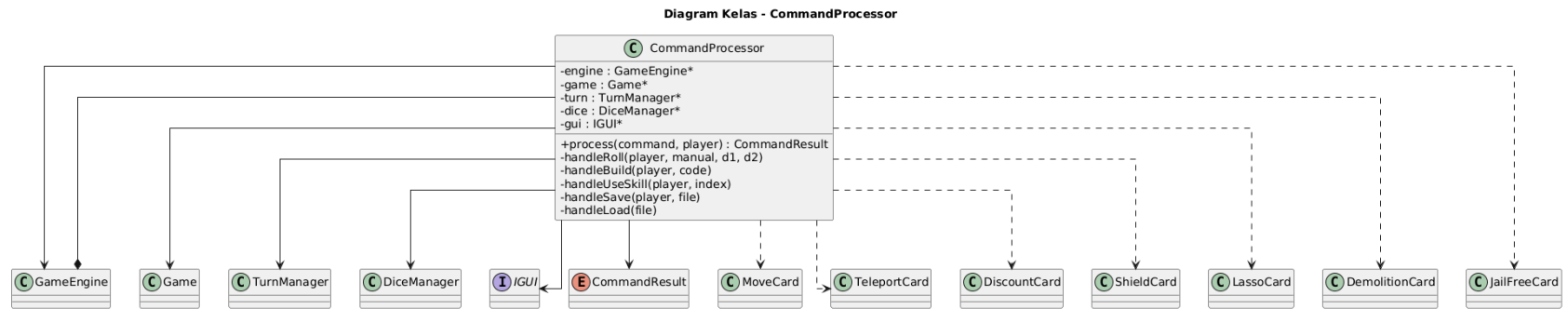
Diagram Kelas - AuctionManager



1.3.2. Diagram Kelas Core - BankruptcyManager



1.3.3. Diagram Kelas Core - CommandProcessor



1.3.4. Diagram Kelas Core - CommandResult & DiceManager

Diagram Kelas - CommandResult

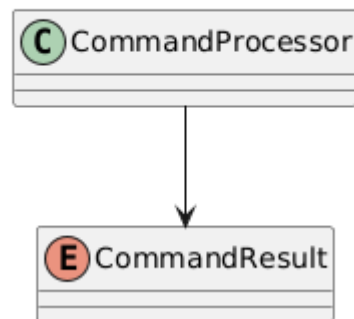
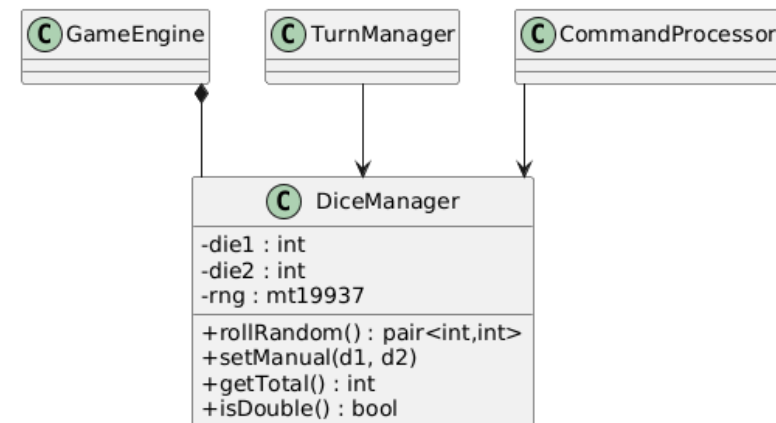
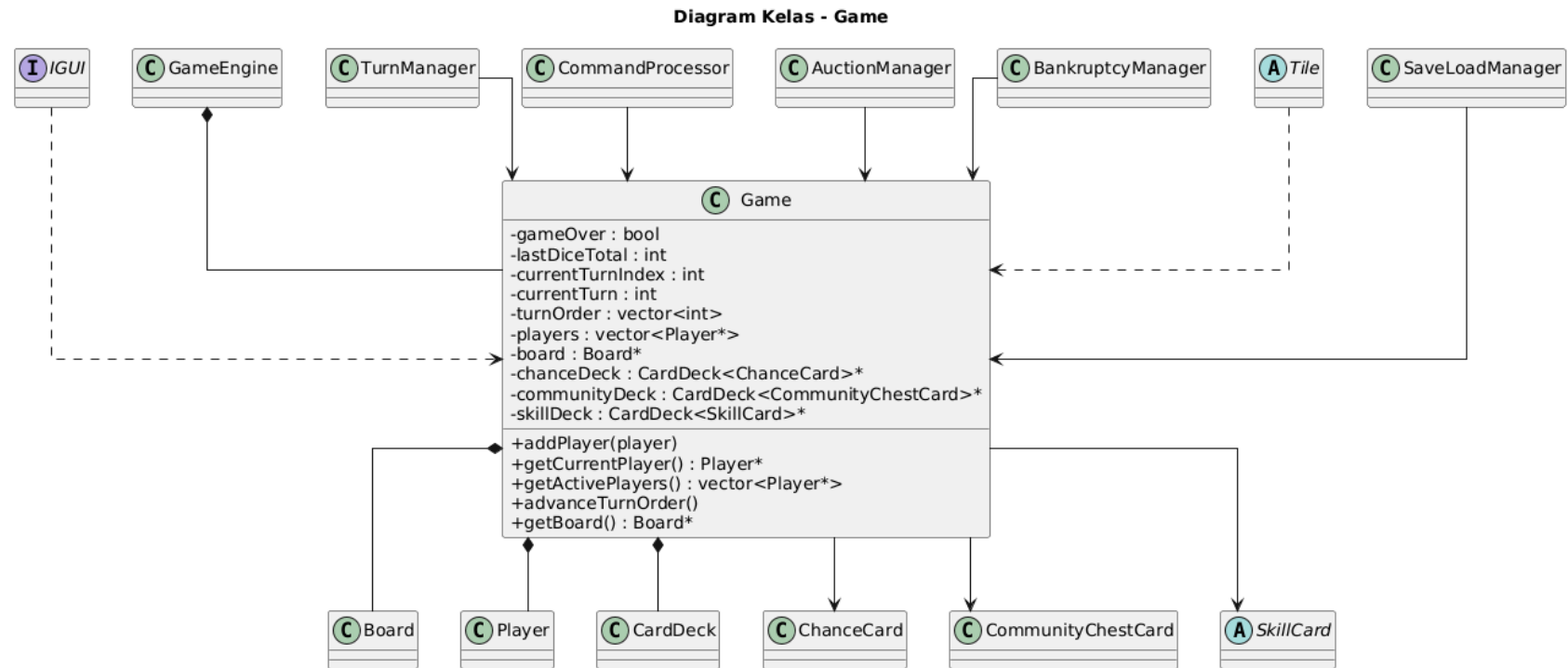


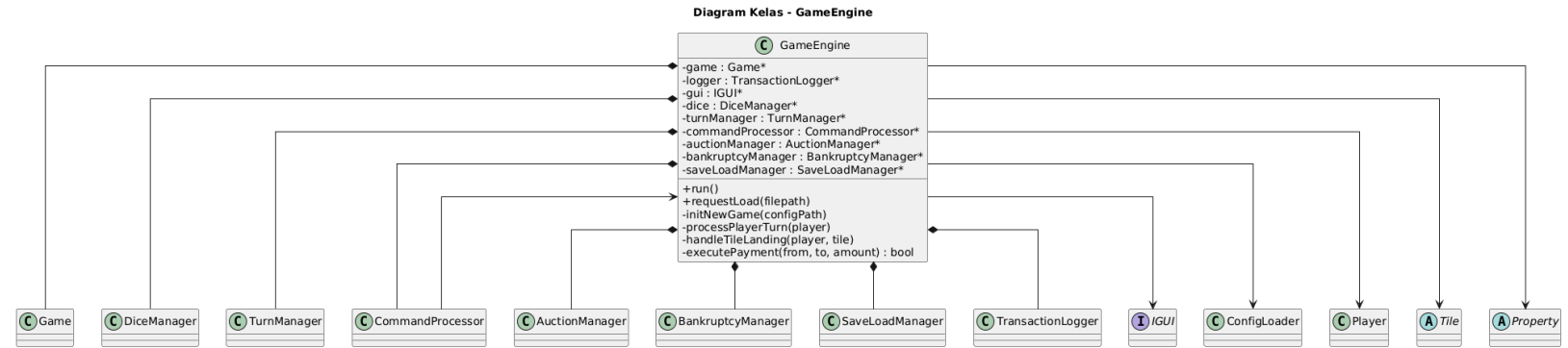
Diagram Kelas - DiceManager



1.3.5. Diagram Kelas Core - Game



1.3.6. Diagram Kelas Core - GameEngine



1.3.7. Diagram Kelas Core - Kind, LiquidationOption, & PendingDeckState

Diagram Kelas - PendingDeckState

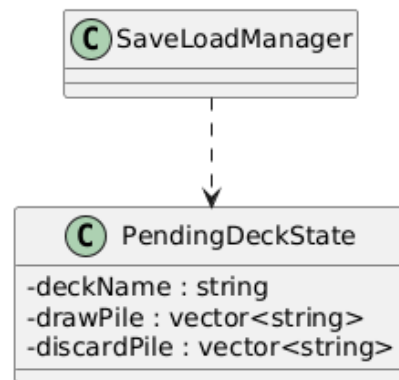


Diagram Kelas - LiquidationOption

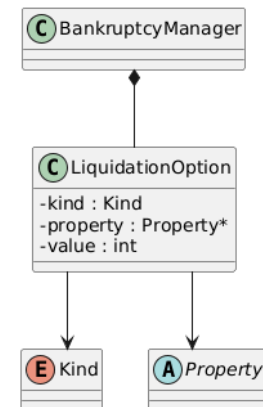
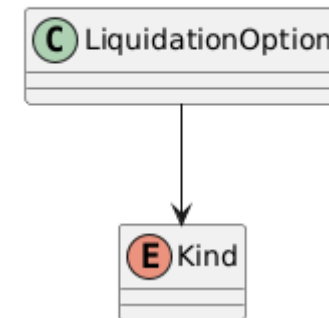


Diagram Kelas - Kind



1.3.8. Diagram Kelas Core - PendingHandCard & PendingProp

Diagram Kelas - PendingHandCard

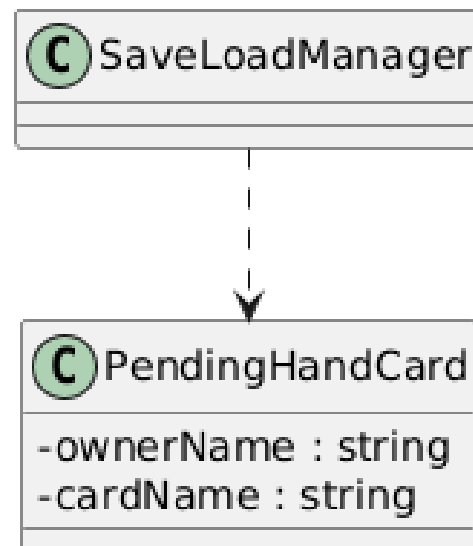
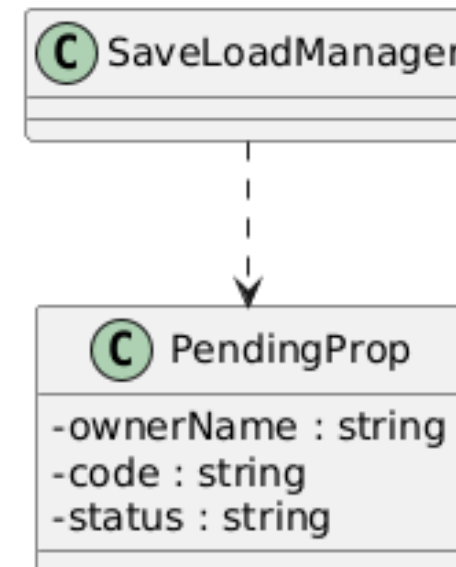


Diagram Kelas - PendingProp



1.3.9. Diagram Kelas Core - TurnManager & TurnPhase

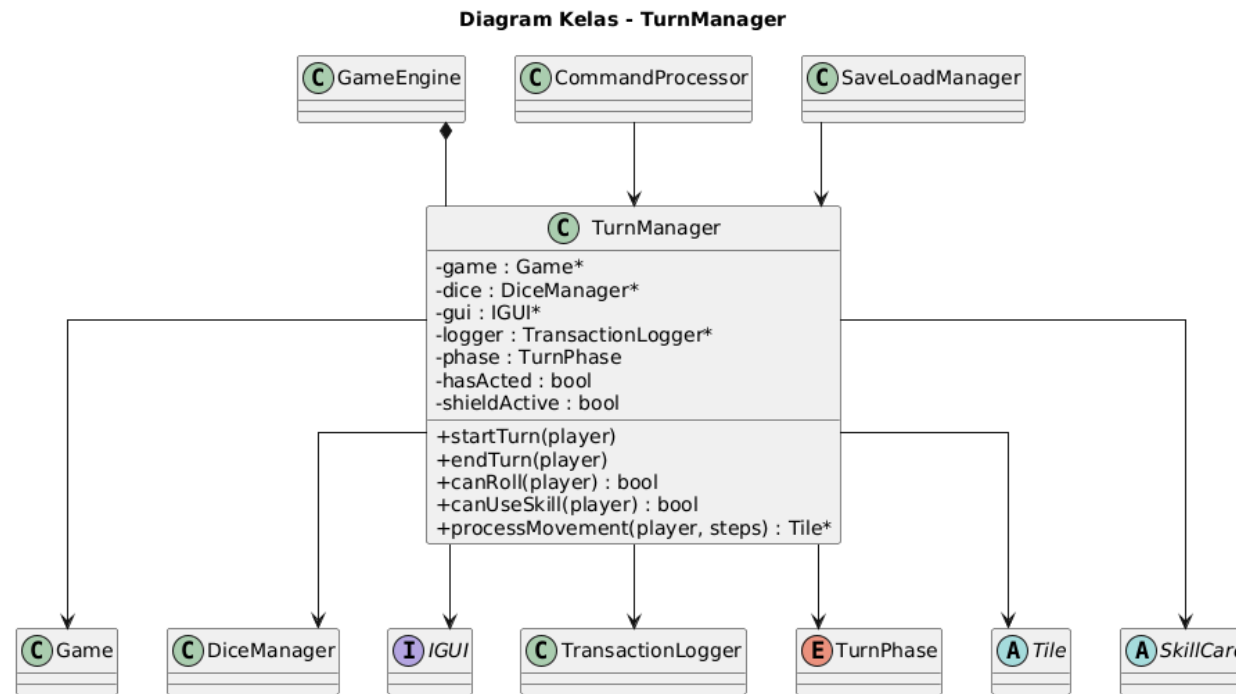
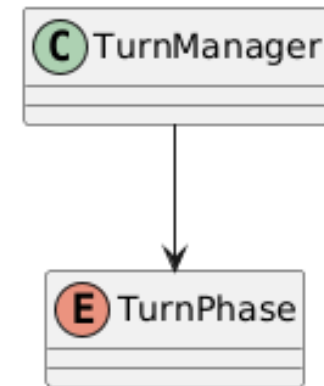


Diagram Kelas - TurnPhase



1.3.10. Diagram Kelas Data - ActionTileConfig

Diagram Kelas - ActionTileConfig

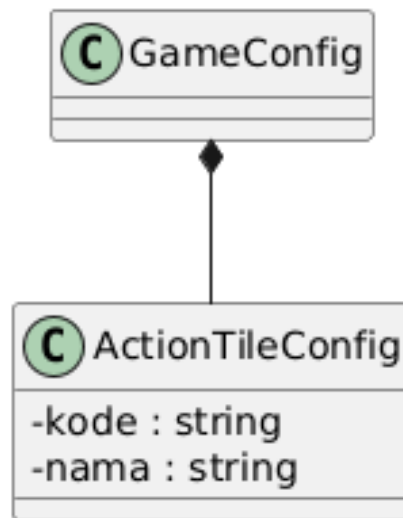
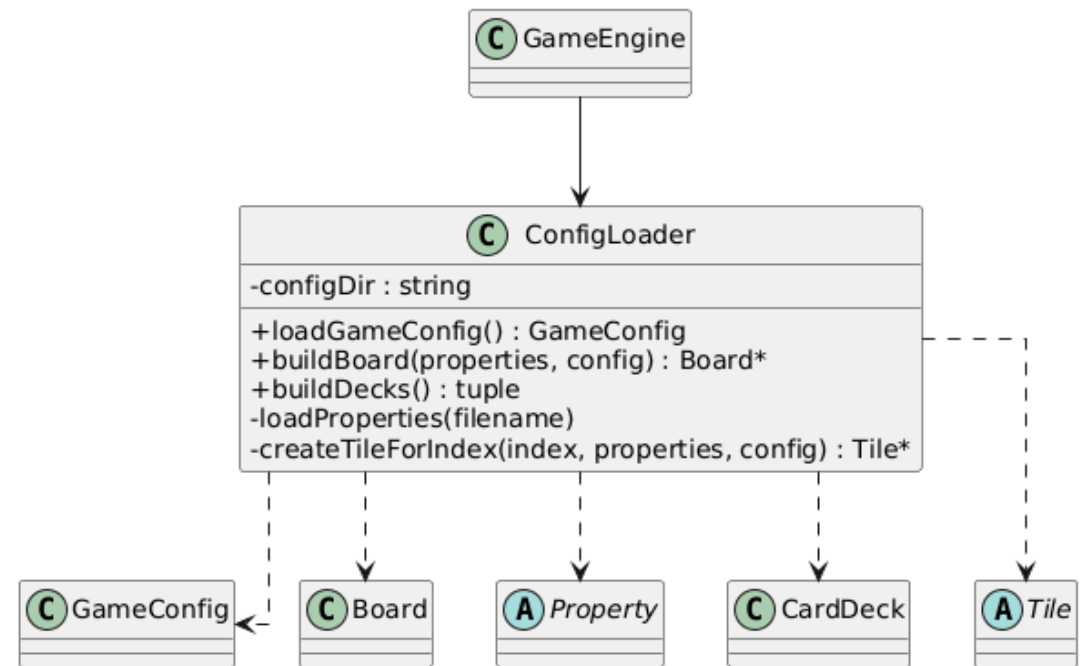


Diagram Kelas - ConfigLoader



1.3.11. Diagram Kelas Data - GameConfig & LogEntry

Diagram Kelas - ConfigLoader

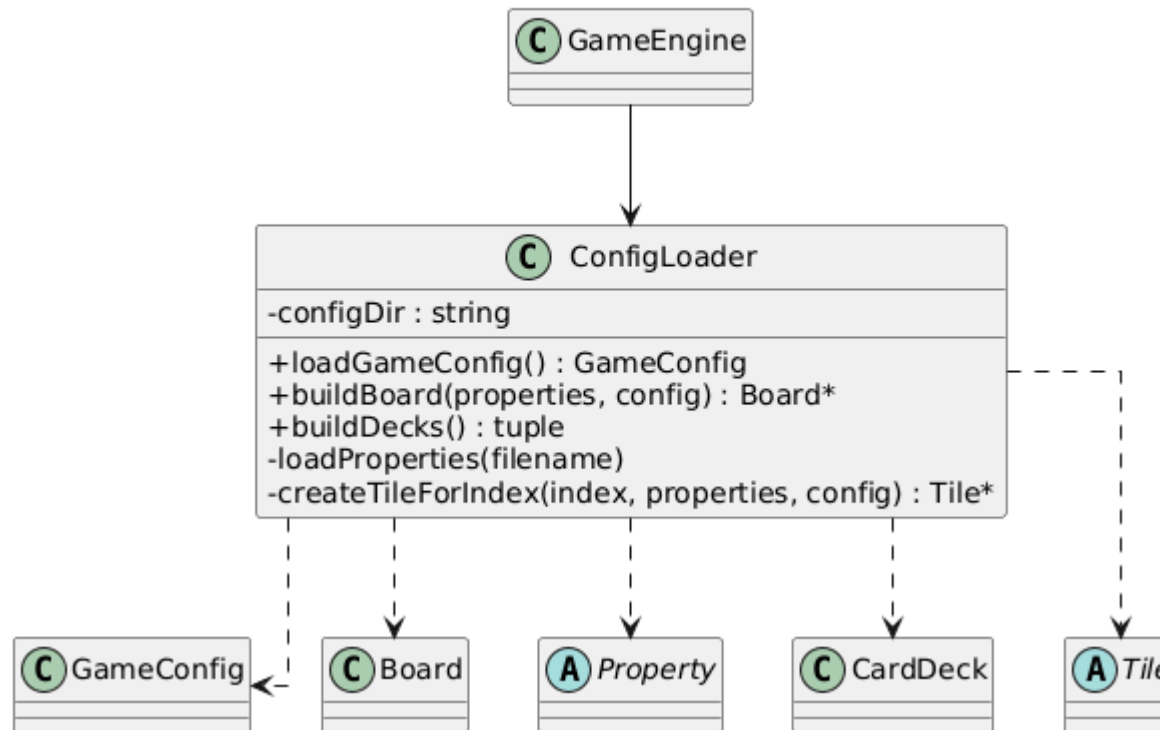
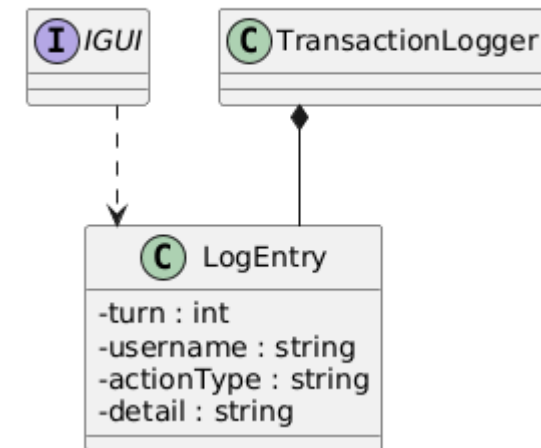


Diagram Kelas - LogEntry



1.3.12. Diagram Kelas Data - MiscConfig & SaveLoadManager

Diagram Kelas - MiscConfig

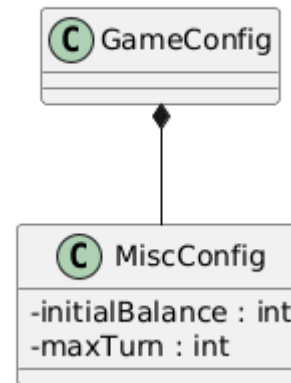
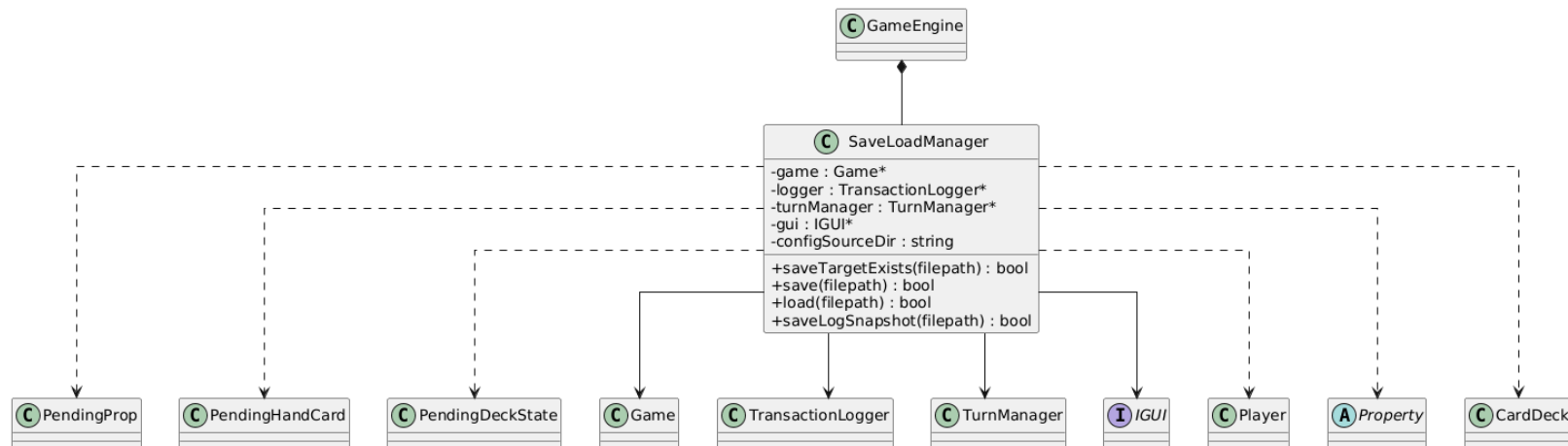


Diagram Kelas - SaveLoadManager



1.3.13. Diagram Kelas Data - SpecialConfig & TaxConfig

Diagram Kelas - SpecialConfig

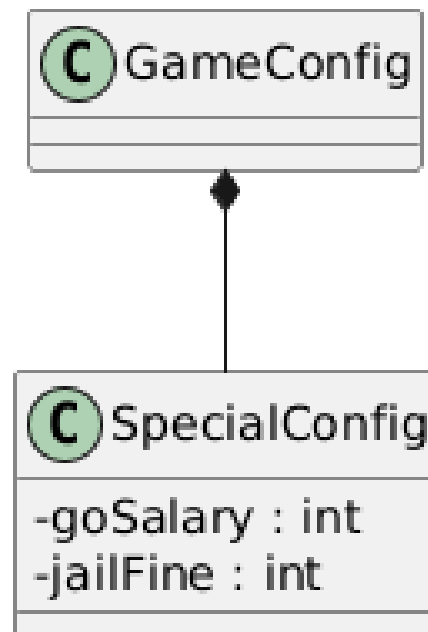
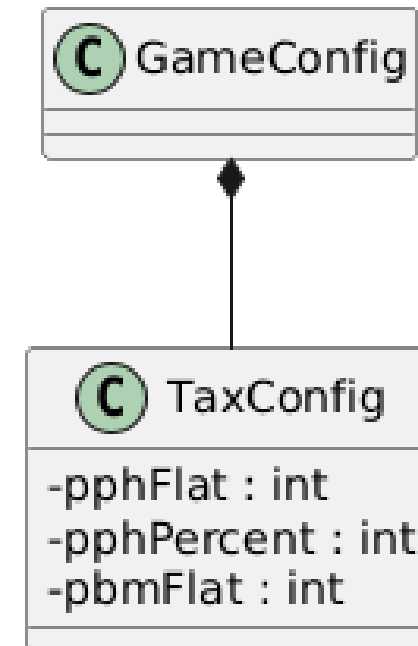
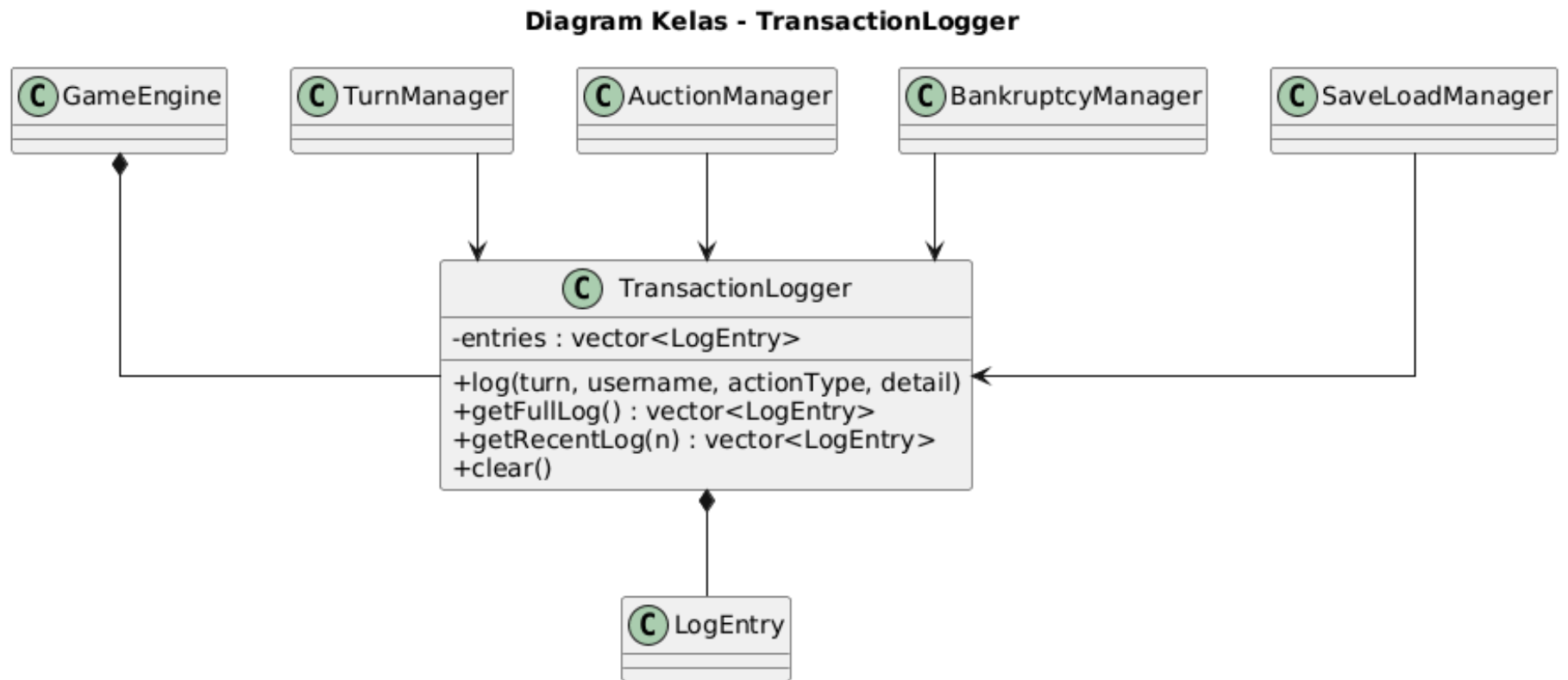


Diagram Kelas - TaxConfig



1.3.14. Diagram Kelas Data - TransactionLogger



1.3.15. Diagram Kelas Exception - DiceAlreadyRolledException & ExistingHotelException

Diagram Kelas - DiceAlreadyRolledException

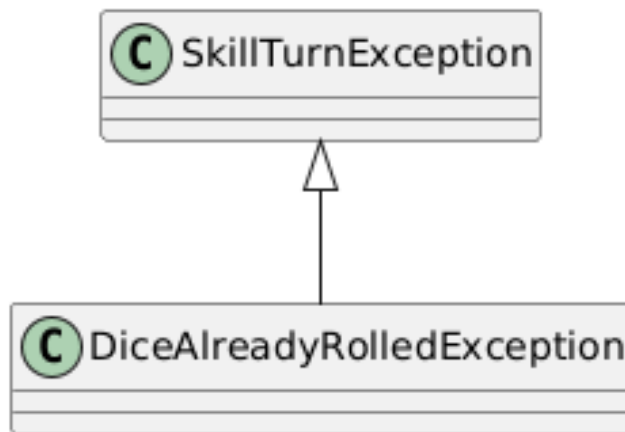
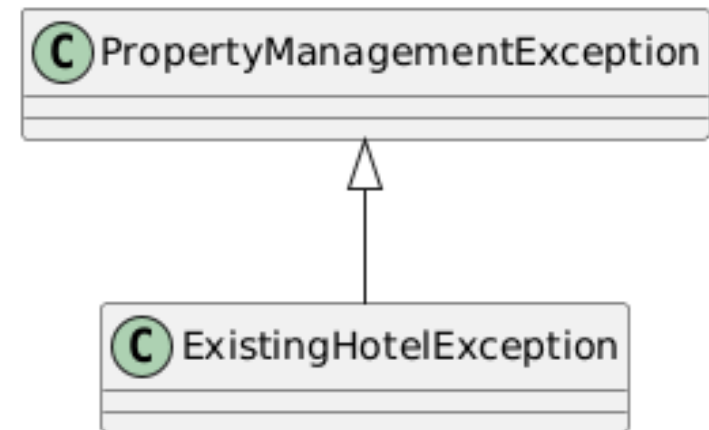


Diagram Kelas - ExistingHotelException



1.3.16. Diagram Kelas Exception - FailedToSaveException

Diagram Kelas - FailedToSaveException

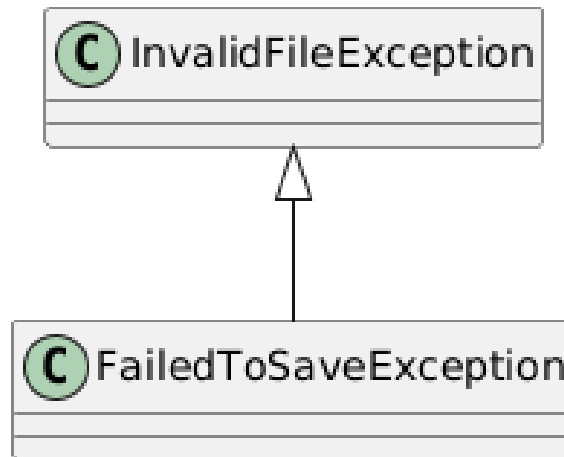
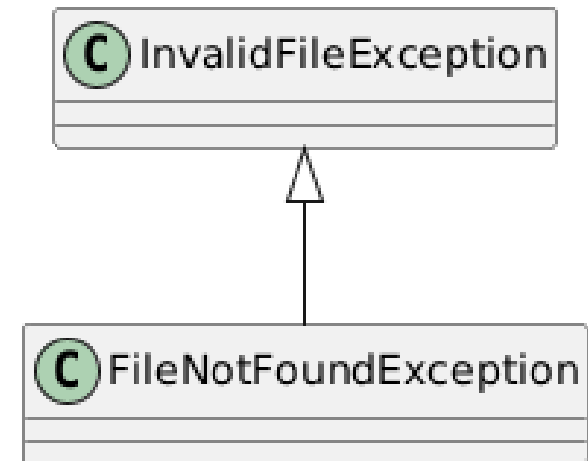


Diagram Kelas - FileNotFoundException



1.3.17. Diagram Kelas Exception - GameException & InsufficientMoneyException

Diagram Kelas - GameException

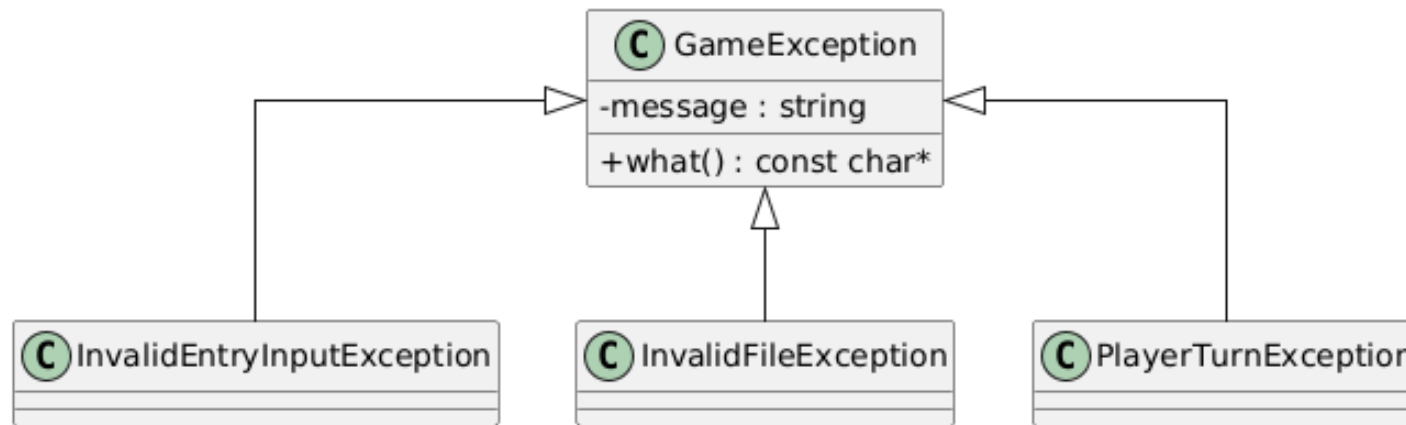
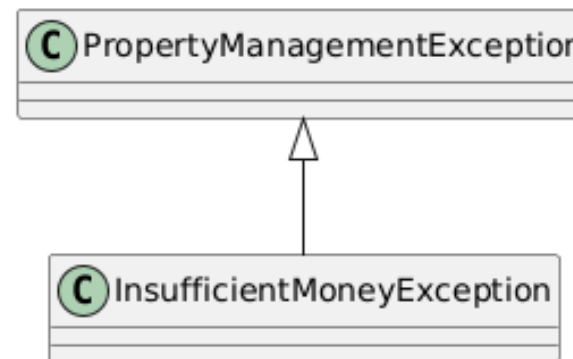


Diagram Kelas - InsufficientMoneyException

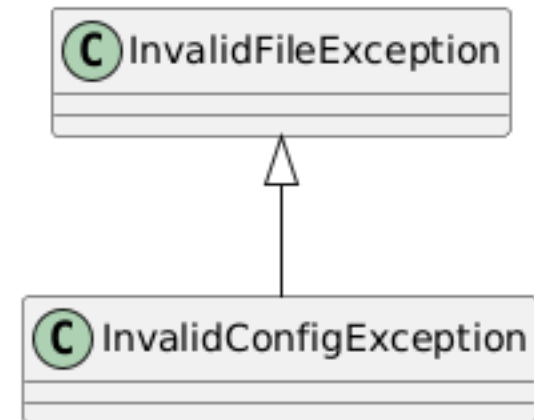


1.3.18. Diagram Kelas Exception - InsufficientOwnedColorException & InvalidConfigException

Diagram Kelas - InsufficientOwnedColorException



Diagram Kelas - InvalidConfigException



1.3.19. Diagram Kelas Exception - InvalidDiceNumberException & InvalidEntryInputException

Diagram Kelas - InvalidDiceNumberException

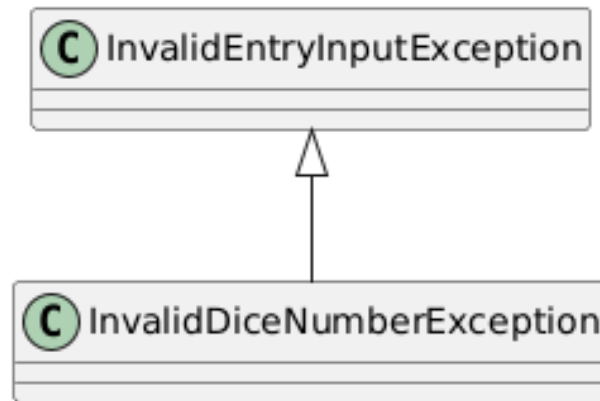
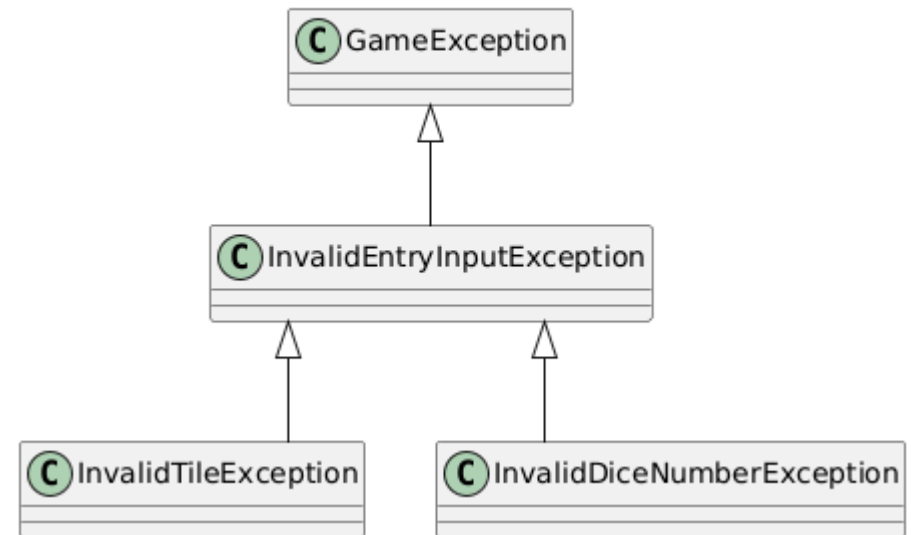


Diagram Kelas - InvalidEntryInputException



1.3.20. Diagram Kelas Exception - InvalidFileException & InvalidTileException

Diagram Kelas - InvalidFileException

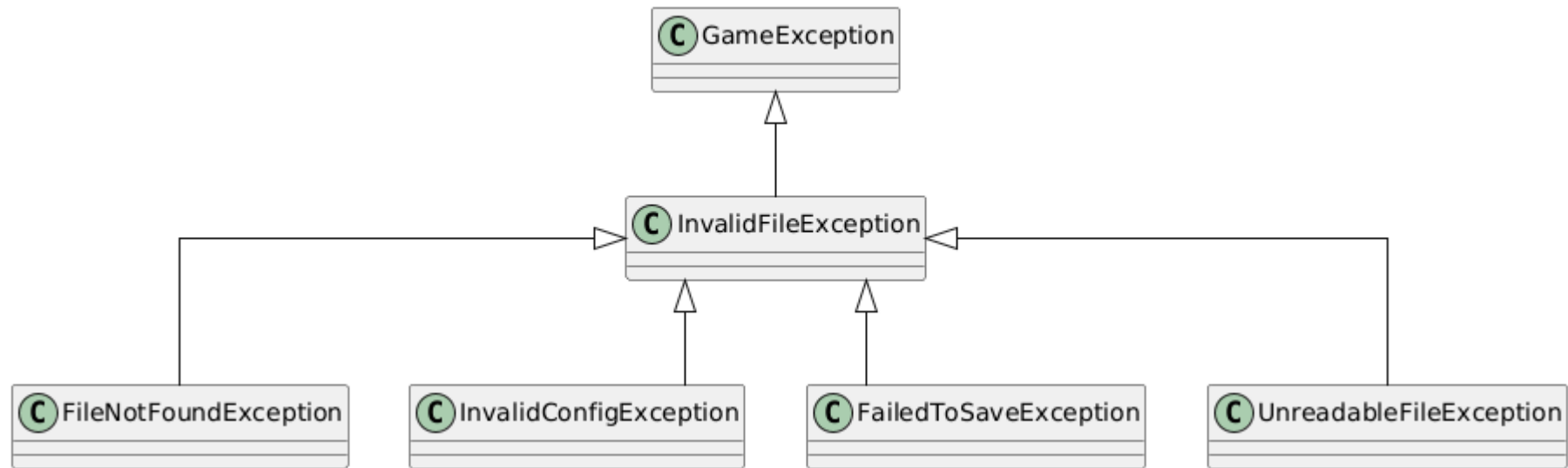
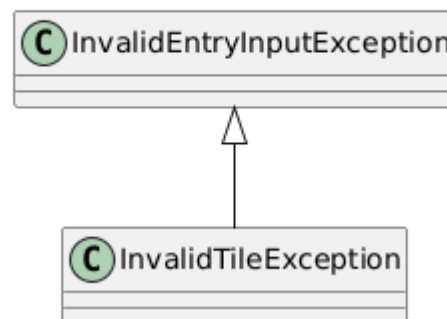


Diagram Kelas - InvalidTileException



1.3.21. Diagram Kelas Exception - InvalidTurnException & PlayerTurnException

Diagram Kelas - InvalidTurnException

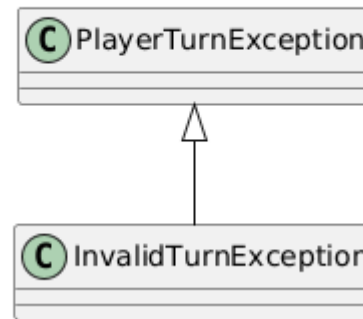
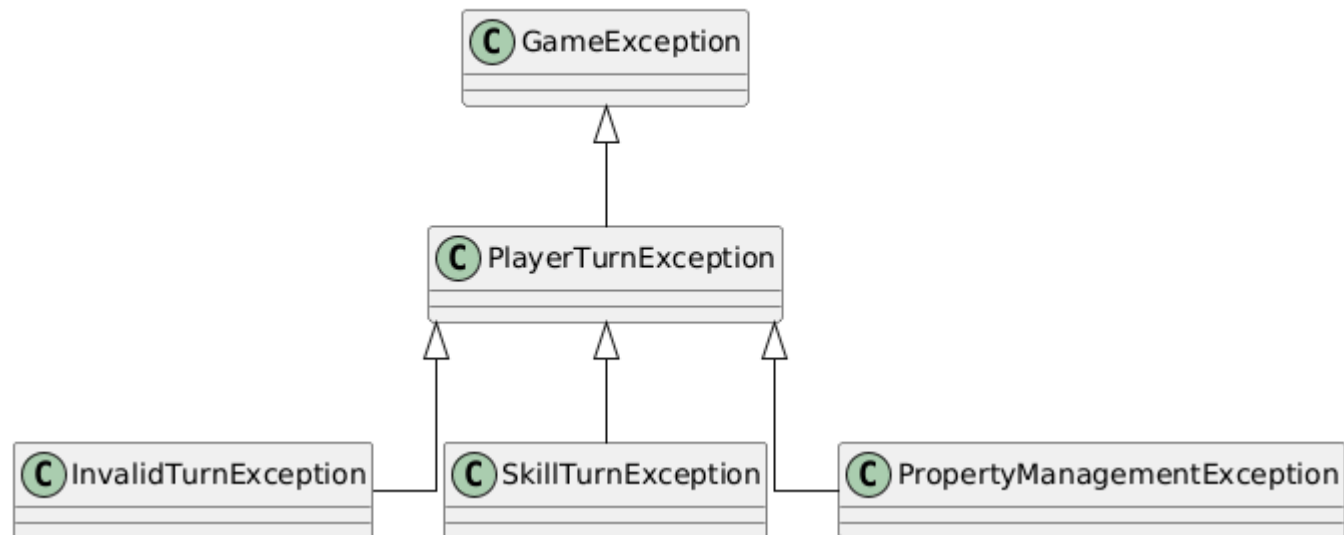
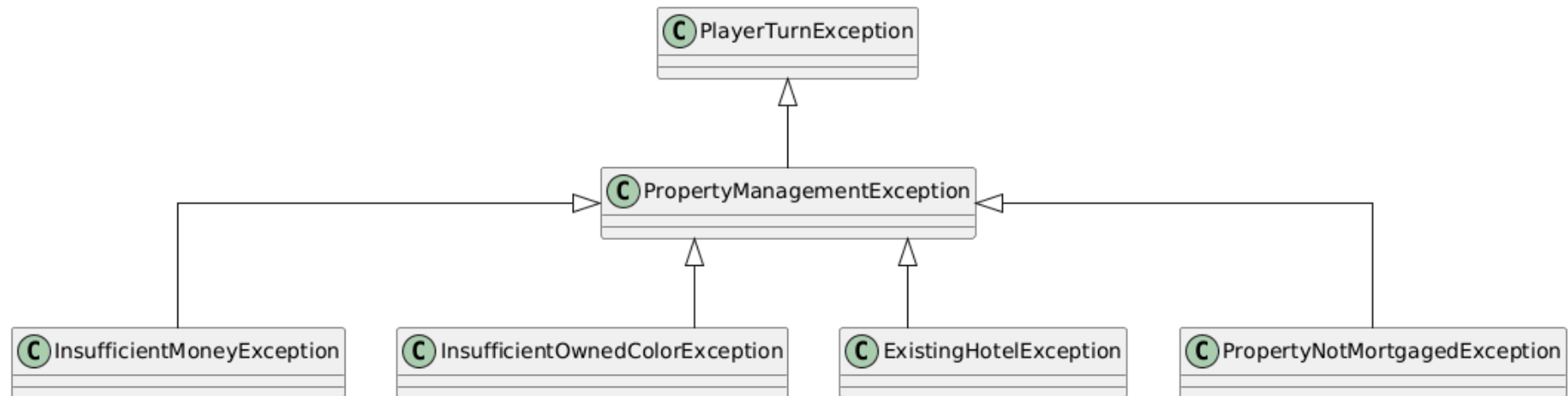


Diagram Kelas - PlayerTurnException



1.3.22. Diagram Kelas Exception - PropertyManagementException

Diagram Kelas - PropertyManagementException



1.3.23. Diagram Kelas Exception - PropertyNotMortgagedException & SkillCardUsedException

Diagram Kelas - PropertyNotMortgagedException

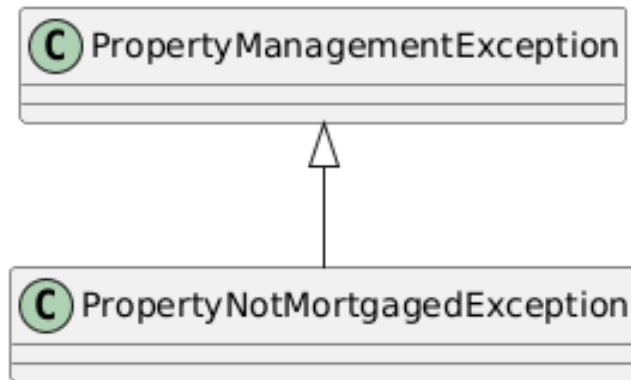
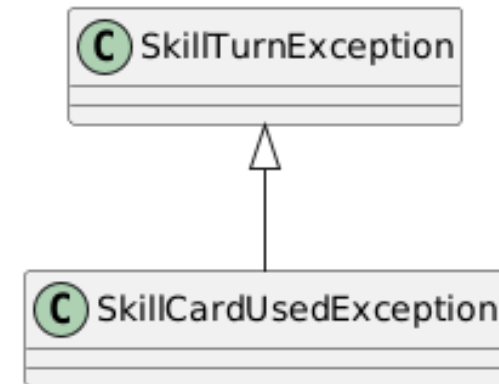


Diagram Kelas - SkillCardUsedException



1.3.24. Diagram Kelas Exception - SkillTurnException & UnreadableFileException

Diagram Kelas - SkillTurnException

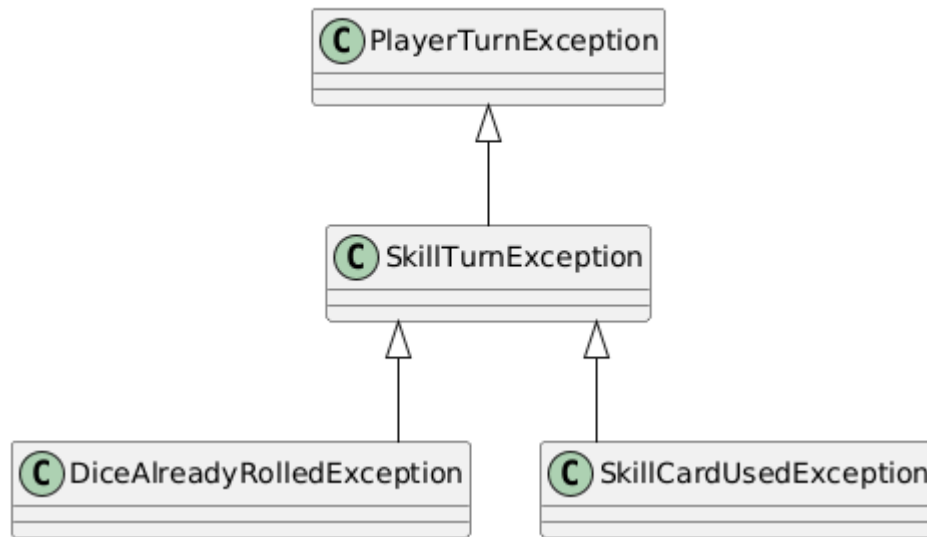
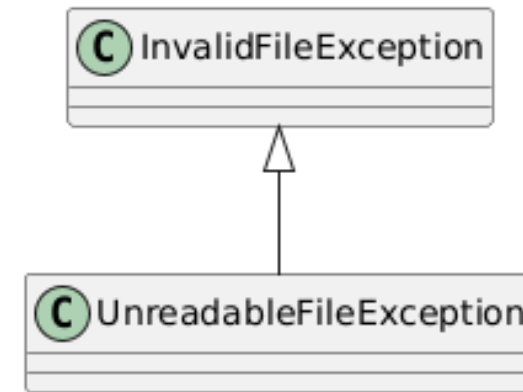
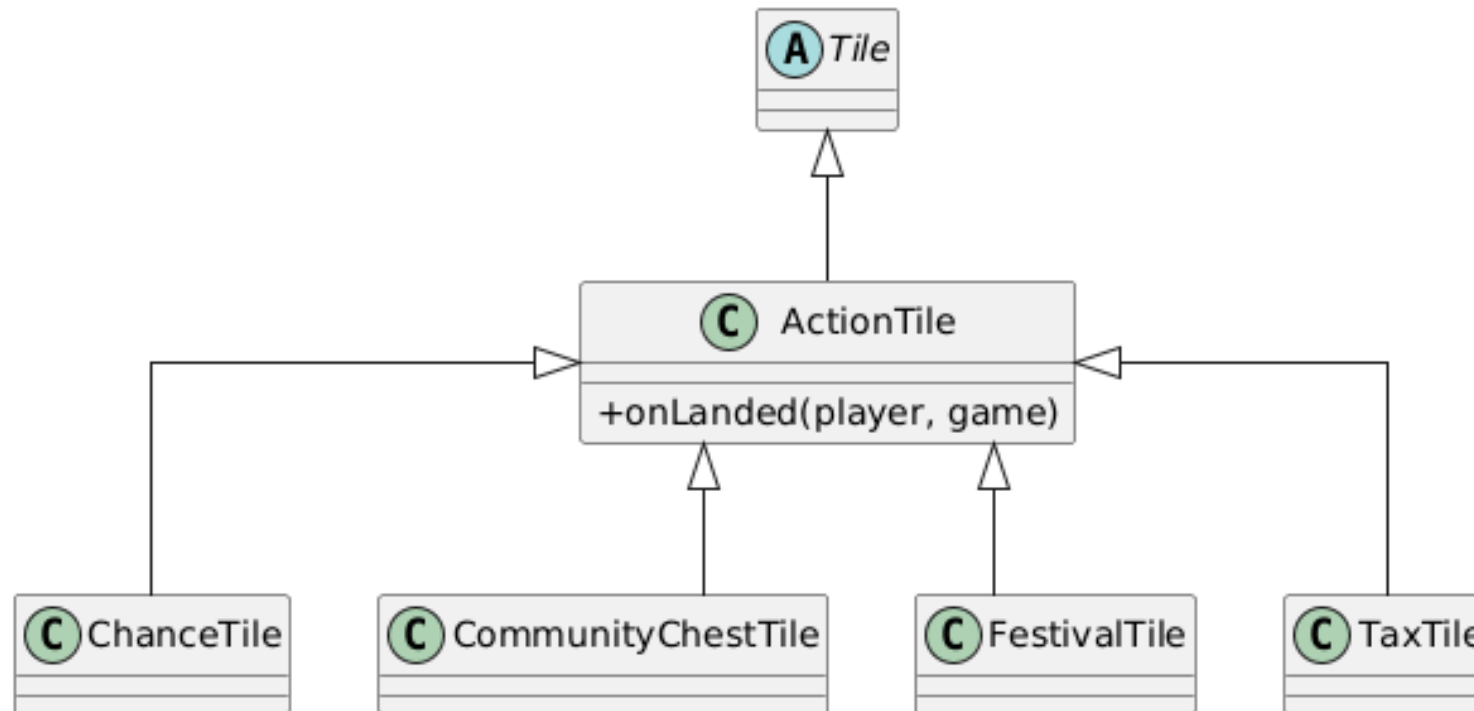


Diagram Kelas - UnreadableFileException

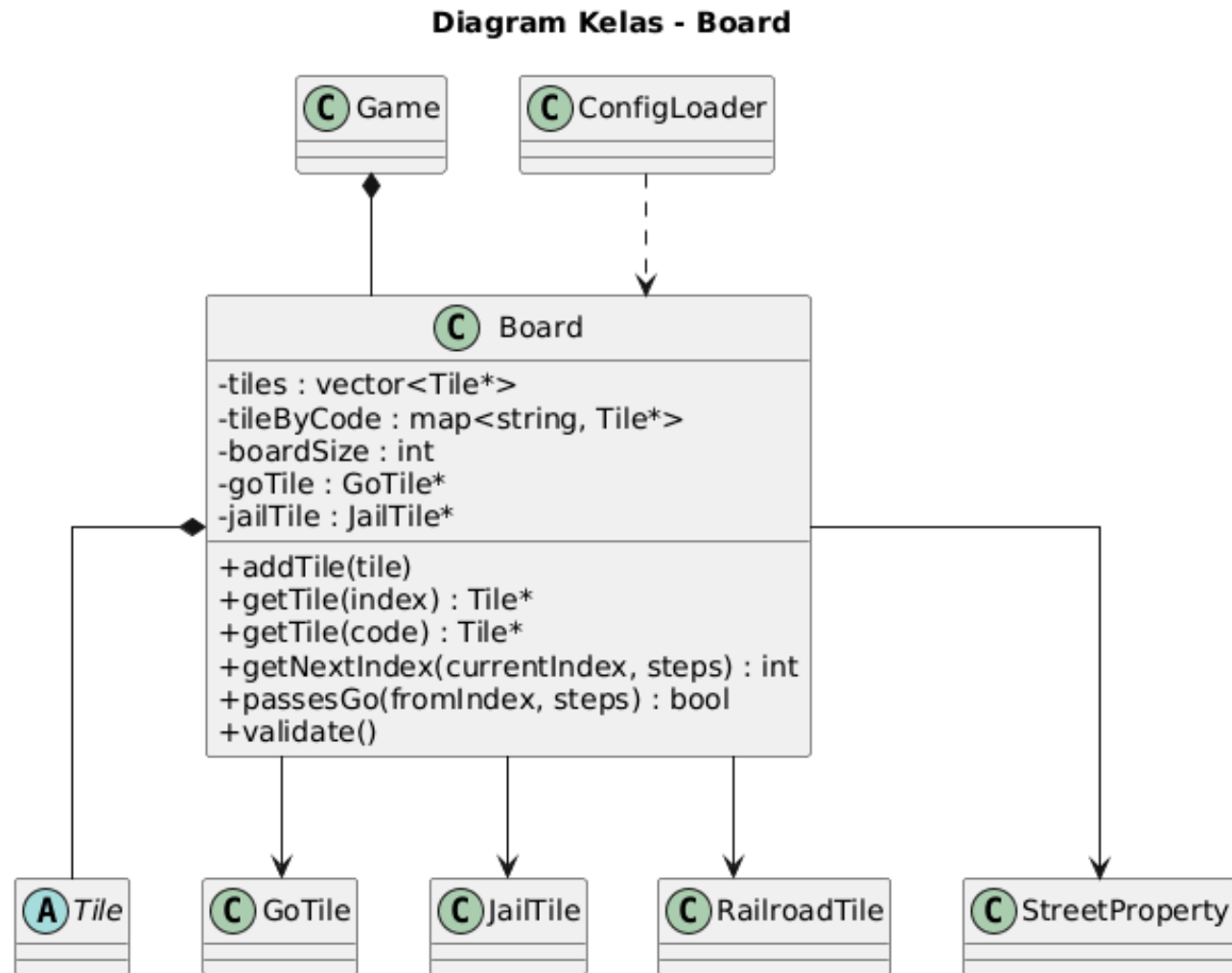


1.3.25. Diagram Kelas Model Board Tiles - ActionTile

Diagram Kelas - ActionTile



1.3.26. Diagram Kelas Model Board Tiles - Board



1.3.27. Diagram Kelas Model Board Tiles - ChanceTile, CommunityChestTile, FestivalTile, & FreeParkingTile

Diagram Kelas - ChanceTile

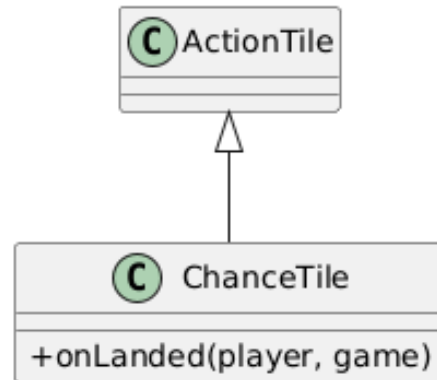


Diagram Kelas - CommunityChestTile

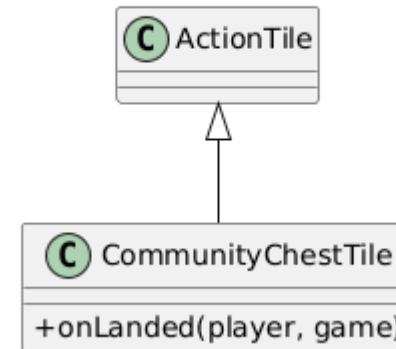


Diagram Kelas - FestivalTile

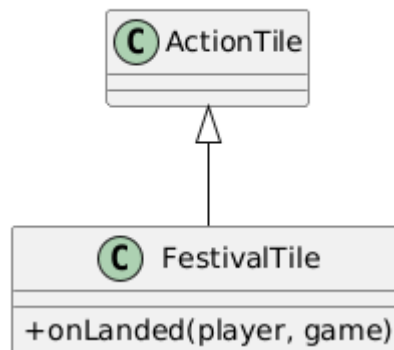
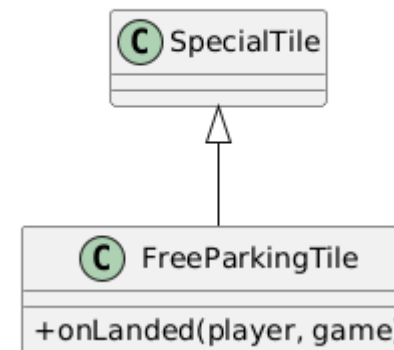


Diagram Kelas - FreeParkingTile



1.3.28. Diagram Kelas Model Board Tiles - GoTile, GoToJailTile, IncomeTaxTile, & JailTile

Diagram Kelas - GoTile

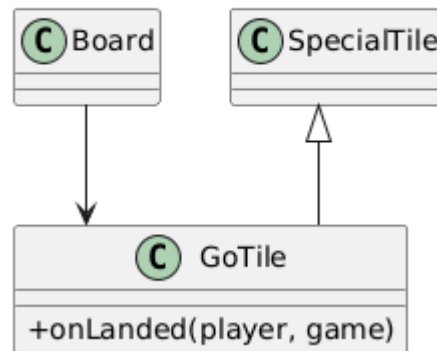


Diagram Kelas - GoToJailTile

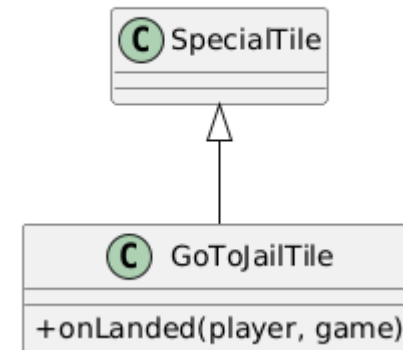


Diagram Kelas - IncomeTaxTile

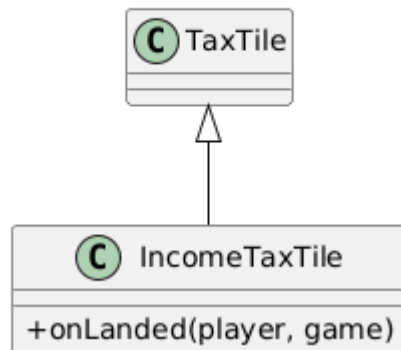
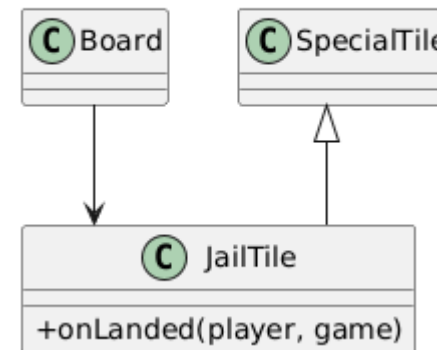


Diagram Kelas - JailTile



1.3.29. Diagram Kelas Model Board Tiles - LuxuryTaxTile & PropertyTile

Diagram Kelas - LuxuryTaxTile

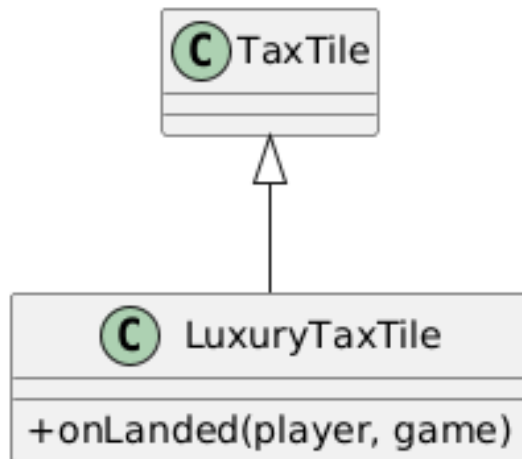
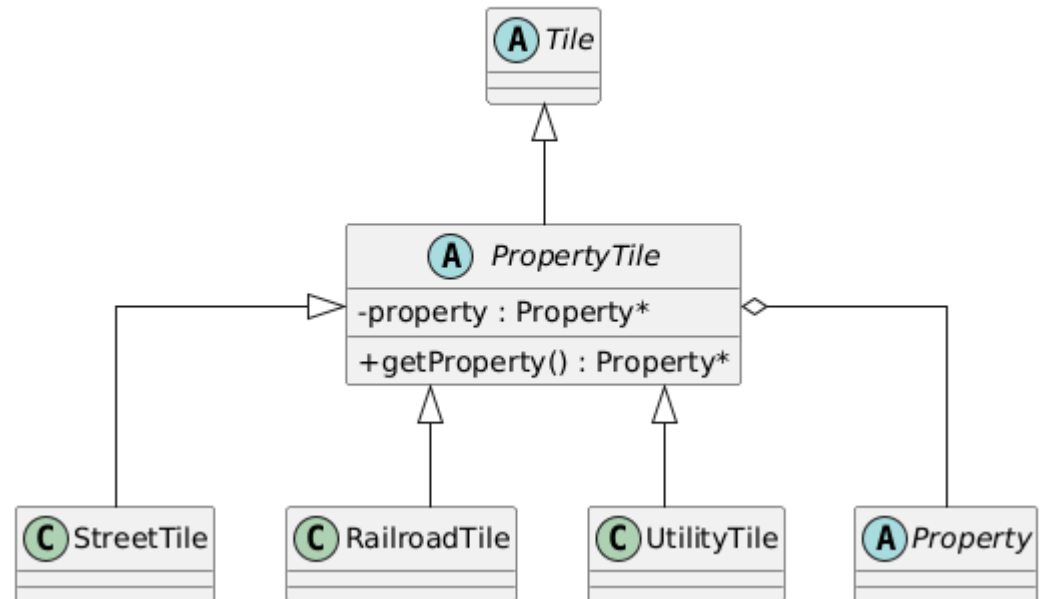


Diagram Kelas - PropertyTile



1.3.30. Diagram Kelas Model Board Tiles - RailroadTile & SpecialTile

Diagram Kelas - RailroadTile

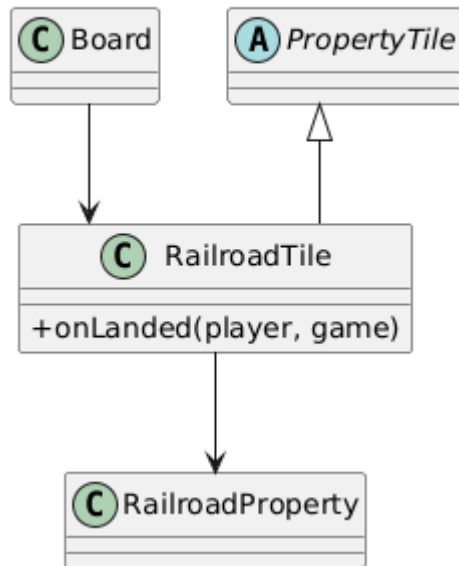
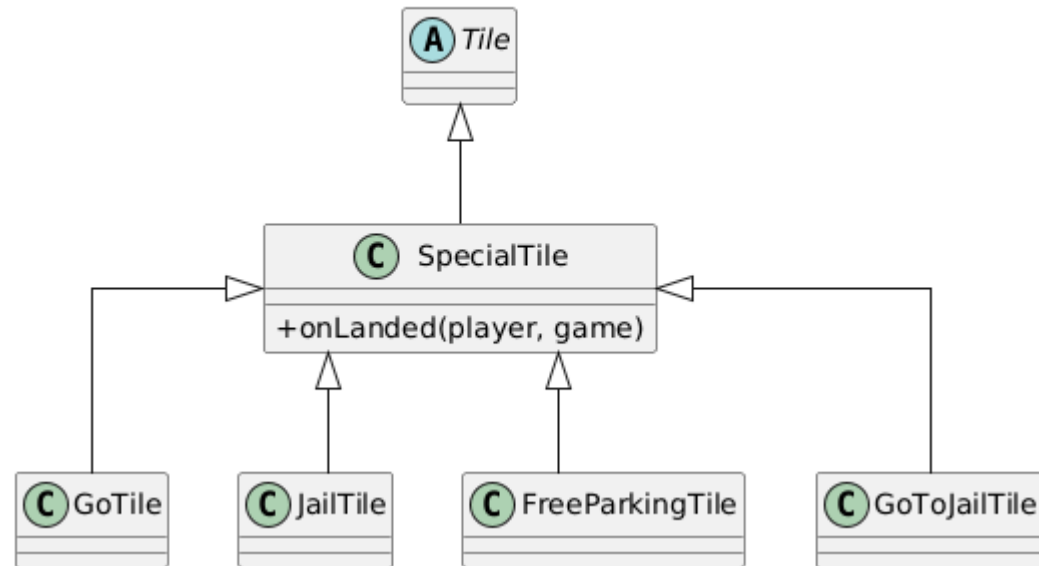


Diagram Kelas - SpecialTile



1.3.31. Diagram Kelas Model Board Tiles - StreetTile & TaxTile

Diagram Kelas - TaxTile

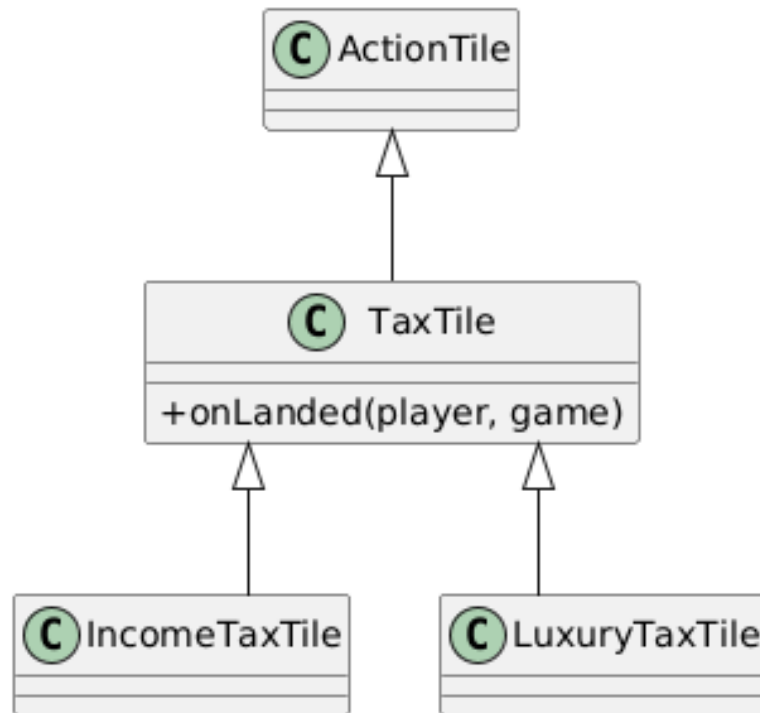
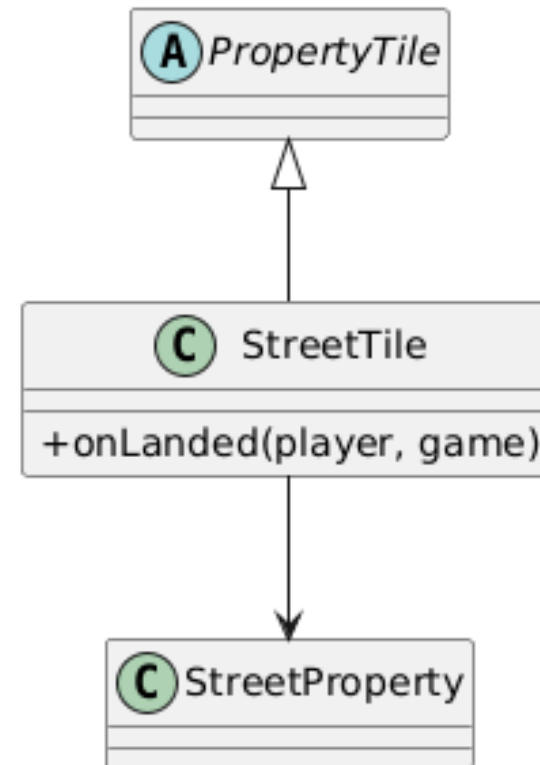
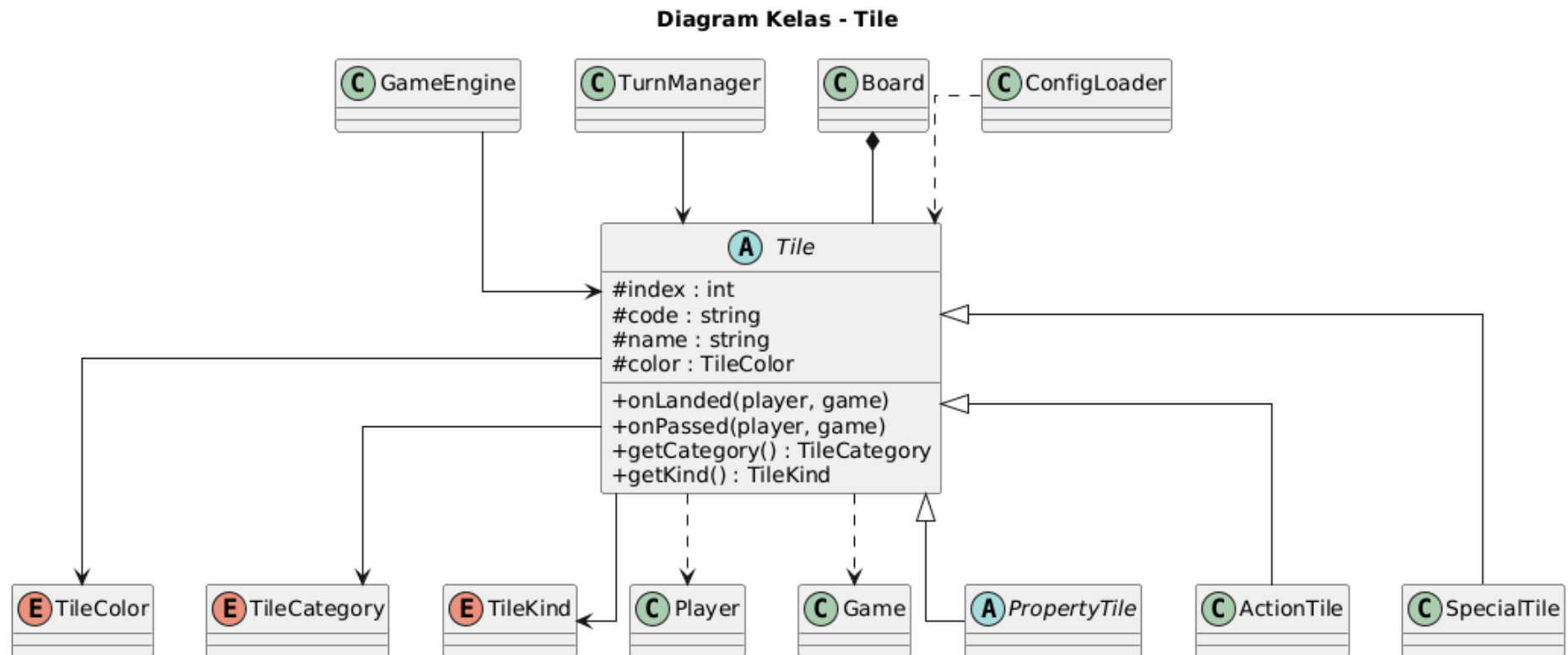


Diagram Kelas - StreetTile



1.3.32. Diagram Kelas Model Board Tiles - Tile



1.3.33. Diagram Kelas Model Board Tiles - TileCategory, TileColor, TileKind, & UtilityTile

Diagram Kelas - TileCategory

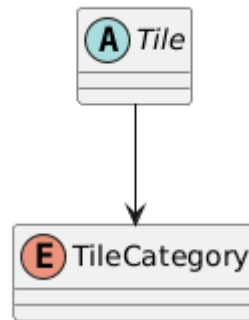


Diagram Kelas - TileColor

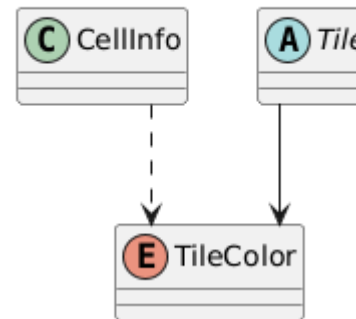


Diagram Kelas - TileKind

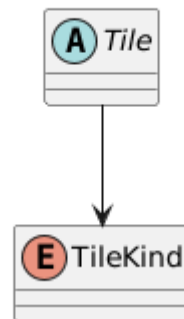
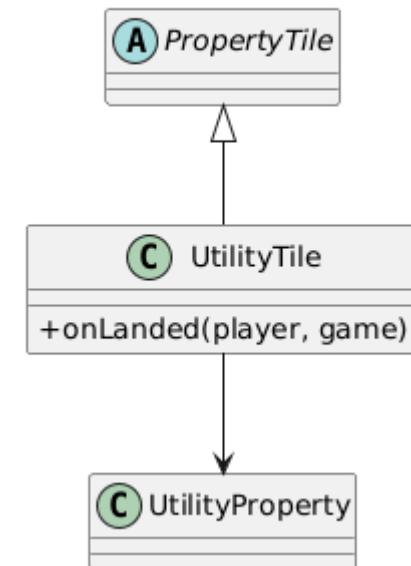
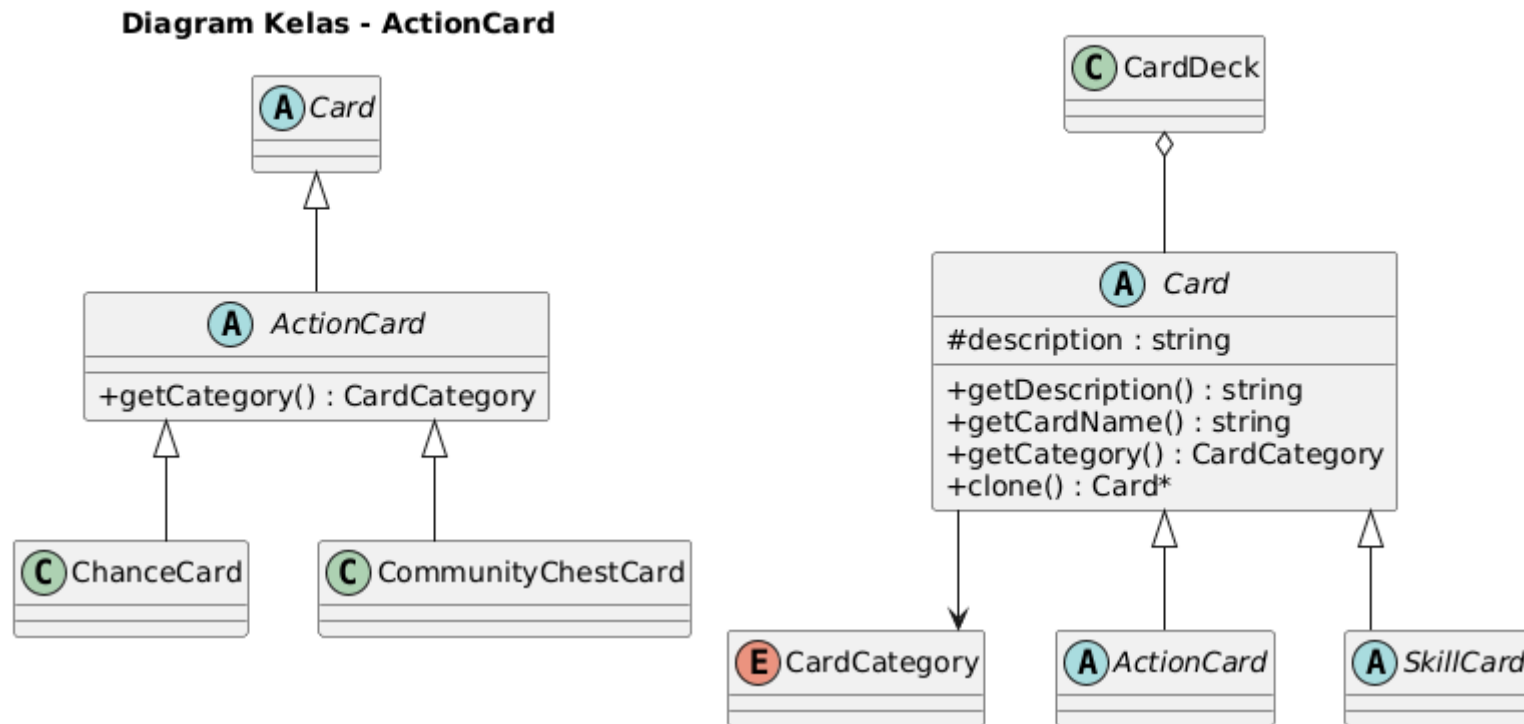


Diagram Kelas - UtilityTile



1.3.34. Diagram Kelas Model Card - ActionCard & Card



1.3.35. Diagram Kelas Model Card - CardCategory & CardDeck

Diagram Kelas - CardCategory

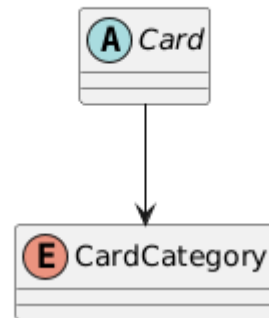
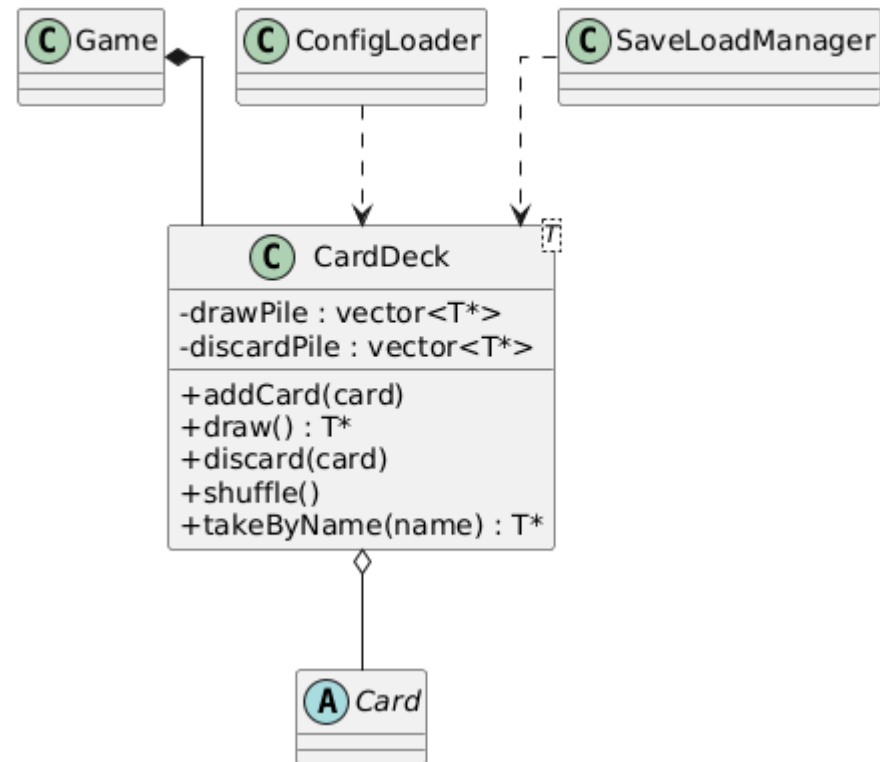


Diagram Kelas - CardDeck



1.3.36. Diagram Kelas Model Card - ChanceCard & ChanceType

Diagram Kelas - ChanceCard

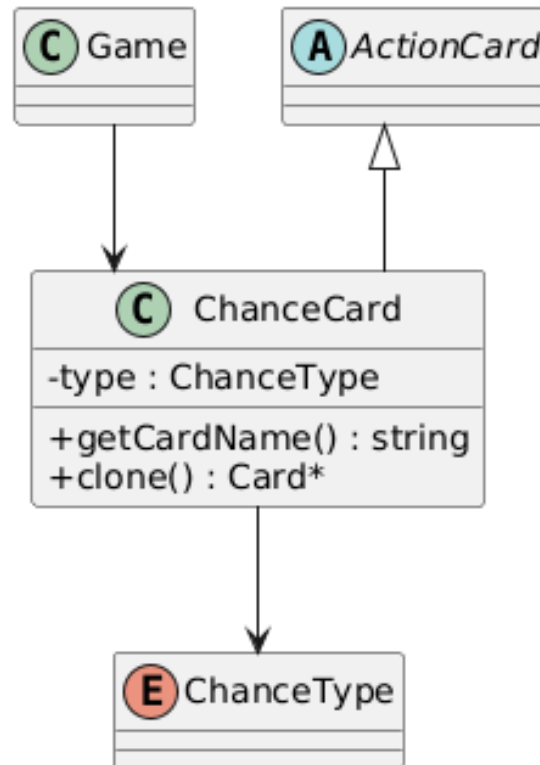
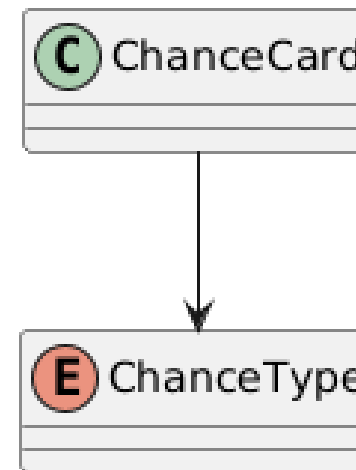


Diagram Kelas - ChanceType



1.3.37. Diagram Kelas Model Card - CommunityChestCard & CommunityType

Diagram Kelas - CommunityChestCard

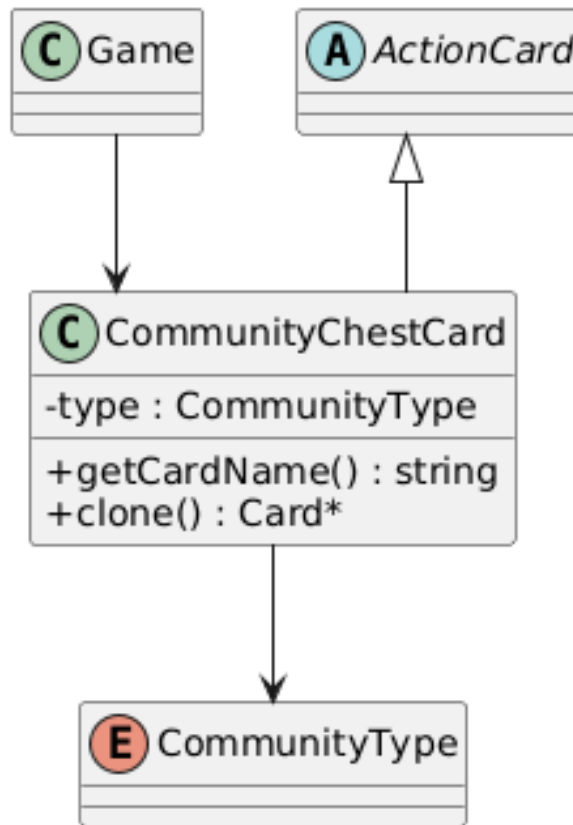
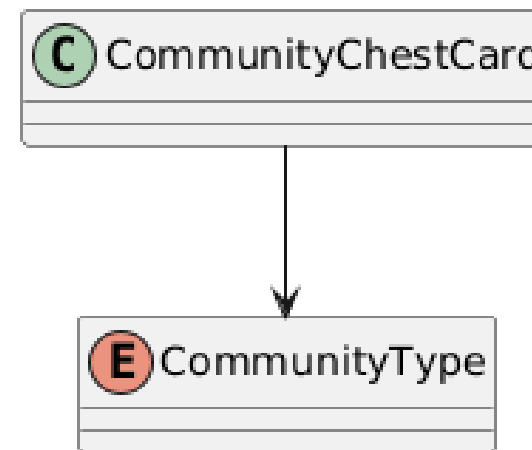
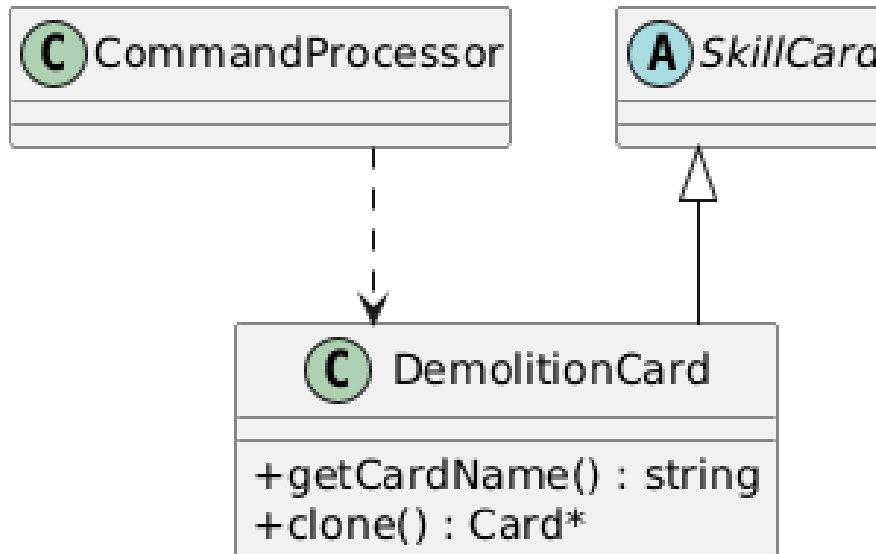
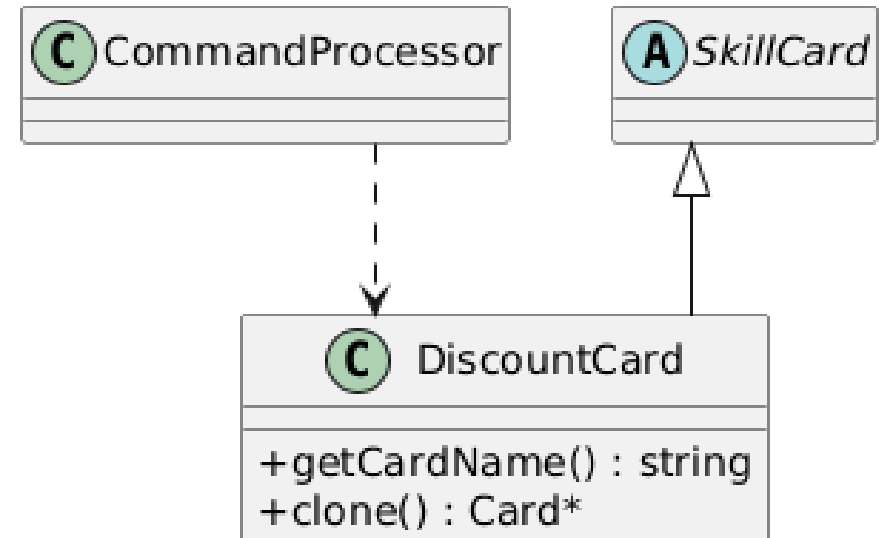
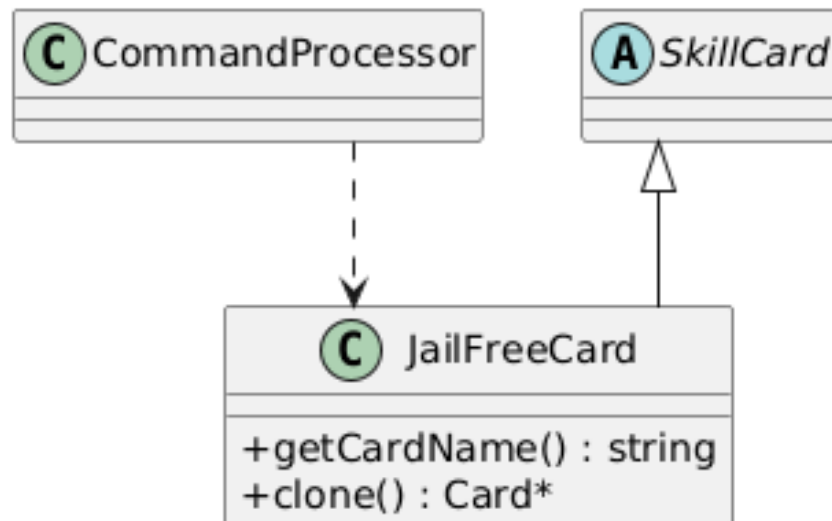
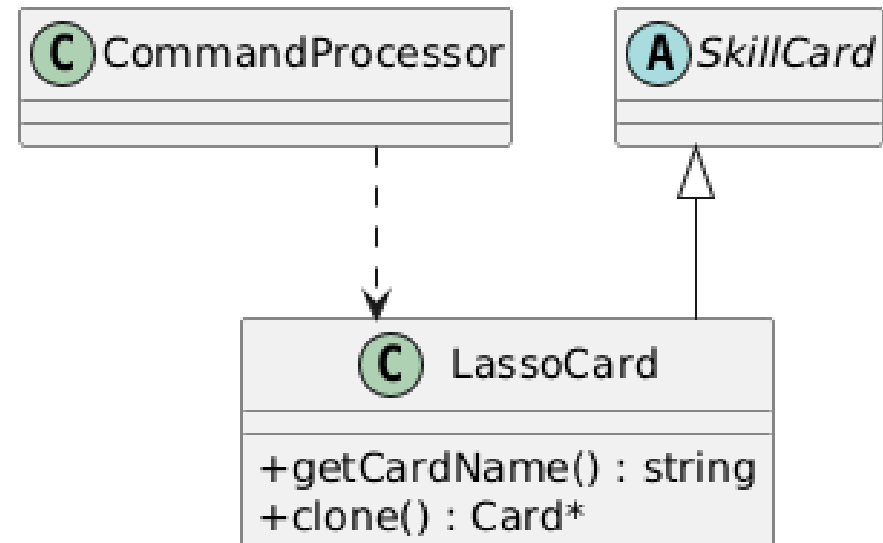
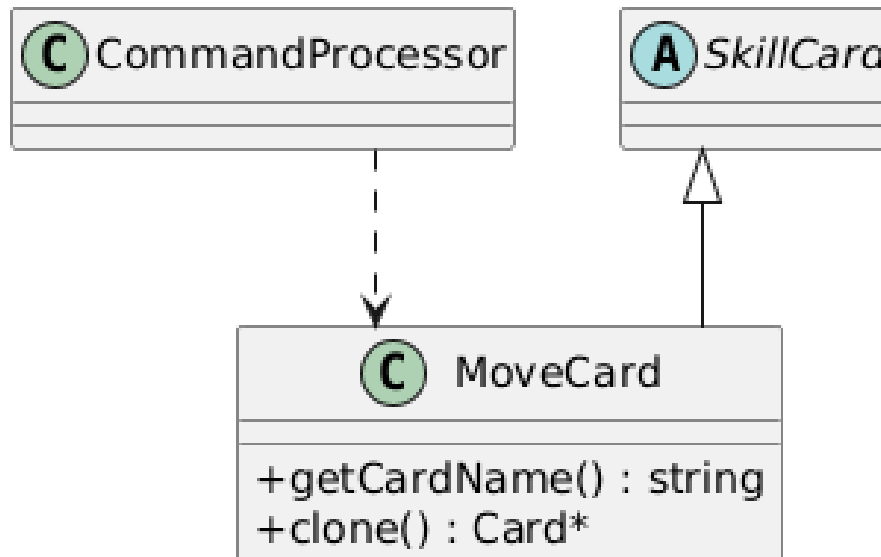
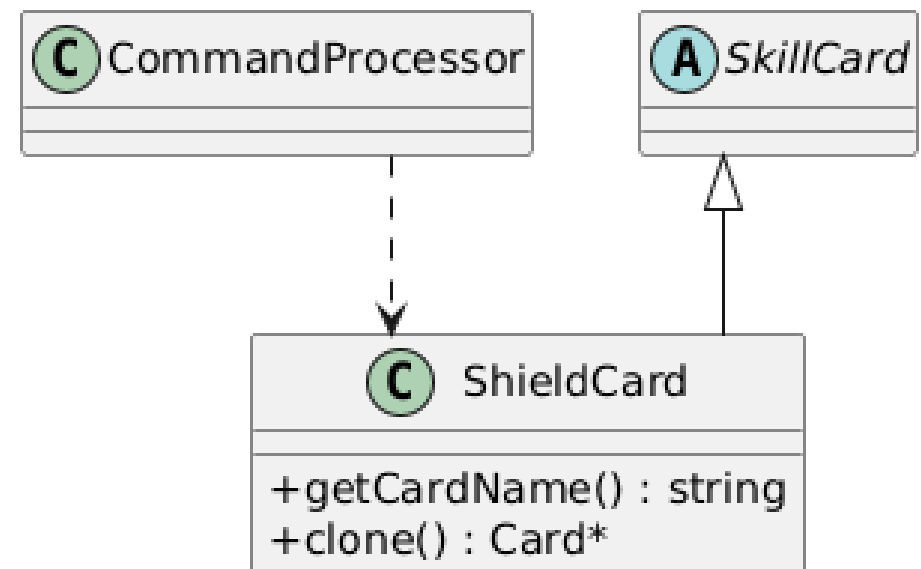


Diagram Kelas - CommunityType

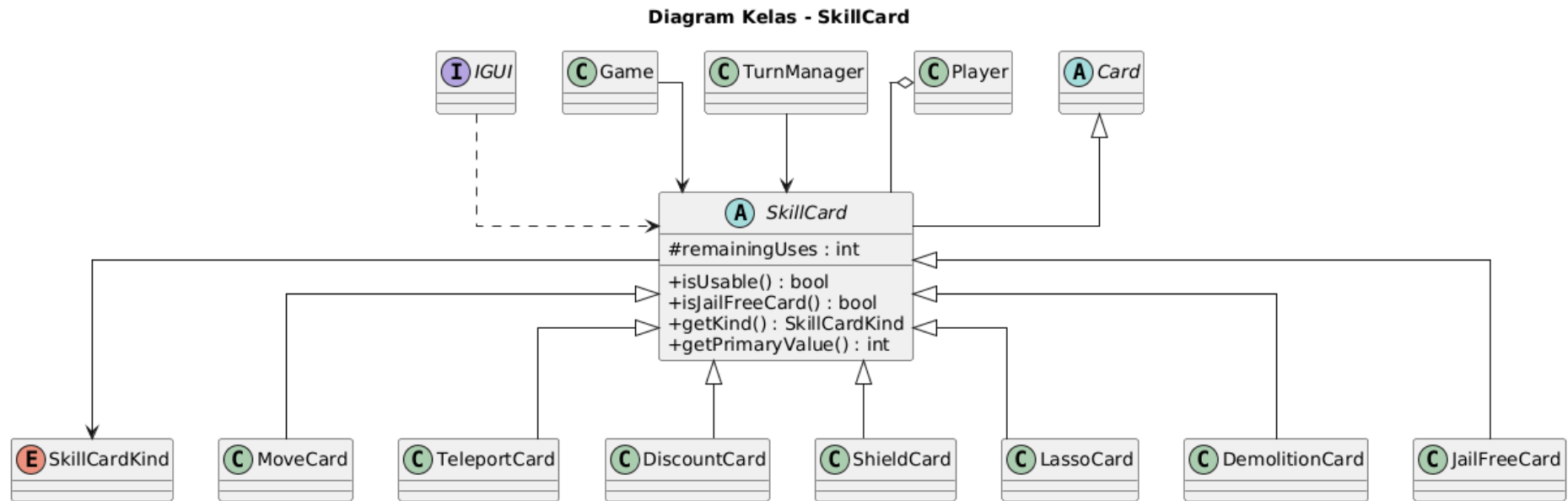


1.3.38. Diagram Kelas Model Card - DemolitionCard & DiscountCard**Diagram Kelas - DemolitionCard****Diagram Kelas - DiscountCard**

1.3.39. Diagram Kelas Model Card - JailFreeCard & LassoCard**Diagram Kelas - JailFreeCard****Diagram Kelas - LassoCard**

1.3.40. Diagram Kelas Model Card - MoveCard & ShieldCard**Diagram Kelas - MoveCard****Diagram Kelas - ShieldCard**

1.3.41. Diagram Kelas Model Card - SkillCard



1.3.42. Diagram Kelas Model Card - SkillCardKind & TeleportCard

Diagram Kelas - SkillCardKind

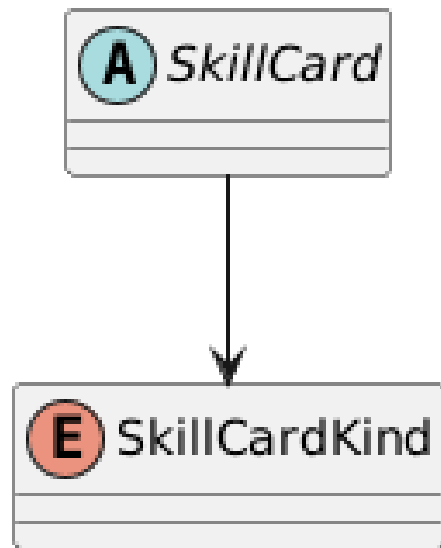
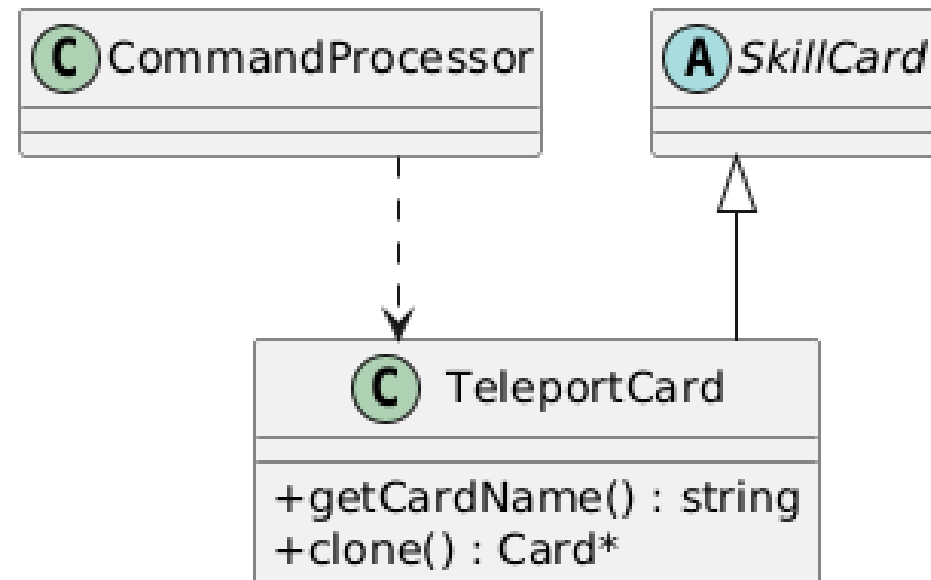
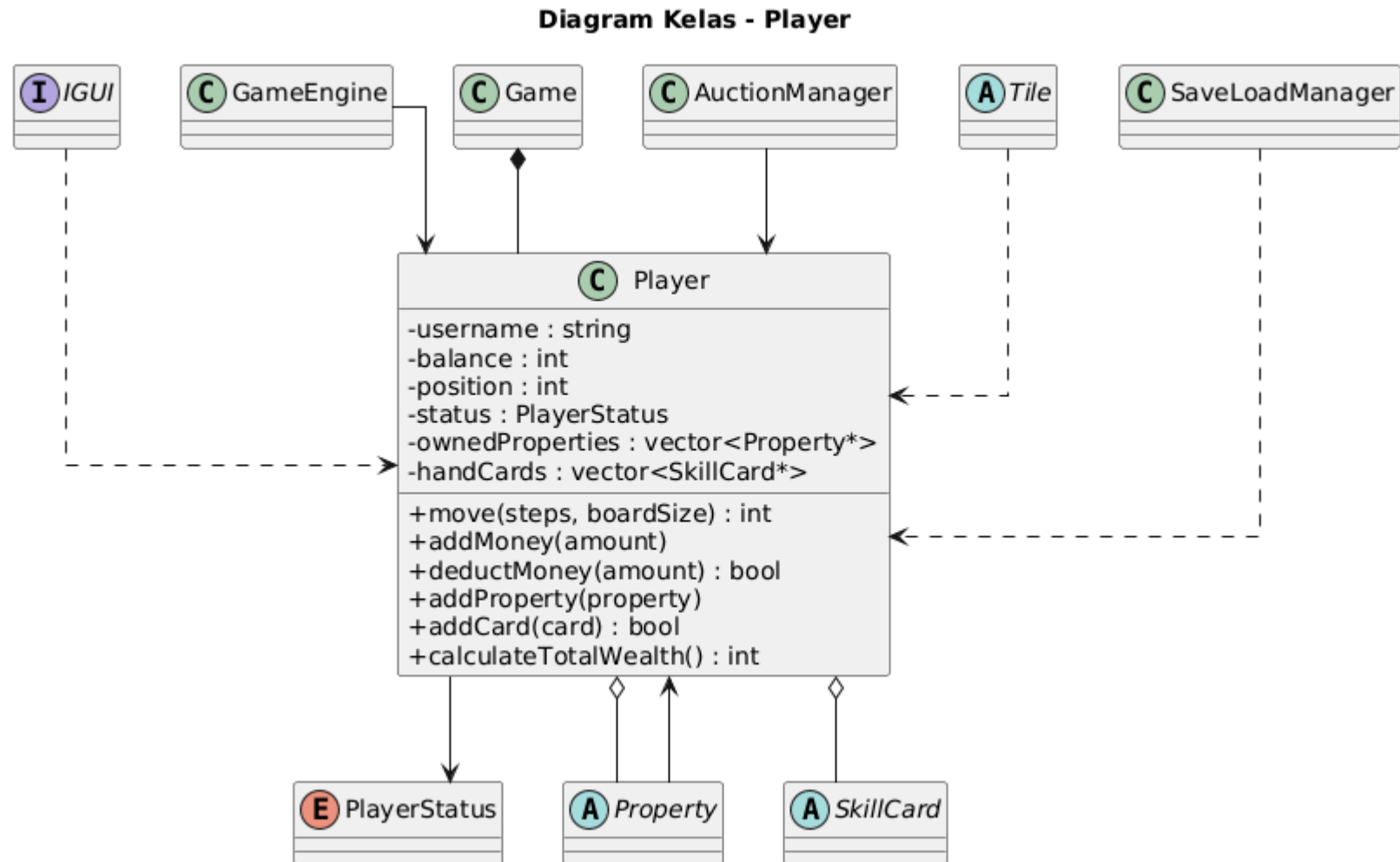


Diagram Kelas - TeleportCard

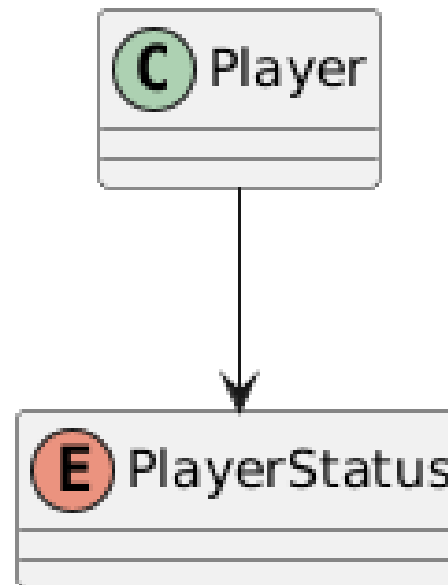


1.3.43. Diagram Kelas Model Player - Player



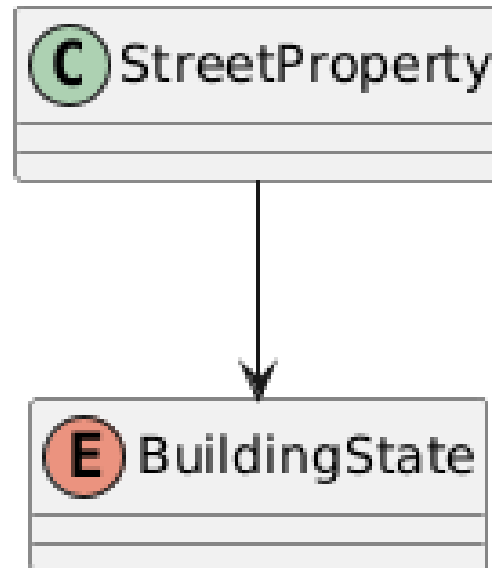
1.3.44. Diagram Kelas Model Player - PlayerStatus

Diagram Kelas - PlayerStatus

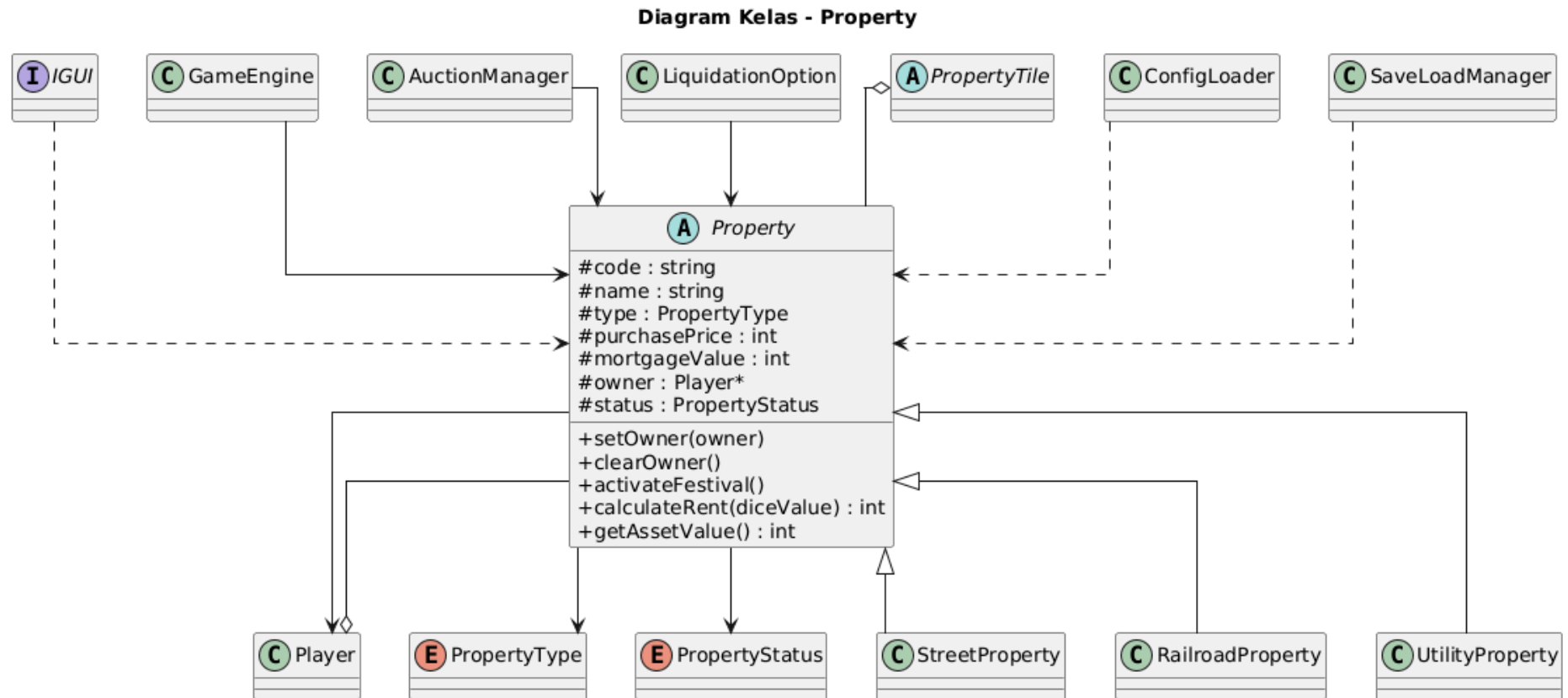


1.3.45. Diagram Kelas Model Property - BuildingState

Diagram Kelas - BuildingState



1.3.46. Diagram Kelas Model Property - Property



1.3.47. Diagram Kelas Model Property - PropertyStatus & PropertyType

Diagram Kelas - PropertyStatus

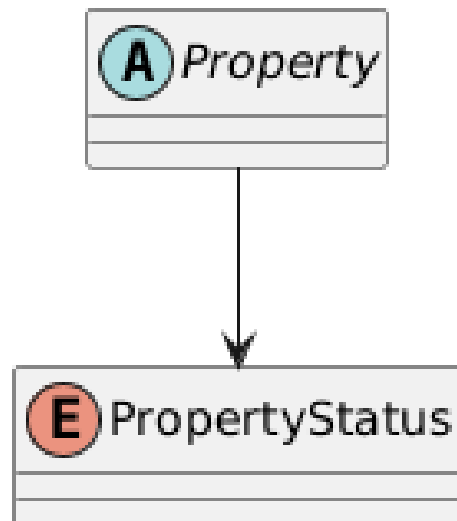
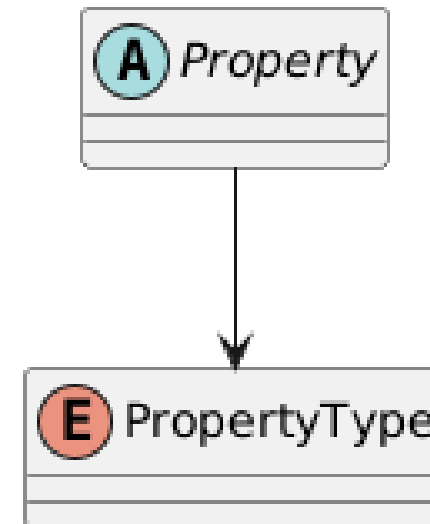


Diagram Kelas - PropertyType



1.3.48. Diagram Kelas Model Property - RailroadProperty & StreetProperty

Diagram Kelas - RailroadProperty

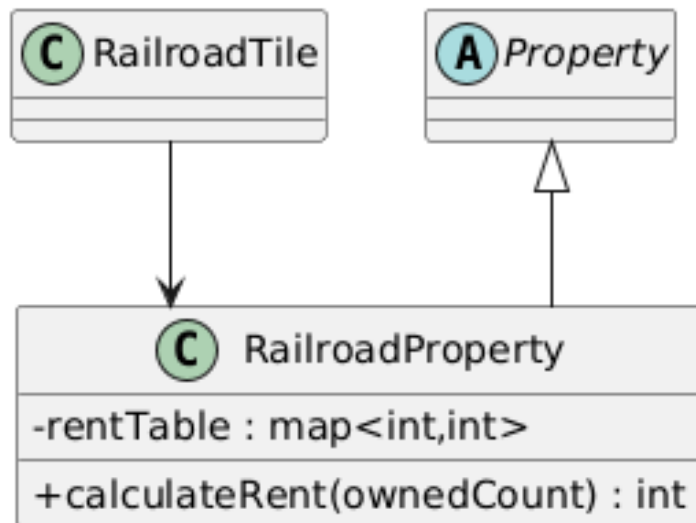
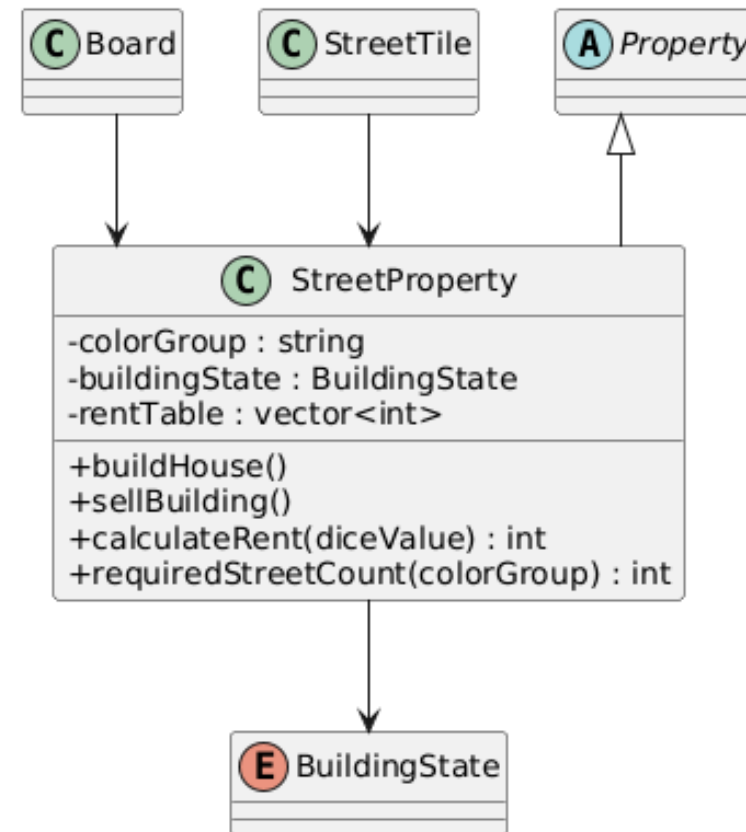
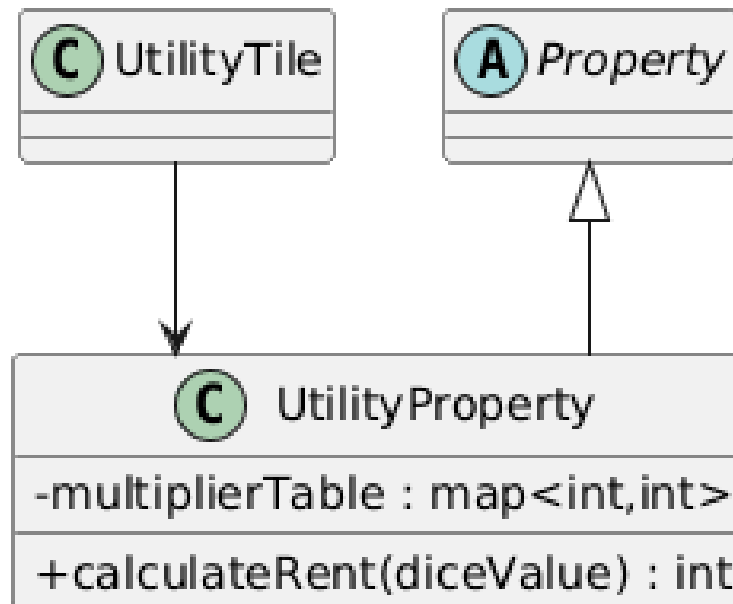


Diagram Kelas - StreetProperty



1.3.49. Diagram Kelas Model Property - UtilityProperty

Diagram Kelas - UtilityProperty



1.3.50. Diagram Kelas Model UI - CellInfo & CLIGUI

Diagram Kelas - CellInfo

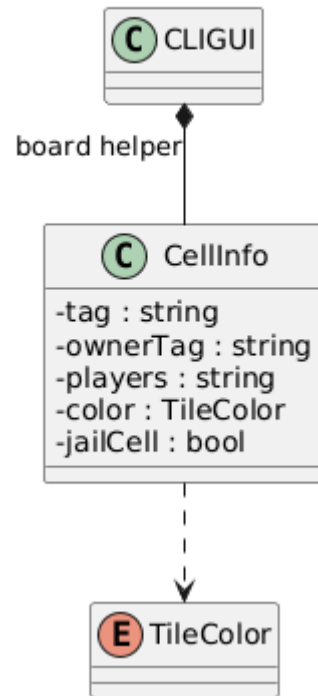
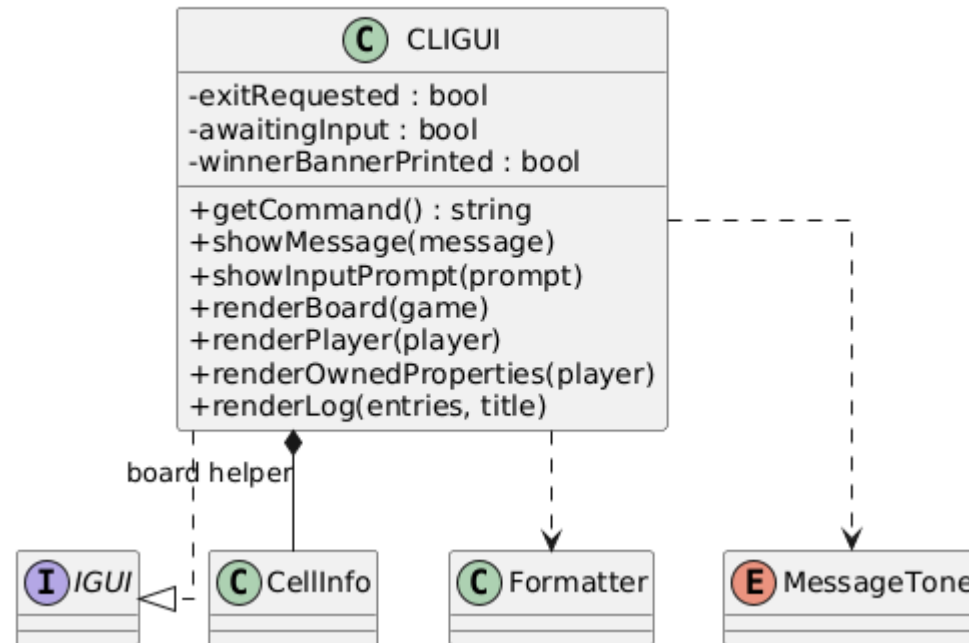


Diagram Kelas - CLIGUI



1.3.51. Diagram Kelas Model UI - Formatter & MessageTone

Diagram Kelas - Formatter

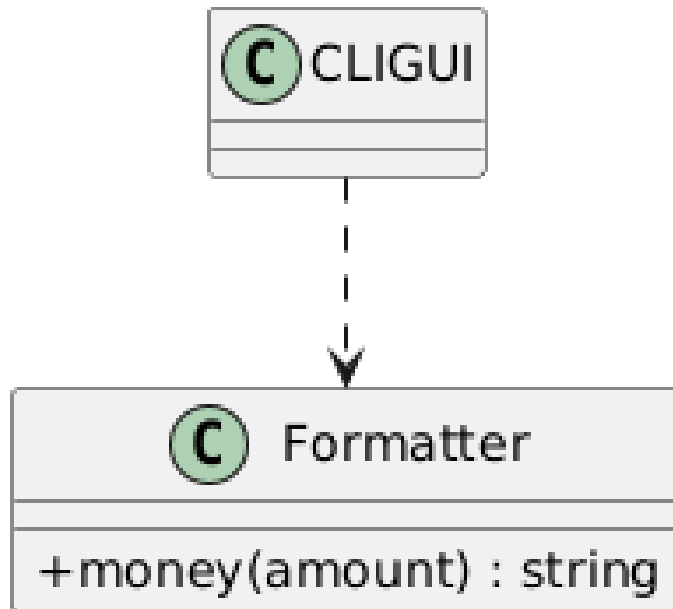
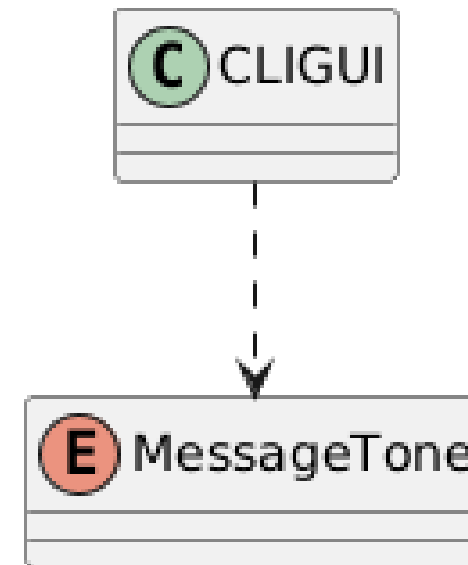
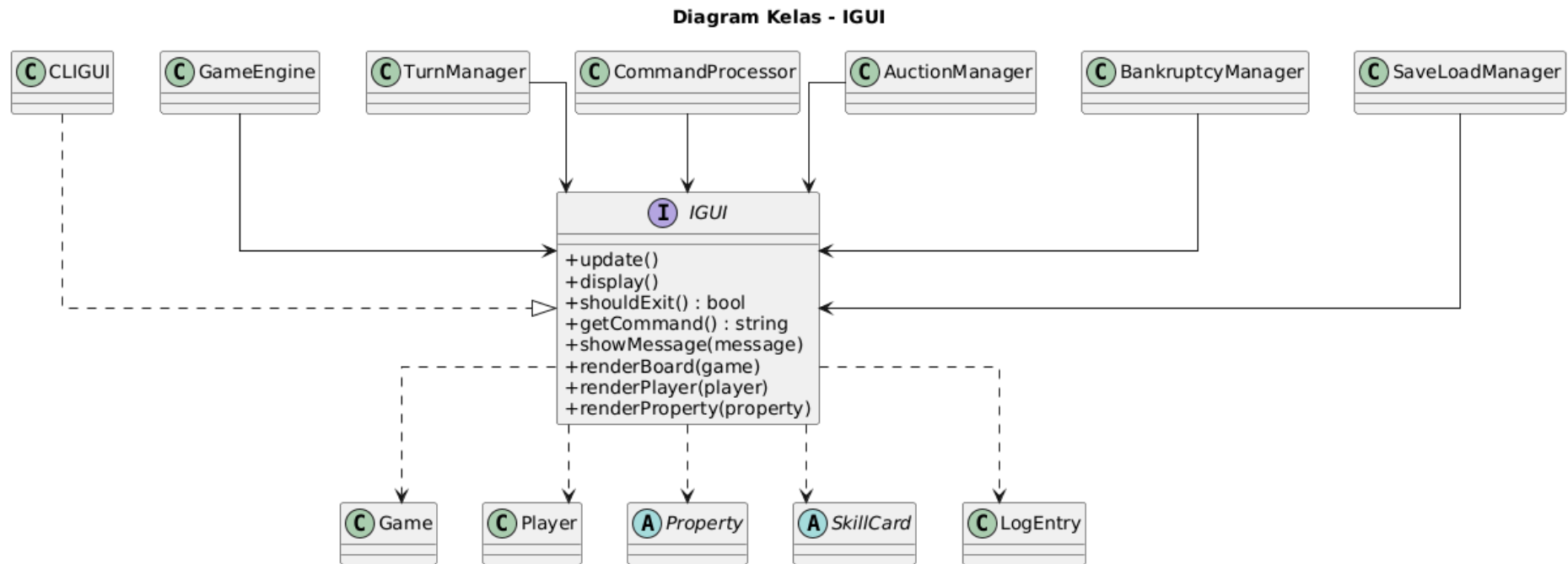


Diagram Kelas - MessageTone



1.3.52. Diagram Kelas Model UI - IGUI



2. Deskripsi Kelas

2.1. Deskripsi Kelas pada Layer Data Access

2.1.1. Deskripsi Kelas ConfigLoader

ConfigLoader bertanggung jawab mengubah file konfigurasi menjadi objek game awal. Kelas ini membaca data dari folder konfigurasi, memvalidasi format, lalu membangun properti, tile, board, dan deck. Kelas ini menyimpan direktori konfigurasi dalam atribut configDir.

Method loadGameConfig() membaca konfigurasi railroad, utility, tax, special, misc, property, dan action tile. Method buildBoard() membuat Board dari konfigurasi. Method buildDecks() membuat deck chance, community chest, dan skill. Method helper seperti loadProperties(), loadRailroadRents(), loadUtilityMultipliers(), loadActionTiles(), dan createTileForIndex() memecah proses parsing file.

Secara relasi, ConfigLoader digunakan oleh GameEngine saat membuat game baru. Kelas ini berada di data access layer karena membaca file dan membangun objek awal.

2.1.2. Deskripsi Kelas GameConfig

Kelas GameConfig bertanggung jawab menjadi container konfigurasi game. Kelas ini menyimpan konfigurasi railroad, utility, tax, special, misc, property, dan action tile. Kelas ini membuat proses inisialisasi lebih rapi karena GameEngine dan ConfigLoader tidak perlu membawa banyak variabel konfigurasi terpisah. Semua nilai dapat diakses dari satu objek konfigurasi.

Method getter pada kelas ini digunakan oleh GameEngine dan ConfigLoader untuk mengambil nilai konfigurasi yang diperlukan saat membangun game.

Secara relasi, GameConfig memiliki TaxConfig, SpecialConfig, MiscConfig, dan daftar ActionTileConfig.

2.1.3. Deskripsi Kelas ActionTileConfig

Kelas ActionTileConfig menyimpan konfigurasi dasar untuk tile aksi. Data ini diperlukan saat ConfigLoader membangun board dari file. Kelas ini menyimpan kode dan nama tile.

Secara relasi, ActionTileConfig dimiliki oleh GameConfig dan digunakan oleh ConfigLoader ketika membangun board.

2.1.4. Deskripsi Kelas TaxConfig

Kelas TaxConfig bertanggung jawab menyimpan konfigurasi pajak. TaxConfig menyimpan nilai yang berkaitan dengan pajak, seperti PPH flat, PPH persentase, dan PBM. Nilai ini digunakan saat membangun tile pajak dan saat game menghitung kewajiban pemain

Secara relasi, TaxConfig dimiliki oleh GameConfig dan digunakan saat ConfigLoader membangun IncomeTaxTile dan LuxuryTaxTile.

2.1.5. Deskripsi Kelas SpecialConfig

Kelas SpecialConfig bertanggung jawab menyimpan konfigurasi tile khusus seperti gaji GO dan denda penjara.

Secara relasi, SpecialConfig dimiliki oleh GameConfig dan digunakan saat membangun GoTile serta aturan penjara.

2.1.6. Deskripsi Kelas MiscConfig

Kelas MiscConfig bertanggung jawab menyimpan konfigurasi umum permainan, seperti saldo awal pemain dan jumlah turn maksimum.

Secara relasi, MiscConfig dimiliki oleh GameConfig dan digunakan oleh GameEngine saat inisialisasi game.

2.1.7. Deskripsi Kelas SaveLoadManager

Kelas SaveLoadManager bertanggung jawab menyimpan dan memuat state permainan. SaveLoadManager mengubah state runtime menjadi file save dan memulihkan file save menjadi state runtime. Kelas ini harus menangani banyak relasi, seperti pemain dengan properti, pemain dengan kartu, deck dengan draw/discard pile, dan board dengan tile. Kelas ini menyimpan referensi ke Game, TransactionLogger, TurnManager, IGUI, dan direktori konfigurasi sumber.

Method save() menulis state game ke folder save. Method load() membaca kembali state game. Method saveTargetExists() memeriksa apakah target save sudah ada. Method saveLogSnapshot() menyimpan log. Method helper seperti

saveConfigSnapshot(), saveLogFile(), loadLogFile(), saveDeckState(), dan findPlayerByUsername() membantu proses serialisasi dan deserialisasi.

Secara relasi, SaveLoadManager berada di data access layer tetapi membaca dan menulis state dari Game. Kelas ini tidak menentukan aturan permainan, hanya mengubah state menjadi file dan file menjadi state.

2.1.8. Deskripsi Kelas TransactionLogger

Kelas TransactionLogger bertanggung jawab mencatat riwayat aksi permainan. Kelas ini menyimpan daftar LogEntry dengan mencatat peristiwa penting selama permainan. Log ini membantu pemain memahami riwayat aksi, seperti pembelian, pembayaran, lelang, penggunaan kartu, dan perubahan status.

Method log() menambahkan log baru. Method getFullLog() mengembalikan semua log. Method getRecentLog() mengembalikan beberapa log terakhir. Method getEntryCount() mengembalikan jumlah log. Method clear() dan loadEntries() digunakan saat reset atau load game.

Secara relasi, TransactionLogger digunakan oleh GameEngine, manager core, dan SaveLoadManager.

2.1.9. Deskripsi Kelas LogEntry

Kelas LogEntry bertanggung jawab merepresentasikan satu baris log permainan. LogEntry merepresentasikan satu catatan transaksi. Informasi seperti turn, username, aksi, dan detail disimpan dalam satu objek agar mudah ditampilkan atau disimpan.

Secara relasi, LogEntry dimiliki oleh TransactionLogger dan ditampilkan oleh CLIGUI melalui command cetak log.

2.2. Deskripsi Kelas pada Layer Exception

2.2.1. Deskripsi Kelas GameException

Kelas GameException bertanggung jawab menjadi base exception untuk seluruh error khusus game. Kelas ini menyimpan pesan error dan menyediakan method what(). Kelas ini menyimpan error code dan pesan error sehingga semua exception turunan memiliki format informasi yang konsisten.

Secara relasi, semua exception khusus game diturunkan dari `GameException`, sehingga error dapat ditangkap secara umum atau spesifik.

2.2.2. Deskripsi Kelas `InvalidEntryInputException`

Kelas `InvalidEntryInputException` bertanggung jawab menjadi parent untuk error input entry yang tidak valid. Kelas ini digunakan ketika input pemain tidak memenuhi aturan dasar.

Secara relasi, kelas ini menjadi parent untuk `InvalidTileException` dan `InvalidDiceNumberException`.

2.2.3. Deskripsi Kelas `InvalidTileException`

Kelas `InvalidTileException` bertanggung jawab merepresentasikan error tile tidak valid, baik karena indeks di luar range maupun kode tile tidak ditemukan.

Secara relasi, exception ini biasanya dilempar oleh `Board::getTile()`.

2.2.4. Deskripsi Kelas `InvalidDiceNumberException`

Kelas `InvalidDiceNumberException` bertanggung jawab merepresentasikan error angka dadu tidak valid. Exception ini digunakan ketika nilai dadu manual berada di luar rentang 1 sampai 6.

Secara relasi, exception ini berhubungan dengan command `ATUR_DADU` dan validasi `DiceManager`.

2.2.5. Deskripsi Kelas `InvalidFileException`

Kelas `InvalidFileException` bertanggung jawab menjadi parent untuk error file. Kelas ini digunakan ketika operasi file gagal atau isi file tidak sesuai format.

Secara relasi, kelas ini menjadi parent untuk `FileNotFoundException`, `InvalidConfigException`, `FailedToSaveException`, dan `UnreadableFileException`.

2.2.6. Deskripsi Kelas FileNotFoundException

Kelas FileNotFoundException bertanggung jawab merepresentasikan kondisi file tidak ditemukan.

Secara relasi, exception ini digunakan oleh data access layer ketika file konfigurasi atau save tidak tersedia.

2.2.7. Deskripsi Kelas InvalidConfigException

Kelas InvalidConfigException bertanggung jawab merepresentasikan konfigurasi yang tidak valid, misalnya jumlah tile tidak sesuai atau nilai konfigurasi tidak memenuhi aturan.

Secara relasi, exception ini terutama digunakan oleh ConfigLoader dan Board::validate().

2.2.8. Deskripsi Kelas FailedToSaveException

Kelas FailedToSaveException bertanggung jawab merepresentasikan kegagalan saat menyimpan game.

Secara relasi, exception ini berhubungan dengan SaveLoadManager.

2.2.9. Deskripsi Kelas UnreadableFileException

Kelas UnreadableFileException bertanggung jawab merepresentasikan file yang ada tetapi tidak dapat dibaca.

Secara relasi, exception ini digunakan oleh data access layer saat proses pembacaan file gagal.

2.2.10. Deskripsi Kelas PlayerTurnException

Kelas PlayerTurnException bertanggung jawab menjadi parent untuk error yang berkaitan dengan giliran pemain.

Secara relasi, kelas ini menjadi parent untuk InvalidTurnException, SkillTurnException, dan PropertyManagementException.

2.2.11. Deskripsi Kelas InvalidTurnException

Kelas InvalidTurnException bertanggung jawab merepresentasikan aksi yang dilakukan pada fase giliran yang salah.

Secara relasi, exception ini berkaitan dengan TurnManager dan validasi command di CommandProcessor.

2.2.12. Deskripsi Kelas SkillTurnException

Kelas SkillTurnException bertanggung jawab menjadi parent untuk error penggunaan kartu skill.

Secara relasi, kelas ini menjadi parent untuk DiceAlreadyRolledException dan SkillCardUsedException.

2.2.13. Deskripsi Kelas DiceAlreadyRolledException

Kelas DiceAlreadyRolledException bertanggung jawab merepresentasikan kondisi pemain mencoba menggunakan skill atau aksi tertentu setelah dadu dilempar ketika aturan tidak mengizinkan.

Secara relasi, exception ini berkaitan dengan validasi TurnManager.

2.2.14. Deskripsi Kelas SkillCardUsedException

Kelas SkillCardUsedException bertanggung jawab merepresentasikan kondisi pemain mencoba menggunakan lebih dari satu kartu skill dalam satu giliran.

Secara relasi, exception ini berkaitan dengan state hasUsedSkillThisTurn pada Player.

2.2.15. Deskripsi Kelas PropertyManagementException

Kelas PropertyManagementException bertanggung jawab menjadi parent untuk error pengelolaan properti.

Secara relasi, kelas ini menjadi parent untuk exception yang muncul pada command BANGUN, GADAI, TEBUS, dan validasi properti lain.

2.2.16. Deskripsi Kelas InsufficientMoneyException

Kelas InsufficientMoneyException bertanggung jawab merepresentasikan uang pemain yang tidak cukup untuk melakukan aksi tertentu.

Secara relasi, exception ini dapat muncul saat membeli, membangun, membayar, atau menebus properti.

2.2.17. Deskripsi Kelas InsufficientOwnedColorException

Kelas InsufficientOwnedColorException bertanggung jawab merepresentasikan kondisi pemain belum memiliki color group lengkap untuk membangun.

Secara relasi, exception ini berkaitan dengan StreetProperty dan validasi monopoli.

2.2.18. Deskripsi Kelas ExistingHotelException

Kelas ExistingHotelException bertanggung jawab merepresentasikan kondisi properti sudah memiliki hotel sehingga tidak dapat dibangun lebih lanjut.

Secara relasi, exception ini digunakan pada validasi pembangunan StreetProperty.

2.2.19. Deskripsi Kelas PropertyNotMortgagedException

Kelas PropertyNotMortgagedException bertanggung jawab merepresentasikan kondisi pemain mencoba menebus properti yang tidak sedang digadai.

Secara relasi, exception ini berkaitan dengan command TEBUS dan status PropertyStatus.

2.3. Deskripsi Kelas pada Layer Game Logic

2.3.1. Deskripsi Kelas GameEngine

Kelas GameEngine bertanggung jawab mengelola keseluruhan alur permainan. Kelas ini mengatur start game, load game, loop utama, giliran pemain, lempar dadu, efek petak, pembayaran, kondisi bangkrut, kondisi menang, dan transisi antar fase. Kelas ini tidak hanya menjalankan loop utama, tetapi juga mengoordinasikan manager lain, mengatur transisi antar fase besar, dan memastikan state Game berubah sesuai aturan. Kelas ini menyimpan referensi ke Game, TransactionLogger, IGUI, DiceManager, TurnManager, CommandProcessor, AuctionManager, BankruptcyManager, dan SaveLoadManager.

Method `run()` menjalankan alur utama program. Method `initNewGame()` memuat konfigurasi melalui `ConfigLoader`, membuat Board, deck kartu, pemain, dan urutan giliran. Method `gameLoop()` menjalankan permainan sampai game selesai atau jumlah turn maksimum tercapai. Method `processPlayerTurn()` menangani satu giliran pemain aktif. Method `resolveRoll()` memproses hasil dadu, perpindahan pemain, bonus double, dan double ketiga. Method `handleTileLanding()` menjalankan efek tile setelah pemain mendarat.

Method `executePayment()` menangani pembayaran dari satu pemain ke pemain lain atau ke bank. Jika uang pemain tidak cukup, method ini mendelegasikan proses likuidasi ke `BankruptcyManager`. Method `checkWinCondition()` mengecek apakah permainan sudah memiliki pemenang. Method `requestLoad()` dan `performPendingLoad()` mengatur proses load game yang diminta melalui command.

Secara relasi, `GameEngine` adalah kerangka utama game logic. Kelas ini memiliki manager-manager core dan menjadi penghubung antara UI, state game, data access, serta model. Detail parsing command, fase giliran, lelang, bangkrut, dan save/load tidak dikerjakan langsung sepenuhnya di kelas ini, tetapi didelegasikan ke kelas yang lebih spesifik.

2.3.2. Deskripsi Kelas Game

Kelas `Game` bertanggung jawab menyimpan state utama permainan. Kelas ini menyimpan objek dan konfigurasi yang mendefinisikan kondisi game saat ini, seperti daftar pemain, board, deck chance, deck community chest, deck skill, urutan giliran, indeks giliran aktif, nomor turn, nilai dadu terakhir, status game over, serta konfigurasi runtime seperti saldo awal, gaji GO, denda penjara, pajak, sewa railroad, dan multiplier utility.

Method `setBoard()`, `setDecks()`, `addPlayer()`, `setTurnOrder()`, dan `setConfigValues()` digunakan saat inisialisasi atau load game. Method `getCurrentPlayer()` mengembalikan pemain aktif berdasarkan turn order. Method `getActivePlayers()` mengembalikan pemain yang belum bangkrut. Method `advanceTurnOrder()` memindahkan giliran ke pemain berikutnya. Method `incrementTurn()` menaikkan ronde permainan. Method getter lain memberikan akses ke board, deck, pemain, dan konfigurasi.

Secara relasi, `Game` dimiliki oleh `GameEngine` dan menjadi pusat state domain. Kelas lain seperti `TurnManager`, `CommandProcessor`, `AuctionManager`, `BankruptcyManager`, dan `SaveLoadManager` membaca atau memperbarui state melalui objek `Game`. Kelas ini tidak menangani input/output.

2.3.3. Deskripsi Kelas DiceManager

Kelas DiceManager bertanggung jawab menyimpan dan menghasilkan nilai dadu. Kelas ini menyimpan die1, die2, random engine, dan distribusi angka dadu.

Method rollRandom() menghasilkan dua nilai dadu acak. Method setManual() menetapkan nilai dadu secara manual untuk kebutuhan command testing. Method getDie1(), getDie2(), dan getTotal() mengembalikan nilai dadu. Method isDouble() mengecek apakah dua dadu bernilai sama.

Secara relasi, DiceManager dimiliki oleh GameEngine dan digunakan oleh TurnManager serta CommandProcessor. Kelas ini memisahkan mekanisme dadu dari logic giliran dan command.

2.3.4. Deskripsi Kelas TurnManager

Kelas TurnManager bertanggung jawab mengatur fase dalam satu giliran pemain. Kelas ini memastikan pemain hanya dapat melakukan aksi pada fase yang benar, seperti melempar dadu sebelum giliran berakhir atau menggunakan skill sebelum dadu dilempar. Kelas ini menyimpan referensi ke Game, DiceManager, IGUI, dan TransactionLogger. Kelas ini juga menyimpan phase, hasActed, dan shieldActive untuk memvalidasi aksi dalam giliran.

Method startTurn() menyiapkan state giliran baru. Method drawSkillCardForTurn() memberikan kartu skill di awal giliran. Method endTurn() menutup giliran dan memperbarui efek durasi seperti festival. Method canRoll() memeriksa apakah pemain boleh melempar dadu. Method canUseSkill() memeriksa apakah pemain boleh menggunakan kartu skill. Method processMovement() menghitung posisi baru pemain dan mengembalikan tile tujuan.

Secara relasi, TurnManager menjadi sumber validasi fase bagi CommandProcessor dan GameEngine. Kelas ini tidak mem-parsing command dan tidak menjalankan semua efek tile, tetapi mengatur status giliran agar aturan turn flow tetap konsisten.

2.3.5. Deskripsi Kelas TurnPhase

Enum TurnPhase bertanggung jawab merepresentasikan fase giliran. Nilainya mencakup fase awal, menunggu lempar dadu, setelah lempar dadu, dan giliran selesai.

Enum ini digunakan oleh TurnManager untuk menentukan apakah aksi seperti lempar dadu, simpan, menggunakan kartu, atau mengakhiri giliran boleh dilakukan.

Secara relasi, TurnPhase melekat pada TurnManager sebagai state validasi giliran.

2.3.6. Deskripsi Kelas CommandProcessor

Kelas CommandProcessor bertanggung jawab menerima string perintah mentah dari IGUI, memecahnya menjadi token, menormalisasi nama command, memvalidasi kondisi giliran, lalu menjalankan handler command yang sesuai. Kelas ini menyimpan referensi ke GameEngine, Game, TurnManager, DiceManager, dan IGUI.

Method process() menjadi pintu masuk utama untuk semua command manual. Method ini menangani command seperti LEMPAR_DADU, ATUR_DADU, CETAK_PAPAN, CETAK_AKTA, CETAK_PROPERTI, CETAK_LOG, GADAI, TEBUS, BANGUN, GUNAKAN_KARTU, BUANG_KARTU, SIMPAN, LOAD_GAME, FESTIVAL, HELP, dan AKHIRI_GILIRAN. Method tokenize() memecah input, sedangkan normalize() menyamakan format command.

Method handler seperti handleRoll(), handleMortgage(), handleRedeem(), handleBuild(), handleUseSkill(), handleSave(), handleLoad(), handleFestival(), dan handleEndTurn() menjalankan aksi spesifik. Untuk kartu, method applyMoveCard(), applyTeleportCard(), applyDiscountCard(), applyShieldCard(), applyLassoCard(), dan applyDemolitionCard() memproses efek setelah validasi target.

Secara relasi, CommandProcessor dimiliki oleh GameEngine dan menjadi penghubung antara input CLI dan core logic. Kelas ini berinteraksi dengan model hanya ketika command membutuhkan objek target seperti properti, pemain, tile, atau kartu.

2.3.7. Deskripsi Kelas CommandResult

Enum CommandResult bertanggung jawab menyatakan hasil pemrosesan command. Nilainya digunakan untuk memberi tahu GameEngine apakah command membuat giliran berlanjut, berakhir, game selesai, save dilakukan di tengah giliran, load game diminta, atau command invalid.

Secara relasi, CommandResult dikembalikan oleh CommandProcessor::process() dan dibaca oleh GameEngine saat menentukan langkah berikutnya dalam loop giliran.

2.3.8. Deskripsi Kelas AuctionManager

Kelas AuctionManager bertanggung jawab menjalankan mekanisme lelang properti. Kelas ini menyimpan referensi ke Game, TransactionLogger, dan IGUI.

Method runAuction() menjalankan proses lelang dari awal sampai akhir. Method buildAuctionOrder() menentukan urutan peserta lelang. Method collectBidOrPass() meminta input dari pemain. Method validateBid() memeriksa apakah bid valid dan pemain mampu membayar. Method finalizeAuction() memindahkan kepemilikan properti ke pemenang dan mencatat transaksi.

Secara relasi, AuctionManager digunakan oleh GameEngine ketika properti tidak dibeli atau pemain tidak mampu membeli. BankruptcyManager juga dapat berhubungan dengan AuctionManager saat likuidasi aset memerlukan lelang.

2.3.9. Deskripsi Kelas AuctionAction

Enum AuctionAction merepresentasikan keputusan pemain dalam lelang. Nilainya membuat hasil input lebih jelas, yaitu pemain melakukan bid, pass, atau input tidak valid.

Enum ini digunakan oleh AuctionManager ketika membaca input peserta lelang dan menentukan apakah lelang berlanjut, berpindah pemain, atau selesai.

2.3.10. Deskripsi Kelas BankruptcyManager

Kelas BankruptcyManager mengelola seluruh proses ketika pemain tidak mampu membayar. Kelas ini mengecek apakah pemain masih dapat menyelamatkan diri melalui likuidasi, menawarkan opsi jual atau gadai, lalu memutuskan apakah pemain bangkrut.. Kelas ini menyimpan referensi ke Game, TransactionLogger, IGUI, dan AuctionManager.

Method handleInsufficientFunds() menjadi pintu masuk saat pemain kekurangan uang. Method calculateMaxLiquidation() menghitung potensi likuidasi maksimum. Method runLiquidationPanel() menyediakan opsi likuidasi kepada pemain. Method sellPropertyToBank() dan mortgageProperty() menjalankan aksi likuidasi. Method declareBankruptcy() mengubah status pemain menjadi bangkrut. Method transferAssetsToPlayer() dan returnAssetsToBank() mengurus aset setelah pemain bangkrut.

Secara relasi, BankruptcyManager dipanggil oleh GameEngine melalui mekanisme pembayaran. Kelas ini juga menggunakan AuctionManager untuk proses yang membutuhkan lelang.

2.3.11. Deskripsi Kelas LiquidationOption

Kelas LiquidationOption bertanggung jawab merepresentasikan satu opsi likuidasi. Kelas ini menyimpan jenis opsi, properti terkait, dan nilai uang yang dapat diperoleh.

Kelas ini digunakan oleh BankruptcyManager untuk membangun daftar opsi jual atau gadai yang dapat dipilih pemain ketika kekurangan uang.

2.3.12. Deskripsi Kelas Kind

Enum Kind bertanggung jawab membedakan jenis LiquidationOption, yaitu menjual aset atau menggadaikan aset.

Enum ini hanya digunakan secara internal oleh BankruptcyManager dalam proses likuidasi.

2.3.13. Deskripsi Kelas PendingProp

Kelas PendingProp bertanggung jawab menyimpan data sementara properti saat proses load game. Kelas ini digunakan ketika data properti sudah dibaca dari file, tetapi relasi ke objek pemilik atau properti masih perlu disambungkan setelah semua objek dibuat.

Secara relasi, PendingProp digunakan oleh SaveLoadManager sebagai helper deserialisasi.

2.3.14. Deskripsi Kelas PendingHandCard

Kelas PendingHandCard bertanggung jawab menyimpan data sementara kartu tangan pemain saat proses load game. Kelas ini menyimpan informasi pemilik kartu dan nama kartu yang harus dikembalikan ke tangan pemain.

Secara relasi, PendingHandCard digunakan oleh SaveLoadManager untuk membangun ulang relasi antara Player dan SkillCard.

2.3.15. Deskripsi Kelas PendingDeckState

Kelas PendingDeckState bertanggung jawab menyimpan urutan sementara draw pile dan discard pile saat load game. Kelas ini digunakan agar state deck dapat dikembalikan sesuai file save.

Secara relasi, PendingDeckState digunakan oleh SaveLoadManager untuk membangun ulang CardDeck.

2.4. Deskripsi Kelas pada Layer Model

2.4.1. Deskripsi Kelas Player

Kelas Player bertanggung jawab merepresentasikan pemain. Kelas ini menyimpan username, saldo, posisi, status, daftar properti, kartu di tangan, jumlah double berturut-turut, percobaan keluar penjara, status sudah lempar dadu, status sudah memakai skill, festival pending, dan diskon pending.

Method move() dan setPosition() mengatur posisi pemain. Method addMoney(), deductMoney(), dan canAfford() mengatur saldo. Method addProperty(), removeProperty(), dan getOwnedProperties() mengatur kepemilikan properti. Method ownsFullColorGroup() mengecek monopoli. Method calculatePropertyAssetValue(), calculateBuildingAssetValue(), dan calculateTotalWealth() menghitung kekayaan pemain. Method addCard(), removeCard(), findJailFreeCard(), dan getHandCards() mengatur kartu skill pemain. Method startTurn(), markRolled(), markSkillUsed(), hasRolled(), dan hasUsedSkill() menjaga state giliran.

Secara relasi, Player dimiliki oleh Game, memiliki banyak Property, dan memiliki beberapa SkillCard. Banyak manager core memanipulasi state Player melalui method yang tersedia.

2.4.2. Deskripsi Kelas PlayerStatus

Enum PlayerStatus bertanggung jawab menyatakan status pemain. Nilainya mencakup aktif, jailed, dan bankrupt. Enum ini digunakan oleh Player, Game, GameEngine, TurnManager, dan BankruptcyManager untuk menentukan apakah pemain boleh bergerak, harus menjalani aturan penjara, atau sudah keluar dari permainan.

2.4.3. Deskripsi Kelas Board

Kelas Board bertanggung jawab merepresentasikan papan permainan. Kelas ini menyimpan daftar Tile, mapping kode tile ke objek tile, ukuran board, pointer ke GoTile, dan pointer ke JailTile.

Method `addTile()` menambahkan tile. Method `getTile(int)` mengambil tile berdasarkan indeks. Method `getTile(string)` mengambil tile berdasarkan kode. Method `getNextIndex()` menghitung indeks tujuan setelah bergerak. Method `passesGo()` menentukan apakah pemain melewati GO. Method `getNearestRailroad()` mencari railroad terdekat. Method `getStreetGroup()` mengambil semua street pada color group tertentu. Method `validate()` memvalidasi struktur board.

Secara relasi, Board dimiliki oleh Game dan berisi banyak Tile. TurnManager, GameEngine, CommandProcessor, dan SaveLoadManager menggunakan board untuk navigasi dan pencarian objek.

2.4.4. Deskripsi Kelas Tile

Kelas Tile bertanggung jawab menjadi abstraksi dasar seluruh petak papan. Kelas ini menyimpan indeks, kode, nama, dan warna tile.

Method `onLanded()` diimplementasikan oleh turunan untuk menentukan efek saat pemain mendarat. Method `onPassed()` dapat dioverride jika tile memiliki efek saat dilewati. Method `getCategory()` dan `getKind()` memberi informasi kategori dan jenis tile.

Secara relasi, Tile menjadi parent untuk PropertyTile, ActionTile, dan SpecialTile. Polymorphism ini membuat board dapat menyimpan seluruh jenis petak dalam satu vector `Tile*`.

2.4.5. Deskripsi Kelas TileColor, TileCategory, dan TileKind

Enum `TileColor`, `TileCategory`, dan `TileKind` bertanggung jawab memberi identitas visual dan kategori logic pada tile. `TileColor` digunakan untuk warna atau color group tampilan, `TileCategory` membedakan property/action/special, dan `TileKind` membedakan jenis tile secara lebih spesifik.

Secara relasi, ketiga enum ini digunakan oleh Tile, Board, CLIGUI, dan logic yang perlu membedakan tipe petak.

2.4.6. Deskripsi Kelas PropertyTile

Kelas PropertyTile bertanggung jawab menjadi tile yang memiliki objek Property. Kelas ini menyimpan pointer ke properti dan menyediakan method getProperty().

Secara relasi, PropertyTile menjadi parent untuk StreetTile, RailroadTile, dan UtilityTile. Kelas ini menghubungkan tile pada board dengan aset yang dapat dimiliki pemain.

2.4.7. Deskripsi Kelas StreetTile

Kelas StreetTile bertanggung jawab merepresentasikan petak jalan. Kelas ini membungkus StreetProperty dan digunakan ketika pemain mendarat di properti street.

Secara relasi, StreetTile merupakan turunan PropertyTile dan berhubungan langsung dengan StreetProperty. Efek beli, sewa, dan bangun pada street diproses oleh core menggunakan property yang terdapat pada tile ini.

2.4.8. Deskripsi Kelas RailroadTile

Kelas RailroadTile bertanggung jawab merepresentasikan petak railroad. Kelas ini membungkus RailroadProperty.

Secara relasi, RailroadTile merupakan turunan PropertyTile dan digunakan oleh Board serta GameEngine ketika pemain mendarat di railroad atau ketika kartu memindahkan pemain ke railroad terdekat.

2.4.9. Deskripsi Kelas UtilityTile

Kelas UtilityTile bertanggung jawab merepresentasikan petak utility. Kelas ini membungkus UtilityProperty.

Secara relasi, UtilityTile merupakan turunan PropertyTile dan digunakan oleh core ketika menghitung sewa utility berdasarkan hasil dadu.

2.4.10. Deskripsi Kelas ActionTile

Kelas ActionTile bertanggung jawab menjadi parent untuk tile yang memicu aksi non-properti. Kelas ini merupakan turunan dari Tile.

Secara relasi, ActionTile menjadi parent untuk ChanceTile, CommunityChestTile, FestivalTile, dan TaxTile. Kelas ini memisahkan tile aksi dari tile properti dan tile khusus.

2.4.11. Deskripsi Kelas ChanceTile

Kelas ChanceTile bertanggung jawab merepresentasikan petak chance. Saat pemain mendarat di tile ini, game mengambil efek dari deck chance.

Secara relasi, ChanceTile digunakan oleh GameEngine untuk memicu ChanceCard dari CardDeck<ChanceCard>.

2.4.12. Deskripsi Kelas CommunityChestTile

Kelas CommunityChestTile bertanggung jawab merepresentasikan petak dana umum. Saat pemain mendarat di tile ini, game mengambil efek dari deck community chest.

Secara relasi, CommunityChestTile digunakan oleh GameEngine untuk memicu CommunityChestCard dari CardDeck<CommunityChestCard>.

2.4.13. Deskripsi Kelas FestivalTile

Kelas FestivalTile bertanggung jawab merepresentasikan petak festival. Tile ini mengaktifkan mekanisme festival yang dapat memengaruhi multiplier sewa properti.

Secara relasi, FestivalTile digunakan oleh GameEngine dan CommandProcessor ketika pemain mendapat kesempatan memilih properti untuk festival.

2.4.14. Deskripsi Kelas TaxTile, IncomeTaxTile, dan LuxuryTaxTile

Kelas TaxTile bertanggung jawab menjadi parent untuk petak pajak. IncomeTaxTile merepresentasikan pajak PPH, sedangkan LuxuryTaxTile merepresentasikan pajak PBM.

Method pada tile pajak menyediakan nilai pajak yang harus dibayar. Pembayaran tetap diproses oleh GameEngine melalui mekanisme executePayment() agar konsisten dengan pembayaran lain.

Secara relasi, tax tile berada di board dan digunakan saat landing effect.

2.4.15. Deskripsi Kelas SpecialTile

Kelas SpecialTile bertanggung jawab menjadi parent untuk petak khusus yang bukan property dan bukan action tile biasa.

Secara relasi, SpecialTile menjadi parent untuk GoTile, JailTile, FreeParkingTile, dan GoToJailTile.

2.4.16. Deskripsi Kelas GoTile

Kelas GoTile bertanggung jawab merepresentasikan petak awal atau GO. Kelas ini menyimpan nilai gaji GO yang dapat diberikan ketika pemain melewati atau mendarat pada GO.

Secara relasi, GoTile disimpan secara khusus oleh Board agar mekanisme passesGo() dan pemberian gaji dapat dilakukan dengan mudah.

2.4.17. Deskripsi Kelas JailTile

Kelas JailTile bertanggung jawab merepresentasikan petak penjara. Tile ini dapat berarti pemain hanya mampir atau sedang berada dalam status jailed tergantung status pemain.

Secara relasi, JailTile disimpan oleh Board dan digunakan oleh GameEngine saat pemain masuk penjara.

2.4.18. Deskripsi Kelas FreeParkingTile

Kelas FreeParkingTile bertanggung jawab merepresentasikan petak parkir bebas. Petak ini tidak memiliki efek pembayaran atau perpindahan khusus.

Secara relasi, kelas ini tetap menjadi turunan SpecialTile agar seluruh petak khusus memiliki struktur yang konsisten.

2.4.19. Deskripsi Kelas GoToJailTile

Kelas GoToJailTile bertanggung jawab memindahkan pemain ke penjara saat mendarat. Kelas ini mengubah posisi pemain ke JailTile dan mengubah status pemain menjadi jailed.

Secara relasi, GoToJailTile membutuhkan akses ke Board melalui Game untuk mengetahui posisi jail.

2.4.20. Deskripsi Kelas Property

Kelas Property bertanggung jawab menjadi abstraksi dasar semua aset yang dapat dimiliki pemain. Kelas ini menyimpan kode, nama, tipe, harga beli, nilai gadai, owner, status, multiplier festival, dan durasi festival.

Method setOwner(), clearOwner(), dan setStatus() mengatur kepemilikan. Method activateFestival(), decrementFestivalDuration(), dan setFestivalState() mengatur efek festival. Method calculateRent() dan getAssetValue() bersifat virtual karena aturan sewa dan nilai aset berbeda untuk setiap jenis properti.

Secara relasi, Property menjadi parent untuk StreetProperty, RailroadProperty, dan UtilityProperty. Objek property dapat dimiliki oleh Player dan dibungkus oleh PropertyTile.

2.4.21. Deskripsi Kelas PropertyStatus dan PropertyType

Enum PropertyStatus bertanggung jawab menyatakan status properti, seperti bank, owned, atau mortgaged. Enum PropertyType bertanggung jawab menyatakan jenis properti, seperti street, railroad, atau utility.

Kedua enum ini digunakan oleh Property, CommandProcessor, BankruptcyManager, SaveLoadManager, dan tampilan CLI untuk validasi serta output.

2.4.22. Deskripsi Kelas StreetProperty

Kelas StreetProperty bertanggung jawab merepresentasikan properti jalan. Kelas ini menyimpan color group, state bangunan, tabel sewa, dan informasi biaya bangunan.

Method buildHouse() menambah bangunan. Method sellBuilding() menjual atau menurunkan bangunan. Method calculateRent() menghitung sewa berdasarkan bangunan, monopoli, status gadai, dan festival. Method requiredStreetCount() menentukan jumlah street yang diperlukan untuk monopoli color group.

Secara relasi, StreetProperty digunakan oleh StreetTile, dimiliki oleh Player, dan diproses oleh command seperti BANGUN, GADAI, TEBUS, dan FESTIVAL.

2.4.23. Deskripsi Kelas BuildingState

Enum BuildingState bertanggung jawab menyatakan tingkat bangunan pada StreetProperty, mulai dari tanpa bangunan, rumah satu sampai empat, hingga hotel.

Enum ini digunakan oleh StreetProperty, CommandProcessor, BankruptcyManager, dan SaveLoadManager untuk validasi bangun, jual bangunan, dan serialisasi.

2.4.24. Deskripsi Kelas RailroadProperty

Kelas RailroadProperty bertanggung jawab merepresentasikan properti railroad. Kelas ini menyimpan tabel sewa berdasarkan jumlah railroad yang dimiliki owner.

Method calculateRent() menghitung sewa railroad berdasarkan jumlah railroad owner. Method getAssetValue() mengembalikan nilai aset.

Secara relasi, RailroadProperty digunakan oleh RailroadTile dan dimiliki oleh Player.

2.4.25. Deskripsi Kelas UtilityProperty

Kelas UtilityProperty bertanggung jawab merepresentasikan properti utility. Kelas ini menyimpan tabel multiplier berdasarkan jumlah utility yang dimiliki owner.

Method calculateRent() menghitung sewa berdasarkan total dadu dan multiplier utility. Method getAssetValue() mengembalikan nilai aset.

Secara relasi, UtilityProperty digunakan oleh UtilityTile, GameEngine, dan Player saat menghitung sewa.

2.4.26. Deskripsi Kelas Card

Kelas Card bertanggung jawab menjadi abstraksi dasar seluruh kartu. Kelas ini menyimpan deskripsi kartu.

Method getDescription() mengembalikan deskripsi. Method getCardName(), getCategory(), dan clone() bersifat virtual agar setiap jenis kartu dapat menyediakan identitas dan salinan sendiri.

Secara relasi, Card menjadi parent untuk ActionCard dan SkillCard.

2.4.27. Deskripsi Kelas CardCategory

Enum CardCategory bertanggung jawab membedakan kategori kartu, yaitu chance, community, dan skill.

Enum ini digunakan oleh Card, deck, dan proses save/load untuk mengenali kelompok kartu.

2.4.28. Deskripsi Kelas ActionCard

Kelas ActionCard bertanggung jawab menjadi parent untuk kartu aksi otomatis. Kelas ini merupakan turunan Card.

Secara relasi, ActionCard menjadi parent untuk ChanceCard dan CommunityChestCard. Kartu jenis ini tidak digunakan langsung oleh pemain melalui command skill.

2.4.29. Deskripsi Kelas ChanceCard dan ChanceType

Kelas ChanceCard bertanggung jawab menyimpan efek kartu chance. Enum ChanceType menyatakan jenis efek chance yang tersedia.

Method getCardName() mengembalikan nama kartu, dan clone() membuat salinan kartu. Efek kartu diproses oleh GameEngine saat pemain mendarat di ChanceTile.

Secara relasi, ChanceCard berada di CardDeck<ChanceCard> milik Game.

2.4.30. Deskripsi Kelas CommunityChestCard dan CommunityType

Kelas CommunityChestCard bertanggung jawab menyimpan efek kartu dana umum. Enum CommunityType menyatakan jenis efek community chest yang tersedia.

Method getCardName() mengembalikan nama kartu, dan clone() membuat salinan kartu. Efek kartu diproses oleh GameEngine saat pemain mendarat di CommunityChestTile.

Secara relasi, CommunityChestCard berada di CardDeck<CommunityChestCard> milik Game.

2.4.31. Deskripsi Kelas SkillCard dan SkillCardKind

Kelas SkillCard bertanggung jawab menjadi parent seluruh kartu kemampuan pemain. Kelas ini menyimpan jumlah penggunaan tersisa. Enum SkillCardKind menyatakan jenis skill card.

Method isUsable() memeriksa apakah kartu masih dapat digunakan. Method isJailFreeCard() membedakan kartu bebas penjara. Method getKind() mengembalikan jenis kartu. Method getPrimaryValue() dan getDurationValue() menyediakan nilai tambahan untuk kartu tertentu.

Secara relasi, SkillCard menjadi parent untuk MoveCard, TeleportCard, DiscountCard, ShieldCard, LassoCard, DemolitionCard, dan JailFreeCard. Kartu ini dimiliki oleh Player dan diproses oleh CommandProcessor.

2.4.32. Deskripsi Kelas MoveCard

Kelas MoveCard bertanggung jawab merepresentasikan kartu skill untuk memindahkan pemain. Kelas ini menyimpan nilai langkah atau parameter utama melalui turunan SkillCard.

Secara relasi, MoveCard digunakan oleh CommandProcessor::applyMoveCard() untuk memindahkan pemain dan memicu efek tile tujuan.

2.4.33. Deskripsi Kelas TeleportCard

Kelas TeleportCard bertanggung jawab merepresentasikan kartu skill untuk berpindah langsung ke tile tertentu.

Secara relasi, TeleportCard digunakan oleh CommandProcessor::applyTeleportCard(). Jika ada beberapa tile dengan kode yang sama, command processor meminta pemain memilih tujuan.

2.4.34. Deskripsi Kelas DiscountCard

Kelas DiscountCard bertanggung jawab merepresentasikan kartu diskon. Kartu ini memberi efek diskon pada transaksi tertentu, seperti pembelian atau pembangunan, sesuai aturan implementasi.

Secara relasi, DiscountCard mengubah state diskon pending pada Player melalui CommandProcessor.

2.4.35. Deskripsi Kelas ShieldCard

Kelas ShieldCard bertanggung jawab merepresentasikan kartu perlindungan. Kartu ini mengaktifkan status shield yang dapat mencegah efek negatif tertentu.

Secara relasi, ShieldCard diproses oleh CommandProcessor dan status aktifnya disimpan melalui TurnManager.

2.4.36. Deskripsi Kelas LassoCard

Kelas LassoCard bertanggung jawab merepresentasikan kartu yang menarik atau memindahkan pemain lain ke posisi tertentu sesuai aturan.

Secara relasi, LassoCard digunakan oleh CommandProcessor dengan target pemain lain. Kelas ini membutuhkan validasi agar pemain tidak menargetkan diri sendiri atau pemain invalid.

2.4.37. Deskripsi Kelas DemolitionCard

Kelas DemolitionCard bertanggung jawab merepresentasikan kartu yang menghancurkan bangunan milik lawan.

Secara relasi, DemolitionCard digunakan oleh CommandProcessor. Command processor mencari properti lawan yang memiliki bangunan, tidak digadai, dan valid sebagai target. Jika efek gagal karena tidak ada target, kartu tidak seharusnya dianggap berhasil digunakan.

2.4.38. Deskripsi Kelas JailFreeCard

Kelas JailFreeCard bertanggung jawab merepresentasikan kartu bebas penjara. Kartu ini dapat digunakan pemain untuk keluar dari status jailed tanpa membayar denda atau berhasil double.

Secara relasi, JailFreeCard dapat dicari melalui Player::findJailFreeCard() ketika pemain sedang berada di penjara.

2.4.39. Deskripsi Kelas CardDeck

Kelas template CardDeck<T> bertanggung jawab mengelola draw pile dan discard pile kartu. Kelas ini digunakan untuk deck chance, community chest, dan skill card.

Method addCard() menambahkan kartu. Method draw() mengambil kartu dari draw pile. Method discard() memasukkan kartu ke discard pile. Method shuffle() mengacak deck. Method reshuffleIfEmpty() memindahkan discard pile menjadi draw pile ketika draw pile kosong. Method takeByName() digunakan saat load game perlu mengambil kartu tertentu berdasarkan nama.

Secara relasi, CardDeck dimiliki oleh Game dan digunakan oleh TurnManager, GameEngine, serta SaveLoadManager.

2.5. Deskripsi Kelas pada Layer UI

2.5.1. Deskripsi Kelas IGUI

Kelas IGUI bertanggung jawab mendefinisikan kontrak umum untuk antarmuka pengguna. Kelas ini tidak menyimpan state permainan, tetapi menyediakan method yang wajib diimplementasikan oleh UI, seperti mengambil command, menampilkan pesan, menampilkan prompt, dan merender state game. Dengan adanya interface ini, game logic tidak perlu mengetahui apakah tampilan yang digunakan adalah CLI atau implementasi UI lain.

Method `getCommand()` digunakan oleh `GameEngine` untuk mengambil input mentah dari pemain. Method `showMessage()`, `showInputPrompt()`, dan `showException()` digunakan untuk mengirim output dari core ke UI. Method render seperti `renderBoard()`, `renderPlayer()`, `renderProperty()`, `renderDice()`, `renderLog()`, `renderSkillHand()`, `renderAuction()`, `renderBankruptcy()`, dan `renderWinner()` digunakan untuk menampilkan state model kepada pengguna.

Secara relasi, IGUI digunakan oleh `GameEngine`, `TurnManager`, `CommandProcessor`, `AuctionManager`, `BankruptcyManager`, dan `SaveLoadManager`. Kelas ini menjadi batas antara UI layer dan game logic layer sehingga core tetap independen dari implementasi tampilan.

2.5.2. Deskripsi Kelas CLIGUI

Kelas CLIGUI bertanggung jawab menjalankan tampilan terminal untuk mode CLI. Kelas ini mengimplementasikan IGUI dan menangani proses membaca input, mencetak pesan berwarna, menampilkan menu utama, menampilkan papan, menampilkan status pemain, menampilkan kartu, menampilkan log, menampilkan akta, menampilkan proses lelang, serta menampilkan kondisi bangkrut atau pemenang.

Atribut `exitRequested` menyimpan apakah pengguna meminta keluar dari program. Atribut `awaitingInput` menandai kondisi saat CLI sedang menunggu input. Atribut `winnerBannerPrinted` mencegah banner pemenang dicetak berulang. Dalam proses render papan, CLIGUI memakai objek bantu `CellInfo` agar data setiap petak dapat disusun terlebih dahulu sebelum dicetak.

Method `getCommand()` membaca command dari terminal. Method `showMessage()` memformat pesan berdasarkan tone pesan. Method `renderBoard()` membaca `Game`, `Board`, `Tile`, `Property`, dan `Player` untuk membangun tampilan papan. Method `renderPlayer()` menampilkan statistik pemain aktif. Method `renderSkillHand()` menampilkan daftar kartu pemain. Method `renderLog()` menampilkan riwayat aksi dari `TransactionLogger`.

Secara relasi, CLIGUI berada di UI layer dan hanya bertanggung jawab terhadap input/output. Kelas ini tidak menentukan aturan sewa, pajak, lelang, bangkrut, kartu, atau perpindahan pemain. Seluruh aturan tersebut tetap dikerjakan oleh game logic layer.

2.5.3. Deskripsi Kelas CellInfo

Kelas CellInfo bertanggung jawab menyimpan data sementara untuk satu petak pada tampilan papan CLI. Kelas ini menyimpan label petak, tag pemilik, token pemain yang berada pada petak, warna petak, dan informasi khusus seperti status petak penjara.

Kelas ini digunakan ketika CLIGUI::renderBoard() membangun tampilan papan. Data dari Tile, Property, dan Player dikumpulkan ke dalam CellInfo, lalu CLIGUI mengubahnya menjadi teks yang ditampilkan di terminal.

Secara relasi, CellInfo hanya berhubungan dengan CLIGUI dan tidak mengubah state permainan. Kelas ini merupakan helper tampilan, bukan bagian dari aturan game.

2.5.4. Deskripsi Kelas Formatter

Kelas Formatter bertanggung jawab menyediakan format teks yang konsisten, terutama format uang. Kelas ini tidak memiliki state dan digunakan sebagai utility.

Method money() menerima nilai integer dan mengubahnya menjadi format mata uang yang mudah dibaca oleh pengguna. Dengan memusatkan format uang di satu kelas, tampilan saldo, harga beli, sewa, pajak, biaya bangunan, gadai, tebus, dan nominal lelang menjadi konsisten.

Secara relasi, Formatter digunakan oleh kelas tampilan dan output command. Kelas ini tidak bergantung pada model atau core logic.

2.5.5. Deskripsi Kelas MessageTone

Enum MessageTone bertanggung jawab mengelompokkan tone pesan pada CLI. Nilai enum ini digunakan untuk membedakan pesan informasi, keberhasilan, aksi, peringatan, dan kesalahan.

Enum ini membantu CLIGUI menentukan warna atau simbol yang sesuai ketika mencetak pesan. Dengan pemisahan tone, wording dan visual feedback pada CLI menjadi lebih konsisten.

Secara relasi, MessageTone hanya digunakan oleh CLIGUI sebagai bagian dari formatting output.

3. Justifikasi

Arsitektur yang digunakan pada pengembangan Nimonspoli mengadopsi pendekatan Layered Architecture yang dipadukan dengan prinsip Model-View-Controller (MVC). Pemilihan desain ini didasarkan pada kebutuhan sistem permainan yang memiliki beberapa tanggung jawab besar, yaitu pengelolaan state permainan, pemrosesan aturan permainan, interaksi dengan pengguna, serta penyimpanan dan pemuatan data. Dengan membagi sistem ke dalam beberapa lapisan, setiap bagian program dapat dikembangkan, diuji, dan diperbaiki secara lebih terpisah.

Secara umum, sistem dibagi ke dalam beberapa lapisan utama, yaitu model, core/game logic, view/user interaction, dan data utility. Lapisan model merepresentasikan objek-objek utama dalam permainan seperti Player, Board, Tile, Property, Card, dan turunannya. Lapisan core berisi logika utama permainan, seperti GameEngine, TurnManager, CommandProcessor, AuctionManager, BankruptcyManager, SaveLoadManager, dan DiceManager. Lapisan view bertanggung jawab terhadap tampilan dan interaksi pengguna melalui abstraksi IGUI, yang kemudian diimplementasikan oleh kelas konkret seperti CLIGUI untuk antarmuka terminal dan GUI untuk antarmuka grafis. Sementara itu, lapisan data utility menangani konfigurasi, log transaksi, dan kebutuhan penyimpanan data.

Desain ini dipilih karena permainan Nimonspoli memiliki banyak mekanisme yang saling berhubungan, seperti giliran pemain, perpindahan di papan, pembelian properti, pembayaran sewa, penggunaan kartu, lelang, pajak, penjara, hingga kondisi bangkrut. Jika seluruh logika tersebut diletakkan dalam satu kelas besar, sistem akan sulit dipahami dan sulit dikembangkan. Oleh karena itu, tanggung jawab dipecah ke dalam beberapa manager. TurnManager bertugas mengatur fase dan validasi giliran, CommandProcessor memproses perintah pemain, AuctionManager menangani mekanisme lelang, BankruptcyManager menangani kondisi kekurangan uang dan kebangkrutan, sedangkan SaveLoadManager menangani proses penyimpanan dan pemuatan state permainan. GameEngine berperan sebagai koordinator utama atau facade yang menghubungkan manager-manager tersebut tanpa harus mengambil alih seluruh detail implementasinya.

Berbeda dengan rancangan pada milestone 1, desain kelas mengalami perubahan karena kebutuhan permainan menjadi lebih konkret setelah implementasi berjalan. Pada milestone 1, beberapa tanggung jawab masih bersifat umum dan belum sepenuhnya dipisahkan. Selain itu, bagian antarmuka dianggap sebagai bagian yang langsung terhubung dengan alur permainan, tetapi pada desain ini menggunakan interface IGUI agar GameEngine tidak bergantung langsung pada implementasi tampilan tertentu.

Arsitektur ini memiliki kelebihan dari desain diagram kelas yang menjadi lebih modular dan mudah diperluas. Penambahan jenis tile baru, kartu baru, properti baru, atau tampilan baru dapat dilakukan dengan mengikuti hirarki kelas yang sudah tersedia. Namun, desain ini memiliki kekurangan dimana jumlah kelas menjadi cukup banyak sehingga diagram kelas keseluruhan menjadi kompleks dan sulit dibaca jika ditampilkan dalam satu diagram besar. Selain itu Beberapa manager juga memiliki dependency ke banyak kelas karena harus mengoordinasikan banyak mekanisme permainan. Hal ini merupakan konsekuensi dari kompleksitas domain permainan yang cukup besar.

Kendala utama dalam pemilihan desain OOP ini adalah menentukan batas tanggung jawab antar kelas. Beberapa fitur yang ada, seperti, pembayaran sewa, efek kartu, dan lainnya, melibatkan banyak objek sekaligus sehingga tidak selalu jelas apakah logika tersebut dilakukan pada kelas tertentu. Selain itu, pembatasan pada salah satu kelas agar tidak menjadi *god class*, seperti GameEngine, CommandProcessor, dan beberapa manager menjadi salah satu tantangan dalam menentukan batasan yang dipunyainya.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

Pada program ini, konsep inheritance dan polymorphism digunakan untuk menyusun hubungan antar kelas yang masih berada dalam satu kelompok besar, tetapi memiliki fungsi yang berbeda-beda. Contoh sederhananya dapat dilihat pada hierarki kendaraan. Misalnya terdapat kelas induk Kendaraan yang dapat diturunkan menjadi Mobil, Motor, dan Bus. Walaupun masih berada dalam satu kelompok yang sama, masing-masing turunan tetap memiliki fungsi yang lebih spesifik. Prinsip yang sama digunakan dalam program ini. Dengan inheritance, bagian yang bersifat umum cukup ditulis pada kelas induk, sedangkan kelas turunan menangani detail yang lebih khusus. Sementara itu, polymorphism memungkinkan objek-objek turunan tersebut tetap diperlakukan sebagai bagian dari kelompok yang sama, tetapi saat digunakan akan tetap menunjukkan perilaku sesuai jenisnya masing-masing.

Penerapannya dapat dilihat pada hierarki kelas Tile. Dalam permainan ini, semua petak pada papan memiliki beberapa kesamaan dasar, seperti memiliki posisi, kode, nama, warna, serta memberikan reaksi tertentu saat pemain berinteraksi dengannya. Karena itu, dibuat kelas Tile sebagai kelas induk yang menampung atribut dan perilaku umum tersebut. Selanjutnya, kelas ini diturunkan menjadi beberapa kelompok besar seperti PropertyTile, ActionTile, dan SpecialTile, lalu masing-masing kelompok diturunkan lagi menjadi bentuk yang lebih spesifik seperti StreetTile, RailroadTile, UtilityTile, ChanceTile, CommunityChestTile, FestivalTile, GoTile, JailTile, FreeParkingTile, dan GoToJailTile.

Adanya turunan-turunan ini diperlukan karena meskipun seluruh petak sama-sama merupakan bagian dari papan permainan, setiap jenis petak memiliki aturan dan respons yang berbeda ketika pemain mendarat di atasnya. Sebagai contoh, StreetTile perlu menangani pembelian atau pembayaran sewa, ChanceTile perlu mengambil dan menjalankan kartu, sedangkan GoToJailTile harus langsung memindahkan pemain ke penjara. Jika seluruh perilaku tersebut dipusatkan dalam satu kelas yang sama, maka kelas Tile akan menjadi terlalu besar, penuh percabangan, dan sulit dikembangkan.

Berikut adalah cuplikan kode Tile (tidak lengkap) beserta contoh anakannya:

```
class Tile {  
protected:  
    std::string code;  
    std::string name;
```

```

public:
    Tile(const std::string& code, const std::string& name)
        : code(code), name(name) {}

    std::string getCode() const {
        return code;
    }

    virtual void onLanded(Player& player, Game& game) = 0;
};

```

```

class StreetTile : public Tile {
private:
    Property* property;

public:
    StreetTile(const std::string& code, const std::string& name, Property* property)
        : Tile(code, name), property(property) {}

    Property* getProperty() const {
        return property;
    }

    void onLanded(Player& player, Game& game) override {
        // logika saat pemain mendarat di street
    }
};

```

```

class ChanceTile : public Tile {
private:

```

```

    std::string deckType;

public:
    ChanceTile(const std::string& code, const std::string& name)
        : Tile(code, name), deckType("Chance") {}

    std::string getDeckType() const {
        return deckType;
    }

    void onLanded(Player& player, Game& game) override {
        // logika mengambil kartu chance
    }
};

```

Dari contoh cuplikan ini, Tile sebagai kelas parent memiliki atribut code dan name, serta method getCode(). Bagian ini diwarisi oleh StreetTile maupun ChanceTile, sehingga kedua kelas tersebut tidak perlu menulis ulang atribut dasar yang sama. Ini menunjukkan fungsi inheritance dalam menghindari pengulangan kode.

Lalu, masing-masing child juga dapat menambahkan atribut dan method sendiri. StreetTile menambahkan atribut property dan method getProperty() karena tile jenis ini perlu terhubung dengan objek properti. Sementara itu, ChanceTile menambahkan atribut deckType dan method getDeckType() karena tile ini berkaitan dengan pengambilan kartu.

Bagian yang di-override terlihat pada method onLanded(). Pada parent dinyatakan bahwa semua tile harus punya perilaku saat pemain mendarat, tetapi tidak menentukan isi perilakunya secara langsung. Isi method tersebut lalu ditentukan ulang oleh tiap kelas turunan sesuai kebutuhannya. Pada StreetTile, onLanded() dipakai untuk menjalankan logika yang berhubungan dengan properti, seperti pembelian atau pembayaran sewa. Pada ChanceTile, onLanded() dipakai untuk mengambil kartu dan mengeksekusi efeknya.

Selain pada Tile, konsep inheritance dan polymorphism juga diterapkan pada hierarki Property. Berbeda dengan Tile, kelas Property tidak merepresentasikan lokasi pada papan, tetapi merepresentasikan aset yang dapat dimiliki pemain. Kelas ini menyimpan informasi umum

seperti kode properti, nama, jenis properti, harga beli, nilai gadai, pemilik, status kepemilikan, serta efek festival. Dari kelas Property kemudian diturunkan StreetProperty, RailroadProperty, dan UtilityProperty.

Berikut adalah cuplikan kode Property(tidak lengkap) beserta contoh anaknya:

```
class Property {
protected:
    std::string code;
    int purchasePrice;

public:
    Property(const std::string& code, int purchasePrice)
        : code(code), purchasePrice(purchasePrice) {}

    int getPurchasePrice() const {
        return purchasePrice;
    }

    virtual int calculateRent() const = 0;
};
```

```
class StreetProperty : public Property {
private:
    std::string colorGroup;

public:
    StreetProperty(const std::string& code, int purchasePrice, const std::string& colorGroup)
        : Property(code, purchasePrice), colorGroup(colorGroup) {}

    std::string getColorGroup() const {
        return colorGroup;
    }
}
```

```

    int calculateRent() const override {
        // hitung sewa street berdasarkan bangunan / monopoly
    }
};

```

```

class RailroadProperty : public Property {
private:
    int railroadCount;

public:
    RailroadProperty(const std::string& code, int purchasePrice, int railroadCount)
        : Property(code, purchasePrice), railroadCount(railroadCount) {}

    int getRailroadCount() const {
        return railroadCount;
    }

    int calculateRent() const override {
        // hitung sewa railroad berdasarkan jumlah railroad
    }
};

```

Dari contoh cuplikan ini, Property sebagai kelas parent memiliki atribut code dan purchasePrice, serta method getPurchasePrice(). Bagian ini diwarisi oleh StreetProperty dan RailroadProperty, sehingga kedua kelas tersebut tidak perlu menulis ulang atribut dasar yang sama.

Lalu, masing-masing child juga dapat menambahkan atribut dan method sendiri. StreetProperty menambahkan atribut colorGroup dan method getColorGroup() karena properti jenis street memang perlu mengetahui kelompok warnanya. Sementara itu, RailroadProperty menambahkan atribut railroadCount dan method getRailroadCount() karena besar sewanya bergantung pada jumlah railroad yang dimiliki. Sama seperti Tile, hal ini menunjukkan bahwa inheritance bukan hanya soal mewarisi, tetapi juga memberi ruang bagi kelas turunan untuk berkembang sesuai kebutuhan objeknya.

Bagian yang di-override terlihat pada method `calculateRent()`. Pada parent, method ini hanya dideklarasikan sebagai kontrak umum bahwa setiap jenis properti harus bisa menghitung nilai sewanya. Namun, cara menghitung sewa untuk setiap properti jelas tidak sama. Karena itu, `StreetProperty` membuat implementasi sendiri yang bisa bergantung pada bangunan atau monopoly, sedangkan `RailroadProperty` membuat implementasi yang bergantung pada jumlah railroad. Jadi, nama method-nya tetap sama, tetapi perilakunya berbeda. Sama seperti `Tile`, hal ini menunjukkan penerapan polymorphism, karena program dapat memanggil method yang sama pada objek bertipe umum `Tile`, tetapi hasil yang dijalankan tetap menyesuaikan jenis objek sebenarnya.

Secara keseluruhan, penggunaan inheritance dan polymorphism pada hierarki `Tile` dan `Property` sesuai dengan kebutuhan program karena keduanya sama-sama terdiri atas objek-objek yang masih berada dalam satu kelompok besar, tetapi memiliki perilaku yang berbeda-beda. Dengan inheritance, bagian yang bersifat umum cukup diletakkan pada kelas induk sehingga tidak perlu ditulis ulang pada setiap kelas turunan. Sementara itu, polymorphism membuat program dapat memproses objek-objek tersebut melalui antarmuka yang sama, tetapi tetap memperoleh perilaku yang sesuai dengan jenis objek sebenarnya. Keuntungan dari pendekatan ini adalah struktur program menjadi lebih rapi, pengulangan kode dapat dikurangi, dan pengembangan fitur baru menjadi lebih mudah dilakukan.

4.2. Method/Operator Overloading

Pada program ini, konsep overloading digunakan agar satu bentuk operasi dapat dipakai dalam lebih dari satu cara, tetapi tetap berada dalam konteks yang sama. Ada dua bentuk overloading, yaitu method overloading dan operator overloading. Method overloading terjadi ketika beberapa method memiliki nama yang sama, tetapi dibedakan oleh jenis atau jumlah parameternya. Dengan cara ini, program dapat memiliki beberapa variasi untuk satu operasi yang sama tanpa harus membuat nama method yang berbeda-beda. Sementara itu, operator overloading digunakan ketika operator bawaan C++, seperti `+=`, `-=`, `>`, atau `<`, diberi makna khusus agar dapat bekerja secara pas pada objek buatan program. Dengan begitu, interaksi terhadap objek menjadi lebih ringkas dan lebih mudah dibaca.

Penerapan method overloading dapat dilihat pada kelas `Board`, khususnya pada method `getTile()`. Dalam permainan ini, sebuah petak pada papan kadang perlu diakses berdasarkan indeks, misalnya saat pemain berpindah posisi, tetapi pada kondisi lain petak juga perlu dicari berdasarkan kode, misalnya saat pemain memasukkan kode tile tertentu pada perintah tertentu. Karena dua kebutuhan ini sama-sama merupakan operasi “mengambil tile”, maka nama method yang dipakai tetap sama, tetapi parameter yang diterima dibedakan.

Berikut adalah cuplikan kode `Board` (tidak lengkap):

```
class Board {  
public:
```

```

    Tile* getTile(int index) const;
    Tile* getTile(const std::string& code) const;
};

```

Dari contoh cuplikan tersebut, terlihat bahwa Board memiliki dua method dengan nama yang sama, yaitu `getTile()`, tetapi dengan parameter yang berbeda. Versi pertama menerima `int index`, sedangkan versi kedua menerima `const std::string& code`. Inilah yang dimaksud dengan method overloading.

Penggunaan method overloading pada kasus ini terjadi karena operasi yang dilakukan sebenarnya masih sama, yaitu mengambil tile dari papan, hanya saja cara mengaksesnya berbeda. Dengan pendekatan ini, penamaan method tetap konsisten dan tidak perlu dipecah menjadi nama yang berbeda seperti `getTileByIndex()` dan `getTileByCode()`. Keuntungannya, antarmuka kelas bisa menjadi lebih rapi, lebih mudah dipahami, dan lebih nyaman digunakan karena programmer cukup mengingat satu nama method untuk satu jenis operasi utama.

Selain method overloading, program ini juga menerapkan operator overloading pada kelas Player. Pada permainan ini, objek Player sering mengalami perubahan saldo, baik saat menerima uang maupun saat membayar sesuatu. Selain itu, pada beberapa kondisi pemain juga perlu dibandingkan, misalnya berdasarkan total kekayaannya. Karena operasi-operasi tersebut cukup sering muncul dalam logika permainan, operator overloading digunakan agar penulisannya menjadi lebih ringkas dan lebih dekat dengan makna yang ingin disampaikan.

Berikut adalah cuplikan kode Player (tidak lengkap):

```

class Player {
public:
    Player& operator+=(int amount);
    Player& operator-=(int amount);
    bool operator>(const Player& other) const;
    bool operator<(const Player& other) const;
};

```

```

Player& Player::operator+=(int amount) {

```

```

    addMoney(amount);
    return *this;
}

Player& Player::operator==(int amount) {
    deductMoney(amount);
    return *this;
}

bool Player::operator>(const Player& other) const {
    return calculateTotalWealth() > other.calculateTotalWealth();
}

bool Player::operator<(const Player& other) const {
    return calculateTotalWealth() < other.calculateTotalWealth();
}

```

Dari contoh cuplikan tersebut, terlihat bahwa operator += dan -= digunakan untuk merepresentasikan perubahan saldo pemain. Walaupun di dalamnya tetap memanggil method seperti addMoney() dan deductMoney(), penggunaan operator membuat penulisannya menjadi lebih singkat dan mudah dibaca. Misalnya, player += 200 akan lebih langsung dipahami sebagai pemain menerima uang, dibandingkan jika seluruh logika selalu ditulis dengan pemanggilan method biasa.

Sementara itu, operator > dan < digunakan untuk membandingkan dua objek Player berdasarkan calculateTotalWealth(). Jadi, perbandingan pemain tidak hanya melihat saldo tunai saja, tetapi total kekayaan yang dimiliki. Ini cocok dengan kebutuhan permainan, terutama pada kondisi penentuan pemenang atau perbandingan kondisi ekonomi antar pemain, karena yang ingin dilihat memang bukan sekadar uang saat ini, tetapi keseluruhan nilai aset yang dimiliki pemain.

Penggunaan operator overloading pada kasus ini sesuai karena operator yang dipilih memang masih memiliki makna yang sejalan dengan perilaku aslinya. Operator += dan -= tetap berhubungan dengan penambahan dan pengurangan, sedangkan > dan < tetap berhubungan dengan perbandingan. Jadi, operator tidak dipakai secara sembarangan, melainkan untuk mewakili operasi yang memang wajar

jika ditulis dalam bentuk tersebut. Keuntungannya, kode menjadi lebih ekspresif, lebih ringkas, dan lebih nyaman dibaca, terutama pada bagian-bagian logika permainan yang sering melakukan perubahan saldo atau perbandingan pemain.

Secara keseluruhan, method overloading dan operator overloading pada program ini digunakan untuk membuat antarmuka program menjadi lebih alami dan mudah digunakan. Method overloading membantu menyediakan beberapa variasi pemanggilan untuk operasi yang sama, sedangkan operator overloading membantu menyederhanakan interaksi terhadap objek yang sering digunakan dalam logika permainan. Dengan penerapan seperti ini, kode menjadi lebih rapi, lebih mudah dipahami, dan tetap sesuai dengan konteks permasalahan yang diselesaikan.

4.3. Template & Generic Classes

Pada program ini, konsep template dan generic class digunakan untuk membuat struktur yang dapat dipakai ulang pada beberapa jenis objek yang masih memiliki pola perilaku yang sama. Secara sederhana, template berguna ketika program memiliki mekanisme yang cara kerjanya serupa, tetapi objek yang ditangani tidak selalu bertipe sama. Daripada membuat kelas yang berbeda-beda untuk setiap tipe, program dapat menuliskan logika umumnya sekali saja, lalu menentukan tipenya saat kelas tersebut digunakan. Dengan cara ini, kode menjadi lebih ringkas dan tidak banyak pengulangan. Pendekatan seperti ini sangat cocok pada program yang memiliki beberapa komponen dengan alur kerja serupa, tetapi isi datanya berbeda, seperti beberapa deck kartu dengan jenis kartu yang tidak sama.

Penerapan konsep ini dapat dilihat pada kelas `CardDeck<T>`. Dalam permainan Nimonspoli, terdapat beberapa jenis deck kartu, yaitu deck Chance, Community Chest, dan Skill Card. Ketiganya sama-sama memiliki perilaku dasar sebagai tumpukan kartu, seperti bisa diisi, diambil kartunya, dikembalikan ke discard pile, lalu dikocok ulang ketika draw pile habis. Walaupun jenis kartu yang disimpan berbeda, mekanisme dasarnya tetap sama. Karena itu, digunakan generic class `CardDeck<T>` agar satu struktur deck dapat dipakai untuk berbagai jenis kartu tanpa perlu membuat implementasi deck yang terpisah-pisah.

Berikut adalah cuplikan kode `CardDeck` (tidak lengkap):

```
template <typename T>
class CardDeck {
private:
    std::vector<T*> drawPile;
    std::vector<T*> discardPile;
```

```
public:
void addCard(T* card);
T* draw();
void discard(T* card);
void reshuffleIfEmpty();
};
```

```
CardDeck<ChanceCard> chanceDeck;
CardDeck<CommunityChestCard> communityDeck;
CardDeck<SkillCard> skillDeck;
```

Dari cuplikan tersebut, terlihat bahwa CardDeck tidak ditulis khusus hanya untuk satu jenis kartu. Kelas ini dibuat dengan parameter tipe T, sehingga tipe kartu yang dikelola bisa ditentukan saat objek deck dibuat. Pada contoh di atas, CardDeck<ChanceCard> digunakan untuk deck ChanceCard, CardDeck<CommunityChestCard> digunakan untuk deck CommunityChestCard, dan CardDeck<SkillCard> digunakan untuk deck SkillCard.

Penggunaan template pada kasus ini berguna karena ketiga deck tersebut memiliki cara kerja dasar yang sama. Semua deck sama-sama berfungsi untuk menyimpan kumpulan kartu dalam drawPile, mengambil kartu dari tumpukan melalui proses draw(), memindahkan kartu yang sudah digunakan ke discardPile melalui discard(), serta melakukan reshuffleIfEmpty() ketika tumpukan utama habis. Jadi, yang membedakan ketiganya bukan alur pengelolaannya, melainkan hanya jenis kartu yang disimpan di dalam deck tersebut.

Jika generic class tidak digunakan, maka program kemungkinan harus membuat kelas seperti ChanceDeck, CommunityDeck, dan SkillDeck secara terpisah, padahal sebagian besar isi method dan mekanismenya akan sangat mirip. Hal ini akan menambah pengulangan kode dan membuat perawatan program menjadi lebih sulit.

Keuntungan dari penggunaan generic class CardDeck<T> adalah implementasi deck menjadi lebih ringkas dan konsisten. Semua mekanisme umum pengelolaan deck cukup ditulis sekali saja, lalu dapat dipakai ulang untuk berbagai jenis kartu. Selain mengurangi pengulangan kode, penggunaan template juga membantu menjaga desain program tetap rapi karena kelas deck memiliki satu tanggung jawab utama, yaitu mengatur alur keluar-masuk kartu di dalam tumpukan, sedangkan jenis kartunya ditentukan melalui parameter template. Dengan

pemisahan seperti ini, logika pengelolaan deck tidak akan tercampur dengan detail jenis kartu tertentu. Hasilnya, struktur program menjadi lebih rapi, lebih mudah dipahami, dan lebih mudah dikembangkan. Jika nanti akan ditambahkan jenis kartu baru, program dapat langsung membuat deck baru dengan tipe yang sesuai tanpa perlu merancang ulang struktur kelas deck dari awal.

Dari contoh tersebut, dapat dilihat bahwa template dan generic class pada `CardDeck<T>` bukan hanya digunakan sebagai formalitas konsep OOP, tetapi memang memberikan manfaat nyata dalam program. Pendekatan ini membuat kode lebih ringkas, lebih konsisten, dan lebih mudah dikembangkan ketika di kemudian hari ada penambahan jenis kartu baru dengan mekanisme deck yang tetap sama.

4.4. Exception

Pada program ini, exception digunakan untuk menangani error yang muncul saat permainan berjalan. Error ini bisa berasal dari input yang tidak valid atau dari aksi pemain yang melanggar aturan permainan. Saat kondisi seperti itu terjadi, program dapat langsung menghentikan proses yang salah dan mengarahkannya ke penanganan error yang sesuai. Dengan begitu, alur utama permainan tidak terlalu penuh oleh banyak pengecekan kondisi.

Pada C++, exception merupakan mekanisme bawaan bahasa yang digunakan untuk menangani error saat program berjalan. Exception ini berbentuk class sehingga bisa dirancang seperti kelas lain pada OOP. Artinya, exception dapat memiliki atribut, method, dan juga dapat diturunkan menjadi beberapa kelas yang lebih spesifik. Hal ini dimanfaatkan pada program ini agar setiap jenis kesalahan memiliki kategori yang jelas. Jadi, program tidak hanya mengetahui bahwa terjadi error, tetapi juga dapat mengetahui error tersebut termasuk jenis apa.

Penerapan konsep ini dapat dilihat pada Kelas `GameException`. Kelas ini adalah anak dari kelas exception dan berperan sebagai kelas dasar untuk berbagai jenis kesalahan. Kelas ini diturunkan lagi menjadi exception yang lebih spesifik seperti `PlayerTurnException`, `PropertyManagementException`, `SkillTurnException`, `InvalidEntryInputException`, dan `InvalidFileException`. Dari sana, masih ada turunan yang lebih khusus lagi, misalnya `InsufficientMoneyException` untuk kondisi uang tidak cukup dan `InvalidDiceNumberException` untuk input dadu yang tidak valid.

Berikut adalah cuplikan kode exception(tidak lengkap):

```
class GameException : public std::exception {
protected:
    std::string errorCode;
    std::string errorMessage;
```

```
class PlayerTurnException : public GameException {
protected:
    Player* player;
```

```
class PropertyManagementException : public PlayerTurnException{
protected:
    Property *property;
```

Dari cuplikan tersebut, terlihat bahwa, GameException memiliki tambahan atribut seperti kode dan pesan error dari kelas exception. Setelah itu, kelas-kelas turunannya dapat menambahkan konteks yang lebih spesifik. Misalnya, PlayerTurnException menambahkan informasi pemain yang terkait dengan error tersebut, sedangkan PropertyManagementException menambahkan informasi property yang terkait dengan error tersebut. Ini menunjukkan bahwa inheritance pada exception membantu program menyimpan informasi error dengan lebih spesifik sehingga lebih sesuai dengan kasus errornya.

Selain hierarki kelasnya, penggunaan exception juga terlihat pada saat program memvalidasi kondisi tertentu. Ketika ditemukan kondisi yang tidak valid, program dapat langsung melempar exception yang sesuai.

Contoh penggunaan exception:

```
if (d1 < 1 || d1 > 6 || d2 < 1 || d2 > 6)
{
    throw InvalidDiceNumberException(d1, d2);
```

```
}
```

```
if (!player.canAfford(rentCost)) {  
    throw InsufficientMoneyException(&player, prop, rentCost);  
}
```

Pada contoh pertama, exception dilempar (throw) ketika nilai dadu berada di luar rentang yang diperbolehkan. Pada contoh kedua, exception dilempar ketika pemain tidak memiliki uang yang cukup untuk melakukan suatu aksi. Dengan ini, program bisa langsung menghentikan operasi yang tidak valid tanpa harus memaksa alur utama terus berjalan dalam kondisi salah.

Exception dapat ditangani menggunakan try-catch. Pada C++, try digunakan untuk menjalankan kode yang berpotensi menimbulkan error, throw digunakan untuk melempar exception, dan catch digunakan untuk menangkap exception yang dilempar dari kode tersebut. Dengan cara ini, program tidak harus langsung berhenti ketika terjadi kesalahan, tetapi masih bisa memberikan respon yang sesuai, misalnya menampilkan pesan error kepada pengguna atau membatalkan aksi yang tidak valid.

Penggunaan exception pada program ini sesuai karena permainan Nimonspoli memiliki cukup banyak aturan dan validasi. Jika semua error hanya ditangani dengan if-else biasa di banyak tempat, maka kode akan cepat menjadi panjang dan sulit dibaca. Dengan exception, kondisi error dapat dipisahkan dari logika utama permainan. Hasilnya, bagian utama program tetap fokus pada jalannya permainan, sedangkan kondisi kesalahan ditangani dengan mekanisme tersendiri.

Keuntungan lainnya adalah error menjadi lebih jelas dan lebih spesifik. Program tidak hanya mengetahui bahwa suatu aksi gagal, tetapi juga mengetahui penyebab kegagalannya. Misalnya, ada perbedaan perilaku yang jelas antara kasus pemain yang gagal karena uang tidak cukup dengan pemain yang gagal karena input yang dimasukkan tidak valid. Hal ini membantu program memberikan pesan yang lebih informatif, sekaligus mempermudah proses pengembangan kode.

Secara keseluruhan, penggunaan exception pada program ini sesuai karena permainan memiliki banyak kondisi khusus yang dapat menyebabkan suatu aksi tidak dapat dijalankan. Selain membantu menangani error, exception yang dirancang sebagai class juga membuat struktur penanganan kesalahan menjadi lebih teratur karena dapat memanfaatkan inheritance seperti kelas-kelas lain dalam OOP. Dengan

pendekatan ini, kode menjadi lebih rapi, lebih mudah dipahami, dan lebih mudah dikembangkan jika di kemudian hari terdapat penambahan aturan atau jenis error baru.

4.5. C++ Standard Template Library

Pada program ini, C++ Standard Template Library (STL) digunakan untuk membantu pengelolaan data yang muncul selama permainan berjalan. STL merupakan kumpulan template bawaan di C++ yang menyediakan berbagai struktur data siap pakai, seperti vector dan map, beserta komponen pendukung lainnya. Dengan adanya STL, program tidak perlu membuat semua struktur data dasar secara manual dari awal. Hal ini membuat kode menjadi lebih praktis, lebih rapi, dan lebih mudah dipahami.

Contoh STL yang jelas digunakan pada program ini adalah vector dan map. Keduanya dipakai karena permainan Nimonspoli memang membutuhkan data yang tersusun berurutan, dan data yang perlu dicari dengan cepat berdasarkan sebuah kunci. Vector akan cocok dipakai untuk menyimpan kumpulan objek dalam urutan tertentu, sedangkan map akan cocok dipakai untuk menyimpan pasangan antara kunci dan nilai.

Penerapan ini dapat dilihat pada kelas Board. Pada kelas ini, seluruh tile pada papan disimpan dalam sebuah vector, lalu kode tile dan objek tile-nya juga disimpan dalam sebuah map.

Berikut adalah cuplikan kode class Board:

```
class Board {  
private:  
    std::vector<Tile*> tiles;  
    std::map<std::string, Tile*> tileByCode;  
};
```

Dari cuplikan kode tersebut, `vector<Tile*> tiles` digunakan untuk menyimpan semua tile sesuai urutan posisinya pada papan. Ini cocok karena papan permainan memang terdiri atas petak-petak yang tersusun berurutan, dan perpindahan pemain juga dihitung berdasarkan posisi. Dengan vector, program bisa lebih mudah mengambil tile pada indeks tertentu, misalnya saat pemain bergerak beberapa langkah setelah melempar dadu.

Untuk `map<std::string, Tile*> tileByCode`, ini digunakan untuk menghubungkan kode tile dengan objek tile yang sesuai. Struktur ini berguna saat program perlu mencari tile berdasarkan kode, misalnya ketika pemain memasukkan kode petak tertentu atau saat sistem perlu melakukan pencarian tile tanpa menelusuri seluruh papan satu per satu. Dengan map, proses pencarian seperti ini menjadi lebih mudah dan lebih jelas.

Penggunaan vector dan map pada bagian ini terasa pas karena keduanya saling melengkapi. vector dipakai untuk kebutuhan data yang berurutan, sedangkan map dipakai untuk kebutuhan pencarian berdasarkan kode. Kalau program hanya memakai vector, maka pencarian tile berdasarkan kode akan lebih merepotkan karena harus dicek satu per satu. Sebaliknya, kalau hanya memakai map, maka pengelolaan urutan tile pada papan akan jadi kurang nyaman.

Keuntungan dari penggunaan STL di sini adalah program menjadi lebih sederhana dari sisi implementasi, tetapi tetap kuat dari sisi fungsi. Programmer tidak perlu membuat sendiri struktur penyimpanan tile atau mekanisme pencarian kode, karena kebutuhan tersebut sudah didukung oleh container STL yang sesuai. Selain itu, penggunaan STL juga membuat arti dari kode lebih mudah dipahami. Saat melihat vector, pembaca langsung tahu bahwa data disimpan dalam bentuk urutan. Saat melihat map, pembaca juga langsung tahu bahwa ada hubungan antara kunci dan nilai.

Selain pada Board, STL juga digunakan di bagian lain program, misalnya untuk menyimpan daftar pemain, entri log, dan data konfigurasi tertentu. Hal ini menunjukkan bahwa STL pada program ini bukan hanya dipakai sebagai pelengkap, tetapi benar-benar membantu jalannya mekanisme permainan.

4.6. Composition

Composition digunakan ketika sebuah kelas memiliki objek lain sebagai bagian internal yang kuat, sehingga objek bagian tersebut dikelola oleh kelas pemiliknya. Contoh penerapan composition dapat dilihat pada CardDeck. CardDeck menyimpan kartu dalam drawPile dan discardPile, lalu menghapus kartu-kartu tersebut pada destructor.

Berikut cuplikan kode:

```
template <typename T>
class CardDeck {
private:
    std::vector<T*> drawPile;
```

```

        std::vector<T*> discardPile;

public:
    ~CardDeck() {
        for (T* card : drawPile) delete card;
        for (T* card : discardPile) delete card;
    }

    void addCard(T* card) {
        drawPile.push_back(card);
    }
};

```

Pada cuplikan tersebut, CardDeck bertanggung jawab mengelola kartu yang berada di dalam deck. Ketika deck dihancurkan, kartu yang masih berada di drawPile dan discardPile juga ikut dihapus. Ini menunjukkan adanya ownership yang lebih kuat dibandingkan sekadar relasi biasa. Penggunaan composition sesuai karena deck memang merupakan kumpulan kartu. Kartu yang berada di deck menjadi bagian dari state deck, baik masih di draw pile maupun sudah masuk discard pile. Dengan desain ini, pengelolaan memori kartu menjadi lebih terpusat dan tidak tersebar di banyak kelas.

Keuntungan dari composition pada kasus ini adalah tanggung jawab pengelolaan kartu menjadi lebih jelas. CardDeck tidak hanya menyimpan kartu, tetapi juga mengatur alur draw, discard, shuffle, dan reshuffle. Hal ini membuat mekanisme deck lebih mudah dipahami dan tidak bercampur dengan logic GameEngine.

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. GUI

Inisialisasi GUI dilakukan pada mainGUI.cpp. File ini membuka window Raylib, memuat asset global seperti model pemain dan font, membaca konfigurasi awal, membangun board, lalu membuat objek GUI sebagai implementasi dari IGUI. Setelah itu objek GUI diberikan ke GameEngine.

```
int main()
{
    SetTraceLogLevel(LOG_NONE);
    SetConfigFlags(FLAG_WINDOW_RESIZABLE);
    InitWindow(SCREEN_W, SCREEN_H, WINDOW_TITLE);
    SetTargetFPS(TARGET_FPS);

    initLoad();

    ConfigLoader *CL = new ConfigLoader("data/config/default");
    GameConfig GC = CL->loadGameConfig();
    Board *b = CL->buildBoard(GC.getProperties(), GC);

    IGUI *gui = new GUI(static_cast<float>(TARGET_FPS), *b);
    GameEngine engine(gui);
    engine.run();

    CloseWindow();

    delete gui;
    return 0;
}
```

Terlihat bahwa mainGUI.cpp tidak menjalankan aturan permainan secara langsung. Tugasnya adalah menyiapkan lingkungan GUI dan dependency awal. Objek GUI dibuat sebagai IGUI*, lalu diberikan ke GameEngine. Ini menunjukkan bahwa GUI digunakan melalui abstraksi, bukan melalui kelas konkret secara langsung.

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Konfigurasi Game	13524015, 13524053, 13524057, 13524059, 13524111	13524015, 13524053, 13524057, 13524059, 13524111
Petak Properti	13524015	13524015
Petak Aksi	13524057	13524057
Petak Spesial	13524057	13524057
Dadu dan Giliran Bermain	13524059	13524053, 13524059
Kepemilikan Properti (Street, Railroad, Utility)	13524015	13524015
Ketentuan Sewa (Owned, Mortgaged)	13524015	13524015
Gadai	13524015	13524015
Peningkatan Properti	13524015	13524015
Lelang Tanah	13524053	13524053
Pajak	13524053	13524053
Festival	13524053	13524053
Kartu Kesempatan dan Dana Umum	13524111	13524111
Kartu Kemampuan Spesial	13524111	13524111
Bangkrut	13524057	13524057

Pengambilalihan Asset	13524057	13524057
Kondisi Game Selesai	13524059	13524059, 13524111
Save/Load	13524059	13524059, 13524111
Transaction Logger	13524059	13524059
Exception dan ExceptionHandling	13524053	13524053
Operasi dan Perintah Game	13524059	13524059, 13524111, 13524015
Bonus: Antarmuka Pengguna Grafis/Graphical User Interface (GUI)	13524015, 13524053, 13524057, 13524111	13524015, 13524053, 13524057, 13524111
Penyusun Laporan	13524015, 13524059, 13524111	13524015, 13524059, 13524111

Lampiran

Lampiran A. Form Asistensi & Kode Diagram Kelas keseluruhan

<https://drive.google.com/drive/folders/1ggcnCd9BCqFOtPv9dRU-9sDaZwHr0Zp-?usp=sharing>